CRATE

User Manual - Version 2.0.0

Visual Effects Asset Management & Browsing for Foundry Nuke

Part I - Getting Started

1. Introduction

Crate is a production-grade visual effects asset management and batch processing toolkit that runs as a native panel inside Foundry Nuke. It is designed for studio deployment with multi-user access control and centralized configuration. Crate gives artists a single interface to browse shared asset collections, import gaussian splats, geometry, textures, plates and more into Nuke. It supports batch conversions across a wide range of formats and object types, all without leaving the compositing application.

1.1 Key Features

Multi-library automated thumbnail browser grid supporting standard files, image sequences, gaussian splats, video, and mixed-type collections. Per-user Cart clipboard persisting across sessions. Market module for browsing team members' carts. Discover browser/ingest module with dual view modes, full file operations, and 13 integrated processing panels that speed-up ingest processes. Templates module for browsable .nk Nuke scripts setups. Camera Database with industry real-world sensor specs and lens simulation. Console log viewer. Settings panel for custom pipe behaviours and full management control. Diagnostic graphical representation module for rapidly identifying asset problems in large infrastructures. Two-tier Curator/Visitor access control with granular permissions. Crash-safe queue files via JSON with os.fsync(). Recoverable deletion through bin folders.

1.2 System Requirements

Foundry Nuke 14 or later is required. PySide2 is used on Nuke 14–15; PySide6 (recommended) is used on Nuke 16 and above. The compatibility layer is automatic, no user configuration needed. USD 3D system features (Camera4, GeoImport, GeoReference, GeoExport) require Nuke 17+.

1.3 Supported Nuke Versions

Nuke 14: Full support via PySide2. Classic 3D system only (ReadGeo2/WriteGeo).

Nuke 15: Full support via PySide2. Classic 3D system only.

Nuke 16: Full support via PySide6. Classic 3D for legacy formats. USD GeoImport/GeoExport available in W-GEO panel.

Nuke 17+: Full support via PySide6. Both Classic and USD 3D systems. USD Camera (Camera4), GeoImport, and GeoReference nodes. ABC import popup offers Classic or USD choice. USD import popup offers GeoImport or GeoReference. Gaussian splats.

3D Models: .obj, .fbx, .stl, .gltf, .glb, .abc, .usd, .usdc, .usda, .usdz (import to Nuke; F3D thumbnails), and more.

Gaussian Splats: .ply, .splat, .spz, .sog, .ksplat, .lcc (SPLAT-C conversion; F3D thumbnails).

Images: .exr, .png, .jpg, .jpeg, .tif, .tiff, .tga, .bmp, .dpx, .hdr, .psd, .webp, .pfm, .pic, .sgi, .gif, .dds (thumbnail generation; IMG-C/SEQ-C conversion; Read node import), and more.

Video: .mp4, .mov, .avi, .mkv, .mxf, .webm, .flv, .wmv, .m4v, .mpg, .mpeg, .m2v, .prores, .dnxhd, .yuv, .y4m (VID-C transcoding; FFmpeg thumbnails), and more.

Nuke Scripts: .nk (paste into node graph; placeholder thumbnails from crate_media/).

1.4 Bin System

Crate never permanently deletes files. All delete operations route through **_Crate_Data** bin folders for safe recovery via the OS file explorer:

Library_Bin: {library_root}/_Crate_Data/Library_Bin/, auto-created per library.

Discover_Bin (Show_Bin): {discover_path}/_Crate_Data/Discover_Bin/, auto-created per Discover project path.

Templates_Bin: {template_path}/_Crate_Data/Templates_Bin/, auto-created per template source.

The **_Crate_Data** folders are hidden from the grid by the default underscore (_) hidden prefix (See custom user “hide prefixes” in “Crate Settings”.)

1.5 Configuration System (Pipe-Delimited `.txt` Files)

All configuration lives in flat text files using “`KEY | value`” pipe-delimited format. This is a deliberate design choice, safe for non-technical users, no Python syntax risk, easy to diff/merge:

`crate_libraries.txt` Asset library definitions (name, path, cache, type) with FOLDER_OVERRIDE and FOLDER_HIDE support

`crate_engines.txt` External engine paths (F3D, ImageMagick, FFmpeg, MediaInfo, OIIO) and ENGINES_BASE

`crate_users.txt` USERS_PATH override

`crate_access_credentials.txt` Curator usernames (one per line; `#PAUSED:` prefix for suspended)

`crate_discover.txt` Discover tree dir branches paths with visitor access control

`crate_templates.txt` Template .nk script collections

`crate_templates_data.txt` Auto-managed hidden template state

`crate_hide_prefixes.txt` Folder/file prefix filters (default “`.`“ and “`_`“)

`crate_open_with.txt` External "Open With" applications

`crate_logs.txt` Log level (basic/dev) and console emoji toggle

`crate_nuke.txt` Nuke menu visibility flags and import backdrop toggle

`crate_update_check.txt` Per-user update check timestamps

`crate_curators_info.txt` Free-text curator contact info

`crate_curators_welcomed.txt` Tracks which curators have seen the welcome dialog

1.6 Code Structure

`menu.py` is the monolith containing the main `ThreeDAssetBrowser` class, thumbnail generation (`ThumbnailCache`), cart system (`CrateCart`), all UI layout, and the Nuke menu registration. Satellite modules are loaded via `importlib`:

- **Conversion panels**:

  `crate_imgc_panel.py` (images)

  `crate_seqc_panel.py` (sequences)

  `crate_vidc_panel.py` (video)

  `crate_splat_panel.py` (Gaussian splats)

  `crate_geo_panel.py` (3D geometry)

  `crate_4mat_panel.py` (reformatter)

  discover script panel (folder with 3 files)

  `crate_script_analyze.py`

  `crate_script_engine.py`

  `crate_script_panel.py`

- **Utility panels**:

  `crate_batch_panel.py` (file renaming)

  `crate_contact_panel.py` (contact sheets)

  `crate_write_panel.py` (EXR Write node panel)

- **Script panel**: `discover script panel/` (bulk Nuke script processing)

- **Analytics**:

  `crate_graphs.py` (storage sunburst + volume donuts)

  `crate_analytics_network.py` (system infrastructure graph)

- **Camera DB**: `crate_camera_database/crate_camdb.py` + `crate_cameras.json`
- **Settings**: `crate_settings/crate_settings_module.py` (GUI settings editor)
- **Locking**: `crate_cache_lock.py` (multi-user cache safety)`crate_manual_lock.py` (curator folder/global locks)
- **Conversion core**: `crate_convert.py` (sequence detection, path builders)

1.7 Module Switching

Crate's modules (Libraries, Cart, Market, Discover, Templates, Camera Database, Settings and System Infrastructure GRAPH) are mutually exclusive, entering one automatically exits any other active module, stashing its scroll position and state for restoration. The active module's toolbar button is highlighted with a gold active state. The path label updates to show the current module name (e.g. "Shopping Cart", "Market", "Camera Database"). Navigation buttons (Back, Forward) are disabled in non-navigable modules like Camera Database and Settings.

1.8 System Security Exclusions (not mandatory)

Ingestion and thumbnail generation are the first real work Crate does for you. During a generation burst, Crate launches external engine executables rapidly and writes hundreds of thumbnail files in seconds. To your operating system's security layers, that pattern can look suspicious, and their interference is the #1 cause of a bad first run: silently blocked engines, throttled generation, or half-finished thumbnails that make Crate look broken when the OS is merely being cautious.

For More Information on **Security Exclusions** or if Crate if not generating thumbnails properly please read the **System Security Exclusions.pdf** appendix on crate dir\crate_licence_info_about

1.9 Deployment Essentials - Set Up Crate Right

Read this before creating your first library, it takes ten minutes and each choice here is made exactly once. Everything *not* in this chapter is a preference you can safely change at any time; the full Settings reference covers those later in the manual.

Crate stores its configuration in plain text files that live next to `menu.py`. You can edit most of them through the Settings module inside Crate, but knowing the files exist - and that they *are* the deployment - is the foundation for everything below.

1.9.1 Choose where Crate lives: solo or team

Crate's configuration files sit beside `menu.py`, so **the install location is the deployment decision.** Install Crate on a shared network plugin path and every artist who loads it sees the same libraries, templates, and users, one deployment, maintained once. Install it locally and it's a personal setup.

Decide this now, because moving later means relocating every configuration file and re-pointing every machine.

For team deployments, three placement rules:

- **Configuration** travels with the install, shared install, shared config. Nothing extra to do.
- **Caches** belong on shared storage, so a thumbnail generated by one artist appears instantly for everyone else. Crate's cache write-lock coordinates machines automatically: it identifies writers by user and machine name, refreshes a heartbeat every 30 seconds while generating, and automatically clears locks left behind by a crashed session after two minutes. No manual cleanup is ever needed, but the staleness detection compares timestamps across machines, so **keep workstations clock-synced (NTP)**. A machine drifting minutes from the others can misjudge a teammate's live lock as stale.
- **Asset paths** must be valid from every workstation. Use UNC paths or identically mapped drives, a library defined as `X:\assets\smoke` only works if `X:` means the same thing on every machine.

1.9.2 Identity is automatic - your one decision is who curates

You never set up your identity in Crate: it manages this automatically from the very start. The moment Crate loads, your identity is your operating-system login name - nothing to configure, no account to create. Ownership of Market cards, edit permissions, and the cache lock's announcements to teammates ("locked by anna@ws-comp-04") all follow your OS login on their own. Crate's user registry (crate_users.txt) is likewise created and maintained automatically - first-run welcomes and user records happen without you touching a file.

The one decision that is yours: who gets Curator access. Crate has two roles - Curators (full control: managing libraries, deleting, regenerating, administering) and Visitors (read-only browsing - the safe default for anyone not explicitly listed). To designate curators, go to “Settings/Access” and add each

person's OS login name, or, do it in crate_access_credentials.txt, not really necessary since the UI way “Settings/Access”is the best recommendation. On a fresh deployment, do this before anything else: until at least one login is listed there, everyone - including you - is a read-only Visitor.

Because identity rides on OS logins, two consequences worth knowing. Your team's naming convention is already decided - it's whatever your IT logins are, automatically consistent across the studio. And the permanence warning now points at IT rather than at Crate: ownership follows the login name, so if an account is ever renamed, everything published under the old name is orphaned - Crate has no "rename user everywhere" operation. Stable logins keep ownership stable.

Second, manual setup option: advanced deployments that want to pre-seed the user registry, relocate it, or manage it by hand can edit crate_users.txt directly - the file is plain text like everything else. This is never required; the automatic path covers normal use entirely.

1.9.3 Install and point to the thumbnail engines

Crate does not decode your assets itself. All heavy lifting is delegated to **six external engines** running as separate processes - which is why a malformed file can never crash Nuke through Crate, and why without an engine, there are no previews for the formats it owns:

Engine	Role
F3D	Thumbnails for 3D formats (geometry, USD, etc.)
ImageMagick	Thumbnails for standard images (JPG, PNG, TIFF, …) and PSD
OIIO (oiiotool)	High-quality thumbnails for film formats - EXR, HDR, DPX
MediaInfo	Metadata extraction for the status bar
FFmpeg	Video previews and best-frame selection - *optional*: without it, Crate gracefully falls back (e.g. a sequence's middle frame) instead of failing

splat-transform (PlayCanvas)	Gaussian-splat conversion - PLY, Compressed PLY, SOG, SPZ, KSPLAT, GLB

Configure them in `crate_engines.txt` (simple `KEY | value` lines): `ENGINES_BASE` points at the folder containing the engines, per-engine path keys (such as `FFMPEG_PATH`) override individual locations when needed, and worker counts such as `F3D_MAX_WORKERS` tune parallelism - the file's own comments document every key, including download links and the expected install layout per platform. One engine installs differently: **splat-transform** is a Node package, installed via npm into `ENGINES_BASE/splat-transform` (the exact command is in the engine header comments and in Settings).

The failure mode to avoid: a missing or wrongly-pathed engine does not produce an error dialog - cards for that format simply sit on placeholders forever, and Crate looks broken when it's merely unequipped. So **verify before your first ingest**: open Settings and confirm each engine reports as detected (Settings also has per-engine test actions), then ingest one test file of each type you care about and watch real thumbnails appear.

On team deployments the engines path must resolve from *every* workstation - either engines on the share alongside Crate, or installed identically on each machine.

1.9.4 Create your first library - and adopt the cache rule

Libraries are defined in `crate_libraries.txt`, one per line, pipe-separated:

name | asset_path | cache_path | visible | enabled | type

`asset_path` is where your assets already live, Crate **never modifies your assets**; it only reads them. `type` tells Crate how to present the folder's contents: `standard` for individual files, `sequence` for image sequences (frame patterns are detected and collapsed into single cards), `video` for movie files with animated hover previews, and `mixed` for folders containing both. A `FOLDER_OVERRIDE | folder_name | type` line placed under a library re-types one subfolder without changing the parent.

`cache_path` is where Crate writes everything it generates, thumbnails, sequence previews, hover sprites, and here lives the single most important rule in this chapter:

One dedicated cache folder per library, named after the library. Never point two libraries at the same cache path.

A clean layout:

Crate Caches/

├── Libraries/

│ ├── Smoke/

│ ├── Fog/

│ ├── Explosions/

│ └── HDRIs/

└── Templates/

├── Keying/

├── Grain/

└── Lens FX/

So the library "Smoke" is defined as:

Smoke | //server/assets/smoke | //server/crate/Crate Caches/Libraries/Smoke | true | true | sequence

Why isolation is non-negotiable, in descending order of pain:

1. **Correctness.** Sequence and video previews are cached in subfolders named after the *asset*, a sequence `fog_####.exr` caches under `fog_exr`. Two libraries sharing a cache folder and each containing a same-named sequence or video will silently overwrite each other's previews and hover sprites. You'll see the wrong thumbnail and never know why.
2. **Team coordination.** The write-lock is one per cache folder. Shared folder = one lock for everything: an artist generating thumbnails in *any* library briefly blocks generation in *all* of them, across all machines. Isolated folders give every library its own lock.
3. **Speed.** Cache folders accumulate files with every ingest. A single shared folder grows into thousands of entries from every library combined, and on network storage every lookup in a fat directory is slower, for everyone, in every library, forever.
4. **Maintenance.** Named per-library folders make cache surgery obvious: to retire a library or force a full thumbnail rebuild, delete exactly one folder and touch nothing else.

Solo artists with local caches: rules 1 and 4 still apply in full. Adopt the layout anyway, it costs nothing today and your deployment may not stay solo.

1.9.5 The same rule for template categories

Template categories (`crate_templates_data.txt`, or the Templates section of Settings) each carry their own `cache_path`, the same per-entry model as libraries, with the same rule: one dedicated folder per category, never shared with another category or with any library. The `Crate Caches/Templates/Keying` branch of the tree above completes the layout. Everything in section 4 -collisions, locks, speed, maintenance - applies identically here.

1.9.6 What you can safely decide later

That's the whole chapter, five decisions. Everything else Crate offers is a *preference*, freely changeable at any time with nothing to redo: the Nuke-menu visitor toggles and import backdrop wrapping (`crate_nuke.txt`), default zoom level, UI and appearance options, per-module behaviors. They're all documented in the Settings reference chapter, and none of them can hurt your deployment.

If the five sections above are done, location chosen, identity set, engines verified, and every library and template category pointing at its own named cache folder, Crate is deployed right, and it will still be right when your library count hits triple digits.

2. Installation & Deployment

2.1 Directory Structure

A clean Crate installation is a single self-contained folder. Everything below ships with the default state. Paths are relative to the Crate root script dir that sits beside `menu.py`.

Core:

menu.py: The main module and entry point. Contains the panel UI, all module views (Libraries, Cart, Market, Discover, Templates, Console, Camera Database, Graph and Settings), the thumbnail/preview engines, access control (Curator vs. Visitor), the recycle-bin and cache routing, and the Nuke menu integration.

Discover Module Tool panels:

Each of Crate's conversion and utility tools lives in its own panel module:

crate_write_panel.py_____W-EXR and W-MOV single/queue write panels.
crate_batch_panel.py_____BATCH file-renaming panel.
crate_seqc_panel.py_____SEQ-C sequence conversion panel.
crate_vidc_panel.py_____VID-C video transcoding panel.
crate_geo_panel.py _____W-GEO geometry conversion panel.
crate_imgc_panel.py_____IMG-C image conversion panel.
crate_contact_panel.py _____CONTACT contact-sheet generator.
crate_4mat_panel.py _____W-4MAT batch reformat panel.
crate_splat_panel.py _____SPLAT-C Gaussian-splat conversion panel.

discover script panel/scripts_____SCRIPT one comp treatment to multi input fan

Conversion & helper engines:

crate_convert.py: The shared file-conversion engine used across the panels.
crate_ply_to_splat.py: Pure-Python converter from Gaussian-splat PLY to `.splat` (no external dependencies; runs on Nuke's built-in Python).
crate_ply_to_ksplat.py: Converter from Gaussian-splat PLY to the `.ksplat` format (mkkellogg GaussianSplats3D).

Infrastructure primitives:

These are small, UI-free support modules used internally:

crate_cache_lock.py: Folder-level cache locking. Writes a `.crate_lock` file inside a cache directory so multiple Crate instances generating thumbnails into the same cache cannot collide. Pure infrastructure - no UI.
crate_manual_lock.py: The Curator-controlled locks: the studio-wide "Lock Crate" mechanism (set from Settings, which blocks all users while you do maintenance) and Curator folder locks placed in a cache directory. Also pure infrastructure.

Subfolders (modules):

crate_settings/: The Settings module. Contains `crate_settings_module.py`, which builds the entire Settings panel.

crate_camera_database/: The Camera Database module. Contains `crate_camdb.py` (browse cameras, view sensor modes, and create Nuke camera nodes with correct film-back aperture values), `crate_cam_export.py` (the multi-format camera/lens exporter, pure-Python writers for various DCC formats), `crate_cameras.json` (the sensor database itself), and `README.txt` (a short description of the folder).

crate_media/: Built-in placeholder imagery. The `placeholders/` subfolder holds the fallback previews Crate uses when an asset has no rendered thumbnail, including the `.nk` “Foundry´s Hobbit” placeholder shown for template scripts and other `.nk` items that have no custom image.

crate_licence_info_about/: Licensing, attribution, and "about" information shipped with Crate. Contains Crate's own `LICENSE` and `NOTICE`, third-party licence texts for the bundled tools it relies on (the splat converters/3dgsconverter, the PlayCanvas splat-transform, and the CrateCamDB camera data), a nested `CrateCamDB/` licence/notice pair, and the informational text files `about.txt`, `donate.txt`, and `what is Crate.txt`.This folder must be edited by CrateTools only or by other developers creating their own Crate internally, but they must always mention CrateTools original creator and developer of the core tool.

users/: The default per-user data root (the `USERS_PATH` default). Ships empty; Crate creates one folder per user inside it at runtime (each holding that user's cart, exported scripts, GUI prefs, `media_cache`, `card_thumbs`, and `profile`). In most studios this is pointed at a network location via Settings/Users instead.

discover script panel/: Part of the SCRIPT panel in the Discover module. SCRIPT is a system for fanning one nuke treatment script to many outputs.

Configuration files (root-level):

Crate's settings live in plain-text files beside `menu.py`. Most are pipe-delimited (`key | value | …`) and edited through the matching Settings section; all are human-readable. The default state ships `.txt` files plus the camera database JSON:

crate_engines.txt: Engine paths and worker limits (Settings/Engines).
crate_access_credentials.txt: The Curator roster, incl. paused entries (Settings/Access).
crate_users.txt: The `USERS_PATH` location (Settings/Users).
crate_libraries.txt: Library definitions (Settings/Libraries).
crate_templates.txt: Template category definitions (Settings/Templates).
crate_templates_data.txt: The hidden-template list that accompanies the above.
crate_discover.txt: Discover project paths, Visitor toggles and allowlist (Settings/Discover).
crate_open_with.txt: External application definitions (Settings/Open With).
crate_hide_prefixes.txt: Folder-name prefixes hidden from the grid (Settings/Hide Prefixes).
crate_logs.txt: Log verbosity and console-emoji toggles (Settings/Logs).
crate_nuke.txt: Nuke menu-integration flags (Settings/Nuke).
crate_curators_info.txt: Free-text Curator metadata (Settings/Curators Info).
crate_curators_welcomed.txt: A small state file tracking which Curators have already seen the first-run welcome (maintained automatically, not edited by hand).

The Camera Database is the one section backed by structured JSON rather than a pipe-delimited file: **crate_camera_database/crate_cameras.json** (Settings/Camera Database).

A note on auto-generated content:

Python creates `__pycache__/` folders (compiled `.pyc` bytecode) next to the modules it runs, you'll see them inside the Crate root and the subfolders after Crate has been launched. These are **not** part of the shipped default and do not need to be distributed; Python regenerates them automatically. A clean default state can safely omit every `__pycache__/` folder. Likewise, per-root `_Crate_Data/` folders (recycle bins and per-location settings) and the contents of `users/` are created at runtime, not shipped.

2.2 Organizing Your Libraries Before You Ingest

One Hour Now, Hundreds Later

Crate will happily browse any folder you point it at. It will generate thumbnails, collapse sequences, and let you search whatever chaos lives on your server. But Crate can only be as clear as the directories underneath it, and the single highest-leverage thing a Curator can do, before installing Crate and exposing a single library to the pipeline, is to spend one focused session organizing those directories by what the assets actually are.

This chapter is a recommendation, not a requirement. Crate imposes no folder rules. But every studio that skips this step ends up doing it later, with ten times more files and twenty artists mid-shot.

The Principle: Typify, Don't Just Store

An asset library is not a backup, it is a menu. Artists don't browse a library to admire your files; they arrive with a need in their head ("I need smoke drifting left, camera-side") and they will find it, or not, in about fifteen seconds before they give up and re-render something that already existed.

The difference between finding and not finding is almost never the asset. It is the folder.

Front Smoke is not the same asset as Side Smoke Rising, Debris Falling is not Exploding Debris. A crackling fireplace loop is not a gas jet. If those distinctions matter to the artist choosing the element, and they always do, they should exist as folders, not as knowledge trapped in one Curator's memory.

The One-Level Rule

You do not need a taxonomy PhD. Aim for at least one level of meaningful sub-organization inside each category, and rarely more than two.

A flat dump is too little:

Smoke/

smk_001.exr … smk_412.exr ← 400 sequences, one folder, zero meaning

This is too much:

Smoke/Exterior/Day/Backlit/Thick/Slow/Left_To_Right/ ← nobody will ever click seven levels deep

This is right

Smoke/

Front Smoke/

Side Smoke Rising/

Smoke Plumes Distant/

Wisps and Tendrils/

One glance at the tree answers the artist's first question, “what kind?” , and the thumbnails answer the second. Over-nesting hides assets just as effectively as under-nesting; deep trees also mean deeper cache paths and more clicking for everyone, every day, forever.

Split by the Characteristics That Drive the Choice

Which characteristic earns a folder? The one an artist would actually use to “reject” an asset. Typical examples per element family:

- Smoke / Fog: direction and behavior - Front, Side Rising, Rolling, Ambient Haze.

- Debris: motion type - Falling, Exploding Outward, Trickling, Ground Impact.

- Fire: scale and behavior - Torch, Wall of Flame, Licks, Embers Only.

- Textures: surface family - Concrete, Metal Painted, Metal Bare, Fabric.

- 3D objects: what it is, then variant - Electronics/Motherboard/, Props/Guns/.

Resolution, frame rate, and codec generally do **not** earn folders, Crate's cards and metadata already surface those. Folders are for meaning; metadata is for specs.

Why This Pays Off Specifically in Crate

Crate rewards a well-typed tree several times over:

1. The tree is the interface. Crate's browser shows your directory structure directly. A meaningful tree means artists navigate by intent instead of scrolling a thousand cards.

2. Search gets sharper. Crate's filter matches names, and folder discipline naturally produces disciplined file names. "side smoke" finds something only if those words exist somewhere.

3. Caches stay manageable. Each library keeps an isolated thumbnail cache. Reorganizing folders **after** ingest means regenerated thumbnails and re-learned locations; organizing **first** means you generate once.

4. Curation stays delegable. Folder colors, Discover branches, and Cart sharing all reference paths. Stable, self-describing paths mean another Curator can maintain the library without a guided tour.

5. Hidden housekeeping stays hidden. Anything prefixed with `.` or `_` is invisible to artists (see “crate_hide_prefixes.txt”), so keep your WIP and staging folders prefixed and your public tree purely meaningful.

A Curator's Pre-Ingest Checklist

Before pointing a new library at Crate:

1. List the 5–15 categories the library really contains.

2. Inside each, split once by the characteristic artists choose by.

3. Name folders in plain, searchable words - "Side Smoke Rising", not "smk_sd_r_v2".

4. Prefix anything artists shouldn't see with **“_”** or **“.”**.

5. Only then add the library in “Settings/Libraries” and let Crate build its cache.

The first time an artist finds “Side Smoke Rising” in eight seconds instead of asking in the channel, re-rendering, or settling for “Front Smoke” flipped and hoping - this chapter has paid for itself.

It will keep paying every day the library exists.

Shared Cache Folder Configuration Recommendation:

For tiny setups, single freelancers or small teams of up to five users working with a very small library, it is technically possible to use a single shared cache folder for all libraries.

However, once you scale to hundreds of libraries, each containing hundreds of subfolders and files and maintaining its own cache on disk, assigning a separate cache path to each library is strongly recommended. We think that even if you are a solo artist using a small library, you should also keep an independent cache folder per each library you setup.

This separation is required for three reasons, listed in order of importance:

1. Correctness

Sequence and video cache subfolders are keyed only by the clean name of the media (for example, `fog_####.exr` becomes a subfolder named `fog_exr`; a video becomes a subfolder named after its basename). No path hash is included.

When multiple libraries share the same cache folder, two libraries that contain a sequence or video with the same name will write to the identical subfolder and overwrite each other's thumbnails and hover sprites. This is the cause of incorrect thumbnails appearing on sequence cards.

2. Locking

Each cache folder uses a single **`.crate_lock`** file. A shared folder therefore causes generation activity in any one library to block generation in every other library on every machine, creating unnecessary studio-wide contention.

3. Performance

A shared folder that accumulates PNG files from every library (often reaching thousands of files) slows every SMB existence check.

Per-library cache folders are the intended design. The libraries file includes a `cache_path` column for each library, and the initialization code comments explicitly describe "isolated cache folders." A shared layout works against this architecture at scale.

Organizing your cache folders (Libraries and Templates). Crate keeps a separate cache for every library and every template category you configure, each one has its own cache path setting.

Take advantage of this: create one parent folder for all Crate caches, then a dedicated subfolder per entry, named to match. A clean layout looks like this:

Crate Caches/

├── **Libraries/**

│ ├── Smoke/

│ ├── Fog/

│ ├── Explosions/

│ └── HDRIs/

└── **Templates/**

├── Keying/

├── Grain/

└── CopyCat/

So the library "Smoke" points its cache path at `Crate Caches/Libraries/Smoke`, and the template category "Keying" points at `Crate Caches/Templates/Keying`.

Avoid pointing two libraries or two template categories at the same cache folder.

2.3 Installing in Nuke

Place the CRATE folder in a location accessible to Nuke. Add the folder to Nuke's plugin path via **init.py** and **menu.py**. On launch, Crate auto-detects its script directory, loads all configuration files, determines the user's access mode by matching their OS username against the credentials file, and registers itself in the Nuke menu bar.

Network / Multi-User Deployment

For studio deployment, place CRATE on a shared network drive. Set USERS_PATH in crate_users.txt to a network-accessible directory for per-user data (cart files, GUI preferences). Library cache paths in crate_libraries.txt should point to a shared cache location. Crate uses atomic file writes (write to temp file, os.fsync(), os.replace()) for crash-safe concurrent access from multiple machines. You can do all this from Crate Settings ui.

Engine Installation

All engines are discovered from ENGINES_BASE, a configurable path. The recommended portable folder structure is:

{ENGINES_BASE}/f3d/bin/f3d(.exe)

{ENGINES_BASE}/imagemagick/magick(.exe)

{ENGINES_BASE}/mediainfo/MediaInfo(.exe)

{ENGINES_BASE}/ffmpeg/bin/ffmpeg(.exe)

{ENGINES_BASE}/OpenImageIO/oiiotool(.exe)

{ENGINES_BASE}/splat-transform/

All engines are provided by CrateTools for direct download to simplify installation and implement Crate in minutes both on single or multiple machines on the same network.

You might want to first set up the engine's location on disk via "crate_engines.txt", but it´s also possible to launch Crate first and then set up the engine's location on the Crate Settings module.

Installing engines locally on each workstation is strongly recommended for best performance, subprocess calls load instantly from local disk, engines use local CPU/GPU, and there is no network dependency. However, pointing ENGINES_BASE to a shared network path is a valid deployment strategy too. For studios exploring further centralisation, Crate's engine layer is pure subprocess invocation with no session state, the architecture is compatible with a job-dispatch wrapper or process-farming proxy if your pipeline infrastructure supports it. In this complex type of design it would probably make sense to have the thumbnails cache generated by the engines to live in the same "bare metal" server/machine/vm.

F3D

3D model and Gaussian splat auto thumbnail generation. Formats: .obj, .fbx, .stl, .gltf, .glb, .abc (Ogawa), .usd, .usdc, .splat, .ply, .spz. Also used by the Inspect 3D right-click action to open an interactive 3D viewer. https://f3d.app

ImageMagick

Image thumbnail generation for all raster formats. Also powers the CONTACT panel's montage command for contact sheet generation. The portable Q16-HDRI version is recommended for HDR support. https://imagemagick.org.

FFmpeg / ffprobe

Video transcoding engine for the VID-C panel, right click menu convert operations, library thumb sprites, etc. ffprobe is used for media probing (duration, codec detection, frame count). https://www.gyan.dev/ffmpeg/builds/.

OpenImageIO (oiiotool)

High-fidelity image conversion with OCIO colorspace support. Preferred over ImageMagick for EXR, HDR, and DPX. Supports tiled EXR reading and Cineon log-to-sRGB conversion. Used by IMG-C and SEQ-C panels and many other conversion operations for Libraries module. https://github.com/AcademySoftwareFoundation/OpenImageIO/releases.

MediaInfo

Metadata extraction for the status bar. Displays resolution, bit depth, colorspace, codec, and file size when hovering over or selecting assets. https://mediaarea.net.

splat-transform

Gaussian splat format conversion in the SPLAT-C panel. Uses @playcanvas/splat-transform, a Node.js CLI package. Requires Node.js to run. Crate first looks for a portable Node.js installation inside the engine directory ({ENGINES_BASE}/splat-transform/node-vXX.X.X/node.exe), falling back to the system PATH if not found. Install by running `npm install @playcanvas/splat-transform` inside {ENGINES_BASE}/splat-transform/. The Settings panel Test button (Test Splat Transform) verifies both the package and Node.js availability. Supports conversion between PLY, Compressed PLY, SOG, SPZ, SPLAT, KSPLAT, LCC, and GLB formats with optional NaN/Inf filtering, SH band reduction, and point decimation.

Crate also uses "crate_ply_to_splat.py" present on the script dir to perform .ply to .splat conversions right on the Library and Discover modules.

3. Access Control

3.1 First Launch and Curator Claim

When Crate is freshly installed, crate_access_credentials.txt is empty or missing. The first user to launch Crate sees a Welcome dialog with their OS username displayed. The dialog has a single "Claim as Curator" button. If they claim, their username is appended to the credentials file, Crate is automatically locked (the CrateManualLock.lock_crate() method is called) and only that first Curator user can access Crate and the rest of the network (if any) sees Crate in Nuke menu but can´t access it, and a second confirmation dialog explains the lock state and tells them to unlock via Settings when they consider Crate is ready to be live for the team. If they dismiss (X or Escape), the file is left empty and the dialog re-appears on the next launch. Once any non-comment line exists in the credentials file, the bootstrap never runs again. Note: the first Curator user can also be entered manually in the “crate_access_credentials.txt” if preferred.

3.2 The Two-Tier Role System

On every launch, Crate reads crate_access_credentials.txt and compares each line against getpass.getuser().lower(). A match means CURATOR; no match means VISITOR. The access mode is set once at startup and persists for the session.

3.2.1 Curators

Full administrative access: manage Libraries (add, remove, configure via crate_libraries.txt), manage Discover project folders, manage Templates, access Settings (engine paths, access control, system preferences), add or remove other Curators, lock and unlock Crate, and perform all file operations including deletion (not real disk deletion, but move to a Crate bin). Curators see all UI elements including hidden-from-visitor buttons, the Settings gear icon, and the Lock/Unlock controls, Infrastructure Graphs and more.

3.2.2 Visitors

Read-only by default. Visitors can browse Libraries, import assets to Nuke, and view thumbnails. Additional capabilities are granted by Curators via granular permission flags via Settings.

3.3 Granular Permissions

Discover Access: SHOW_VISITOR in crate_discover.txt. When enabled, Visitors can browse project folders. Further scoped by per-path allowlists.

Templates Access: Controls whether Visitors can browse and import Templates.

File Operations: FILE_OPS_TO_VISITORS flag. When disabled, Visitors cannot rename, create folders, cut/copy/paste, or delete in Discover.

Nuke Menu: Add to Discover VISITOR_MENU_DISCOVER in crate_nuke.txt. Default: true.

Nuke Menu: Add to Cart VISITOR_MENU_CART in crate_nuke.txt. Default: true.

Nuke Menu: Add to Templates VISITOR_MENU_TEMPLATES in crate_nuke.txt. Default: true.

Import Backdrop IMPORT_BACKDROP in crate_nuke.txt. Wraps imported nodes in a labelled backdrop. Default: true.

3.4 Adding and Removing Curators

Existing Curators add new Curators via Settings. Usernames are appended to crate_access_credentials.txt. Curators can be temporarily paused by prefixing their line with #PAUSED: this revokes access without deleting the entry. A paused line still counts as a "claim" so the first-launch bootstrap will not re-trigger.

3.4.1 Pre-Seeded Curator Welcome

When a new Curator username is added to the credentials file by an existing Curator, the new Curator sees a one-time Welcome dialog on their first launch. This is different from the first-launch bootstrap, it is informational only (no lock, no claim). The dialog explains their Curator capabilities and mentions the Lock option. It is marked as seen regardless of whether they click "Got it" or dismiss via X, to avoid nagging. Welcomed usernames are tracked in crate_curators_welcomed.txt (one per line, auto-managed).

3.5 Lock / Unlock Crate

Curators can lock Crate from Settings. When locked, Visitors see a "Crate is locked" message instead of the panel. Curators always have access. Locking is intended for the setup phase, building Libraries, maintenance operations, Discover folders, and Templates before opening to artists. Unlock from Settings when deployment is ready.

3.6 Lock System Internals

Crate uses two kinds of advisory lock files, managed by the CrateManualLock class:

Global lock: File: .crate_global_lock in the “CRATE” directory. Written by LOCK CRATE, removed by UNLOCK CRATE. Contains USER, MACHINE, and TIMESTAMP fields. When present, Visitors see a Crate-styled popup showing who locked it and from which machine.

Folder lock: File: .crate_folder_lock in a library cache directory. Placed by a Curator to announce they are working on that folder, library or sub-library. Other Curators see a lock icon on the folder and can override if needed.

Lock files are plain text with the format: USER=curator_name, MACHINE=workstation, TIMESTAMP=epoch. Locks are advisory and persist until explicitly removed. The Settings panel has per-section editing locks (30-minute auto-expiry) to prevent two Curators from editing the same configuration section simultaneously.

3.7 Camera File Indicator Thumbnail/Placeholder

Files whose name contains the word "camera" and whose extension is one of the geometry or Nuke script formats `.abc`, `.fbx`, `.usd`, `.usdc`, `.usda`, or `.nk` display a gold camera icon on their card

instead of the standard thumbnail. This applies across Libraries, Cart, Market, and Discover (thumbnail view), giving camera assets a distinct visual identity at a glance without requiring the file to be opened or inspected.

The indicator respects a priority order: a custom thumbnail set via the pencil button always takes precedence, and HDF5-format Alembic files keep their conversion notice. Removing a custom thumbnail restores the camera icon automatically.

Part II - Core Modules

4. Libraries

4.1 Overview

Libraries are the default view when Crate opens. They provide a thumbnail grid browser for shared asset collections. Each library is defined in crate_libraries.txt with six pipe-delimited fields: name, asset_path, cache_path, visible, enabled, and type. The first enabled library is shown on launch.

4.2 Library Types

standard: Individual files (models, textures, standalone images). Each file gets its own thumbnail cell in the grid.

sequence: Image sequences. Crate scans filenames for frame-number patterns (digits before the extension), collapses matching files into a single entry showing the sequence name and frame range (e.g. render_0001-0120.exr). Detection groups files by prefix, extension, and padding width. The largest group in a folder is selected as the primary sequence. Supported sequence image extensions: .exr, .dpx, .png, .jpg, .jpeg, .tif, .tiff, .tga, .bmp, .hdr, .sgi, .psd, .pfm, .pic. A minimum of 3 frames is required for sequence detection.

video: Video files. Supported extensions: .mp4, .mov, .avi, .mkv, .mxf, .webm, .flv, .wmv, .m4v, .mpg, .mpeg, .m2v and more. Thumbnails extracted via FFmpeg.

mixed: Combines sequence detection with standard file handling. Both collapsed sequences and individual non-sequence files appear in the same grid.

4.3 Folder Overrides

A FOLDER_OVERRIDE line in crate_libraries.txt (placed after a library definition) changes how a specific subfolder is treated, regardless of the parent library's type. Format: FOLDER_OVERRIDE | folder_name | type. Example: a standard library can have a subfolder treated as sequence type for VFX plate collections.

4.4 Folder Hides

A FOLDER_HIDE line (FOLDER_HIDE | folder_name) hides a specific subfolder from the grid within its parent library. This is in addition to the global hidden-prefix system on Settings/Hide Prefixes.

4.5 Thumbnail Generation

Thumbnails are generated on demand when a library folder is first browsed. The engine priority is:

3D models and Gaussian splats: F3D renders a thumbnail from the 3D viewport.

EXR, HDR, DPX: OpenImageIO (oiiotool) with OCIO colorspace conversion. Falls back to ImageMagick if OIIO is unavailable.

Standard images (PNG, JPG, TGA, TIFF, BMP): ImageMagick.

Video files: FFmpeg extracts a frame.

.nk Nuke scripts: A placeholder thumbnail from crate_media/placeholders/.nk/.

Thumbnails are cached to the library's configured cache_path. The cache uses a hash-based naming scheme for uniqueness. Worker limits per engine (e.g. F3D_MAX_WORKERS, MAGICK_MAX_WORKERS) prevent overloading the system.

Regenerate Thumbnail. Right-clicking a Library or Discover card offers **Regenerate Thumbnail** (a Curator cache operation). It deletes that asset's cached preview and re-renders it from the source through the appropriate engine, F3D for 3D/splats, OIIO/ImageMagick for stills, FFmpeg for video. Reach for it when a source file has changed underneath Crate or a thumbnail rendered incorrectly. It only touches the cache; the source asset is never modified. For 3D and splat assets whose *framing* (not freshness) is the problem, use **Apply F3D screenshot** below instead.

Apply F3D screenshot (fixing a better 3D/Splat thumbnail framing)

Crate auto-generates thumbnails for 3D assets and Gaussian splats by rendering them in F3D and auto-framing the result. For most assets this is fine, but auto-framing can't always pick a flattering angle on splats due to off-centred pivots and or inaccurate gaussian bounding boxes: a splat may come out **off-centre or partly out of frame**, **zoomed in so tightly you can't tell what the asset is**, or shown from an angle that hides the recognisable part. When the generated card simply isn't useful, **Apply F3D screenshot** lets you set the thumbnail by hand from a view you framed yourself.

The idea is straightforward: you open the asset in F3D, compose the shot exactly how you want the card to look, capture it, and tell Crate to use that capture as the thumbnail. The procedure is:

1. Right-click the asset and choose **Inspect 3D** to open it in F3D.
2. Rotate, pan, and zoom until the asset is framed the way you want it to appear on the card.
3. Press **Ctrl+F12** to take a *minimal* screenshot, F3D renders the current view with no grid and no overlays, on a transparent background, which is exactly what you want for a clean thumbnail. (Plain F12 also works but includes the grid and overlays.)
4. Close F3D, then right-click the same asset again and choose **Apply F3D screenshot**.

Crate then locates the screenshot you just took, copies it into that asset's thumbnail cache as the new card image, invalidates the old auto-generated thumbnail, and refreshes the grid. It picks the **most recent** matching screenshot, so if you captured several while framing, the last one wins. Your original screenshot files are never touched - Crate only copies them - so they remain in the F3D folder for re-use or manual cleanup.

One detail makes the matching work: F3D names screenshots after the loaded model, using the pattern `<assetname>_<number>.png` (for example, opening `dragon.ply` produces `dragon_1.png`, `dragon_2.png`, and so on). Crate matches screenshots to the asset by that base name, so always capture while the correct asset is the model loaded in F3D. Crate accepts the common image types F3D can emit (PNG, JPG, TIFF, BMP).

Where F3D saves the screenshot (by operating system). F3D writes captures into an `F3D` subfolder, but the parent location differs per platform. Crate looks in exactly these places:

Windows: your Pictures folder: `%USERPROFILE%\Pictures\F3D`

macOS: your home folder: `~/F3D` (note: this is your home directory, *not* Pictures)

Linux (incl. Rocky Linux): your XDG Pictures directory if one is configured (`$XDG_PICTURES_DIR/F3D`); otherwise it falls back to your home folder, `~/F3D`

If Crate can't find a screenshot for the asset, it will tell you which folder it checked and recap the capture steps, usually the fix is simply that the capture went to a different folder, or the asset name didn't match.

4.6 Navigating Libraries

Library selector dropdown: Switches between configured libraries. Only visible and enabled libraries appear.

Double-click a folder: Navigates into that folder.

Back / Forward buttons: Navigation history, browser-style.

Path label: Shows the current path. Can be used for direct path reference.

4.7 Selection

Click: Select a single asset (deselects others).

Ctrl+Click: Toggle individual assets in/out of selection.

Shift+Click: Select a range from the last selected to the clicked asset.

Select All: Selects all assets in the current folder.

Selected assets display a golden highlight border.

4.8 Importing to Nuke

Two import modes are available via toolbar buttons or context menu:

Import Selected: Creates Read nodes (images/sequences), ReadGeo2 or GeoImport nodes (3D models), or pastes .nk scripts. Nodes are arranged in a 5-column grid layout starting from the current cursor position.

Import set |1001|: Same as Import Selected but appends a TimeOffset node (set to 1000) after each Read. Does not apply TimeOffset to 3D models or .nk scripts.

Nodes are arranged in a 5-column grid layout. The IMPORT_BACKDROP flag in crate_nuke.txt controls whether a backdrop node labelled "Crate Import" is placed behind the imported nodes for visual grouping in the node graph.

When importing .abc files on Nuke 17+, a popup appears once asking: Classic 3D System (creates ReadGeo2) or USD 3D System (creates GeoImport). When importing USD files on Nuke 17+, a popup appears once asking: GeoImport (whole scene import with geometry, lights, cameras, materials) or GeoReference (lightweight reference, ideal for instancing and modular assembly, no path conflicts). Both popups appear once for the entire multi-selection, not per file. If the user cancels either popup, the entire import is aborted. On Nuke versions below 17, .abc files always use ReadGeo2 (Classic) and USD files always use GeoImport.

4.9 Hidden Folder Prefixes

Folders whose names begin with a configured prefix are hidden from the grid. Configured in crate_hide_prefixes.txt. The default prefix is underscore (_), which hides _Crate_Data internal folders.

4.10 Library_Bin

Each library automatically creates a _Crate_Data/Library_Bin/ folder at its root. Delete operations move files here rather than permanently removing them. Recovery or permanent deletion is via the OS file explorer and not Crate.

4.11 Status Bar & MediaInfo

Selecting an asset displays metadata in the status bar: resolution, bit depth, colorspace, codec, file size, and duration (for video files). Without MediaInfo, the status bar shows limited information (filename and file size only).

4.12 Zoom Controls

Three zoom buttons in the toolbar control thumbnail grid density:

– (Zoom Out): Decreases zoom by 0.2 per click. Minimum: 0.5x.

+ (Zoom In): Increases zoom by 0.2 per click. Maximum: 3.0x.

Fit [ ]: Resets to default zoom level (1.8x).

Zoom applies in-place without rebuilding the grid, resizing existing items for smooth interaction.

4.13 Import as Cam

Available in the right-click context menu for .fbx and .abc files only. Creates a Camera2 node in Nuke with the "read from file" checkbox enabled and the file knob set to the selected path. The camera node is labelled "Camera from file" with the filename. This is intended for matchmove and previs camera exports that are standalone camera animation files. Multi-selection is supported, one Camera2 node is created per file.

4.14 View Meta

Opens a resizable Crate-styled dialog displaying comprehensive metadata for the selected asset. The dialog has a gold-bordered header showing the filename and a scrollable body with key-value

rows (gold labels, light grey values, selectable text). Metadata is extracted using available engines in priority order:

EXR/HDR/DPX: OpenImageIO first, then MediaInfo.

Standard images: MediaInfo first, then OpenImageIO fallback.

Video: MediaInfo first, then FFprobe fallback.

3D models / Gaussian splats: MediaInfo for basic file info.

Basic file information is always shown regardless of engines: File name, Path, Size (formatted as bytes/KB/MB/GB), Modified date, and Extension. Each engine's output is shown in its own section with a separator and engine label.

4.15 Main Toolbar Layout

The Crate panel toolbar is arranged in two rows:

Row 1 (top): Library selector dropdown, Go to Library button (returns to library from other modules), Path label (shows current location, Curator-only for security), Camera Database, Lock, System infrastructure GRAPH, Console and Settings... dropdown button.

Row 2 (bottom): Up button, Back button, Forward button, Home button, Refresh button, Hide Textures toggle, Search box, Templates button, Cart button, Market button, Discover button, Zoom Out, Zoom In and Fit buttons.

4.15.1 Settings... Dropdown Menu

The "Settings…" button opens a dropdown menu (flat, no submenus): Open Crate Settings (Curator only), Show Crate Status, Curators Contact Info, then the Engine tests (Test F3D, Learn F3D Shortcuts, Test ImageMagick, Test OpenImageIO, Test FFmpeg, Test MediaInfo, Test Splat Transform), User Manual, About and Donate.

4.15.2 Hide Textures Toggle

Toggles visibility of texture files in the library grid. When active, common texture formats are hidden to declutter the view when browsing for 3D assets only.

4.15.3 Search Box

A live-filtering text field in the toolbar. As you type, the grid filters to show only items matching the search term. The placeholder text changes per module to indicate context: "Search assets..." (Libraries), "Search cart..." (Cart), "Search users..." (Market user list), "Search cart items..." (Market user's cart), "Search files..." (Discover), "Search templates..." (Templates categories), "Search scripts..." (Templates .nk files). The search resets automatically when switching modules.

4.15.4 Show Crate Status

Opens a popup displaying comprehensive diagnostic information: access mode (Curator/Visitor), current OS username, all configured libraries with path status ([OK] or [MISSING]) and the active library highlighted, current browsing path, all six engine paths with exists/not-exists status, Nuke and Python version strings, cache directory path with thumbnail count, active thumbnail generations in progress, and failed generation attempts.

4.16 How Crate Stores Things (overview)

It helps to understand one principle before going further: **Crate never modifies your source assets, and nothing it writes is irreplaceable.** Everything Crate creates on disk is either a preview it can rebuild from the source, or a small piece of per-user personalization. Source files are read-only to Crate, it reads them to display and import, and the only times it ever writes into a source tree are two deliberate, user-initiated publishes (Add to Discover and Add to Templates) and a delete, which moves the file to a recoverable bin rather than erasing it.

Crate keeps its previews and personalization in six distinct stores. They fall into two kinds. **Shared, auto-generated caches** are rendered from the asset and rebuild themselves on demand, so they can be deleted at any time:

- **Library cache:** each library's configured `cache_path`. Auto-rendered thumbnails and hover previews for that library's assets, shared by everyone.
- **Discover scratch:** a temporary folder under the OS temp directory. Quick previews generated while browsing Discover; ephemeral and auto-pruned (roughly a 7-day / 750 MB cap), not persistent across sessions.

The other four are **per-user**, living inside each person's folder under the users root (`<users>/<username>/`):

- **media_cache:** auto-rendered thumbnails for the items in that user's own Cart.
- **card_thumbs:** custom card images that the user has chosen (the pencil-Browse Image).
- **profile:** that user's chosen Market card image, one square image per user.
- **Templates cache:** strictly speaking shared (it sits at each template category's `cache_path`), but like the per-user image stores it holds *chosen* images rather than rendered ones: a Curator sets each template's thumbnail by hand, and with none set the card falls back to the "NK" placeholder.

The distinction matters when you delete a cache. An auto store simply regenerates on the next view. A chosen-image store (card_thumbs, profile, Templates) instead reverts to its fallback, the auto-render, the cart placeholder, or the NK placeholder, until the image is set again.

Two boundaries hold everywhere. Crate **never writes one user's data into another user's folder**, when you browse the Market you read a peer's previews read-only, and a copy only enters your own `media_cache` at the moment you import a card and it becomes yours. And **deletion is always recoverable**: assets removed from a Library, Discover project, or Templates source are moved to that location's `_Crate_Data` bin — `Library_Bin`, `Discover_Bin`, or `Templates_Bin`, from which they can be recovered, or permanently removed, only through the OS file explorer.

In short, everything outside your source folders is safe to delete: caches rebuild, chosen images revert to placeholders, and bins are recoverable. For a complete map of where each store lives on disk, what each role may touch, and exactly what every kind of deletion affects, see the

Crate Storage Architecture appendix pdf.

5. Cart

5.1 Overview

The Cart is a per-user collection clipboard. Artists add assets from any Library, Discover folder, or the Nuke node graph. The Cart persists across sessions in a user-specific **cart.txt** file in the USERS_PATH directory.Items inside a cart are called cards.

5.2 Adding Assets

From Libraries: Right-click an asset and select Add to Cart, or select multiple and Add N to Cart.

From Discover: Same right-click Add to Cart action.

From Nuke: Use the Nuke menu bar Crate > Add to Cart. Adds the selected node's file path.

From Market: Right-click on another user's Cart item and select Add to My Cart.

5.3 Cart Persistence

The Cart is saved to {USERS_PATH}/{username}/cart.txt. It is loaded on launch and saved on every add/remove operation. The file stores one file path card per line.

5.4 Importing from Cart

"Import Selected" and "Import set |1001|" work identically to Libraries. The same .abc and USD popups apply when multi-selecting geometry files.

5.5 Right-Click Context Menu

Import to Nuke: Import selected items to the node graph.

Import set |1001|: Import with TimeOffset(1000) appended.

Import as Cam: Import camera-compatible files as Camera nodes.

Open With…: Submenu of configured external applications.

Remove from Cart: Remove selected item(s) from the Cart (this is not a deletion).

Clear Cart (N items): Remove all items from the Cart (this is not a deletion).

View Meta: Display metadata for the selected asset.

A note on thumbnails and per-user caches. Browsing the Market never copies anything: while you view another user's cart, Crate reads each card's thumbnail directly from *that* user's cache, read-only. A second thumbnail cache copy only happens at the moment you **import** a card into your own cart, because the item then becomes yours and Crate caches owned items in the owner's own user folder called "user/media_cache". For video and image-sequence cards this is visible as a brief regeneration, the best-frame thumbnail and its hover sprite are rebuilt into your `media_cache`, so the same preview now exists in both your folder and the original owner's. This duplication is by design.

Each user's folder is deliberately self-contained, which is what lets a user be archived and restored as a single unit, lets removing a cart item clean only your own caches, and guarantees your cards never break if another user later clears their cart or items, and it upholds the rule that Crate never writes into another user's folder. The extra space is modest and bounded: assets that come from a Library use the shared central library cache and are not duplicated at all, so per-user copies only occur for owned items that have no shared source cache, such as those pulled from Discover or exported from Nuke.

5.6 Custom Card Thumbnails

Any card in your Cart can carry a custom image of your choosing, instead of the auto-generated thumbnail. Each Cart card has a small golden pencil (✎) button in its top-right corner; clicking it opens a Crate-styled dropdown with **Browse Image** (or **Add Image** if the card has none yet) and **Remove Image**.

Browse Image lets you pick any image file; Crate scales it to a centred square and stores it in your personal `card_thumbs` folder, keyed to that asset's path. The card updates immediately. **Remove Image** deletes your custom image and the card reverts to its normal auto-rendered thumbnail.

This is a per-user personalization: the image lives in *your* user folder and only affects *your* view of the card. It also never touches the source asset or any shared library cache. For a **video or image-sequence** card, the custom image becomes the card's resting frame (replacing the auto-picked "best frame"), while the hover sprite animation still plays on mouse-over, so you get a chosen poster image at rest and the motion preview on hover, the same way these cards behave in the Market.

Because these custom images are deliberately chosen rather than rendered, they persist independently of the auto-thumbnail system: clearing a library or media cache never removes them, and only **Remove Image** (or removing the card from your Cart) clears one.

6. Market

6.1 Overview

Market lets team members browse each other's Carts. Every user who has a saved cart.txt appears in the Market view. Market uses a two-level navigation:

Level 1 - User List: A grid of user Carts. Each Cart shows drawn icon (with item count badge), the username, and "(you)" for the current user. Carts follow the same zoom scaling as library thumbnails. Double-click to enter that user's Cart. If no users have Carts, a centred message reads "No carts available, Users need to add items to their carts first".

Level 2 - User's Cart: Shows that user's Cart items as cards thumbnail grid identical to your own Cart view. Clicking the Market button while viewing a user's Cart returns to the user list (Level 1). Clicking it again exits Market entirely.

Market scans the USERS_PATH directory for subdirectories containing a non-empty cart.txt. Only users with at least one card item appear.

6.2 Right-Click Context Menu

Import to Nuke / Import set |1001|: Import items from another user's Cart.

Import as Cam: For camera-compatible files.

Open With…: External application submenu.

Show in Explorer/Finder: Reveal the file on disk.

Copy Path: Copy the file path to the clipboard.

Add to My Cart / Add N to My Cart: Copy items to your own Cart.

View Meta: Display metadata.

6.3 Your Market Card Image

Your Market card (the tile other team members see in the Level 1 user list) can be personalized with your own image. On **your own** card, a golden pencil (✎) button appears in the corner; opening it gives you **Add Image** and **Remove Image**. The pencil only ever appears on your own card: you can set your image, but never anyone else's.

Add Image lets you choose any picture; Crate scales and centre-crops it to a square and saves it as a single image in your personal user/`profile/` folder. Your Market card then displays that image with your username beneath it. **Remove Image** reverts the card to the default look, the drawn cart icon with its item-count badge. (Setting a profile image uses ImageMagick, the same engine behind Crate's other image work.)

A few things worth knowing: there is exactly one profile image per user (it is not per-cart-item), and it is purely cosmetic, it changes how your card *looks*, not what's in your Cart. It persists independently of your cart contents (clearing your Cart never removes it), and because it lives inside your user folder it travels with you automatically if a Curator archives and later restores your account. You always appear in the Market with your own card, even when your Cart is empty.

7. Discover

7.1 Overview

Discover is an asset management application in Crate, is an ingest tool, an in/out transcode system and a series of panels for bulk/queue operations on a wide variety of asset types. Directory trees called **Branches** are configured in the “Settings/Discover” panel; this panel lives on ”crate_discover.txt. The Discover module, just like all modules in Crate, has its own right click menu options to access tools and operations quickly.

7.2 Detail List View

A sortable column-based list with four columns:

Name: Filename with icon.

Date Modified: Last modification timestamp.

Type: File type.

Extension: File extension.

Size: File size in human-readable format.

Click a column header to sort ascending; click again for descending. Sort preference is saved per-user and restored on the next session.

7.3 Thumbnail Grid View

A thumbnail grid identical to the Library view. This is the only view type where both the OS and Crate Engines work together in charge of temp thumbnail generation, and are not fully persistent cross user sessions. Two toggle buttons in the toolbar switch between Detail List and Thumbnail Grid views. The default is Detail List. The selected view mode persists per session.

7.4 Folder Tree

A hierarchical folder tree on the left contains the BRANCHES. Click a Branch to navigate Folders. Right-click for Set Folder Color. (configured in the “Settings/Discover” panel)

7.5 Address Bar

In addition to clicking on folders like in any explorer system, click anywhere in the path bar above to jump to any dir.

7.6 File Operations

Rename: Select a file or folder and rename in-place.

New Folder: Create a new folder in the current directory.

Cut / Copy / Paste: Standard clipboard operations.

Delete: Moves to _Crate_Data/Discover_Bin inside the Discover path. Recoverable via OS file explorer.

Set Folder Color: Palette of preset colours for visual organisation, plus Reset to Default and Reset All to Default. This is per user and setting stays cross sessions via the Crate users folder /user/gui.

All file operations are gated by the FILE_OPS_TO_VISITORS flag for Visitors.

7.7 Processing Panel Toolbar

13 buttons across the top of Discover launch processing panels:

W-EXR W-EXR-Q W-MOV W-MOV-Q SPLAT-C W-GEO IMG-C CONTACT SEQ-C W-4MAT BATCH VID-C SCRIPT

7.8 File Context Menu

Import to Nuke / Import set |1001|: Standard import.

Import as Cam: Camera-compatible files.

Inspect 3D: Launches F3D as an interactive 3D viewport for the selected model file. Supports OBJ, FBX, STL, glTF, GLB, ABC, USD, PLY, SPLAT, SPZ. If F3D is not installed, the menu entry appears greyed out with "Inspect 3D (F3D not found)". The F3D viewer supports orbit, pan, zoom, animation playback (Space), camera reset (Enter), grid toggle (G), and a full hotkey cheatsheet (H).

Open With…: External applications from crate_open_with.txt, setup on Settings/OpenWith.

Show in Explorer/Finder: Reveal on disk.

Copy Path: Copy to clipboard.

Add to Cart: Add to user's Cart.

Send to Write: Submenu: W-EXR, W-EXR-Q, W-MOV, W-MOV-Q, SPLAT-C, W-GEO, IMG-C ,CONTACT,SEQ-C, W-4MAT, BATCH,VID-C.

Convert/Scale/Sequence to Video/Extract Frames: These functions differ from the "Send to Write/Process toolbar" in that they do not allow access to settings and, while the output quality is good, they do not offer the same level of detail in the settings as the 13 process toolbar panels.

Convert: Format conversion shortcuts for images.

Scale: Resize presets.

Sequence to Video: Convert image sequence to video.

Extract Frames: Extract specific frames from sequences.

Export Layers: For multi-layer PSD to > EXR files.

Delete Object: (Curator) Move to Discover_Bin.

7.8.1 Convert Submenu

For images: Convert to .exr, .png, .jpg, .tiff, .dpx, .tga, .psd, .webp (excludes the current format). For sequences: Convert sequence to .exr, .png, .jpg, .tiff, .dpx, .tga. For video: Convert video to .mp4, .mov, .mkv, .avi. Requires oiiotool or ImageMagick for images; FFmpeg for video. Multi-selection labels adapt (e.g. "Convert 5 files to...").

7.8.2 Scale Submenu

Presets: 25%, 50%, 75%. Available for images and sequences. Output files are named with a _scale{N} suffix. Multi-selection labels adapt (e.g. "Resize 3 files...").

7.8.3 Sequence to Video Submenu

Available when right-clicking a folder containing an image sequence and FFmpeg is installed. Formats: MP4, MOV, MKV, AVI. Output named {folder_name}.{ext} with auto-versioning if file exists.

7.8.4 Extract Frames Submenu

Available for video files when FFmpeg is installed. Extracts all frames to a new folder as: EXR, PNG, JPG, TIFF, DPX.

7.8.5 Export Layers Submenu

Available only for .psd (Photoshop) files. Each layer exported as a separate file to a folder. Formats: EXR, PNG, TIFF, TGA, JPG.

7.9 Folder Context Menu

Import to Nuke / Import set |1001|: Imports all compatible files in the folder.

Open With…: External app submenu.

Show in Explorer/Finder: Reveal on disk.

Copy Path: Copy to clipboard.

Rename / New Folder: File operations (Curator or when permitted).

View Meta: Display folder metadata.

Layer Contact Sheet: Generates a contact sheet showing all layers in an EXR file side by side. Only available when the folder contains EXR files; otherwise greyed out with "Layer Contact Sheet (no EXR)". Uses ImageMagick montage to extract and tile each layer.

Send to Write: W-EXR, W-EXR-Q, W-MOV, W-MOV-Q, SPLAT-C, W-GEO, IMG-C ,CONTACT,SEQ-C, W-4MAT, BATCH,VID-C.

Delete Folder (Curator): Move to Discover_Bin.

7.10 Folder Tree Context Menu

Set Folder Color: Palette of preset colour dots.

Reset to Default: Reset selected folder's colour.

Reset All to Default: Reset all folder colours.

7.11 Detail View Context Menu

New Folder: Create folder (Curator or when permitted).

Paste (N cut/copied): Paste from clipboard.

Set Folder Color: Colour palette submenu.

Import to Nuke / Import set |1001|: Standard import.

Import as Cam / Inspect 3D: For compatible files.

8. Graph (GRAPHS - ANALYTICS)

It´s made of 10 Pages/Tabs:

8.1 THE GRAPH PAGE

GRAPH is the front page of GRAPHS · ANALYTICS. It answers one question at a glance, is anything wrong in my Crate setup, and where, and then lets you walk straight to the source.

It offers two ways of looking at the same information:

Node View: Relationships, dependencies, topology and how the parts connect.
Sunburst: Status at a glance, and drilling from a whole subsystem down to one item.

Both are driven by the same live data and the same health model, so anything flagged in one appears in the other. Use the “view toggle” wheel in the dock to switch between them; each remembers where you were.

Reading Health

Everything in the GRAPH pane is colour-coded the same way:

Colour - State - Meaning

🟢 Green	Healthy	Nothing to do
🟠 Orange	Warning	Worth a look, not urgent
🔴 Red	Critical	Needs attention
⚪ Grey	No data	Not measured, or nothing to measure

This key is always visible at the left of the dock.

Health flows upward. A container takes the worst state of anything inside it. For example one library turns red, which turns Libraries group red, which turns the CRATE core red.

Acts as a system-wide status light. If CRATE is green, everything beneath it is good.

The practical consequence is that you never have to hunt. Start at whatever is red or orange and drill in the colour will lead you to the source.

The Node View

The Node View draws Crate as a nodegraph.

What you are looking at:

- The CRATE core, the largest hub, labelled in gold. Its colour is the worst health found anywhere in the system.
- Other nodes are the subsystems: Volumes, Engines, Cache, Libraries, Users, Modules, File Types, Open Width and Camera DB. Each is a hub circle with a white halo, coloured by its own health.
- Node Satellites are the members of each node, the individual libraries, users, file types, cameras and so on. They fan out around their hub in concentric orbits, and each dot carries its own health colour.
- Connection lines show real dependencies, drawn beneath the nodes so they never obscure anything. As of Crate v2.0.0 lines are just visual, and they don't carry function nor action over Crate, but they might be a visual help for the user since they can be added and removed to represent connections. The lines interact with the node's gravity simulation system.

Fans grow with your data. A handful of members forms a single ring; hundreds form a dense dandelion across many orbits. The camera database, for example, draws all of its entries as one large fan.

Curators are marked with a thick white outline on their satellite dot, so you can spot them at a glance in the Users fan.

The cross-links are what makes the graph read as a connected system rather than a star with everything hanging off the middle.

The Node View gravity field:

Nodes sit in a physics simulation, but it behaves differently from most, it only pushes. Each node carries a protection bubble sized to its whole fan. Bubbles repel each other so fans never overlap, but nothing pulls nodes back together.

- Your arrangement sticks. Drag a node anywhere and it stays there. Spread the graph as wide as you like; it will not spring back.
- Overlaps resolve themselves. Drop one node on top of another and they push apart just far enough to clear, then stop.
- The core anchors the view. CRATE stays where it is, and if you drag it, it stays wherever you drop it.

When the page first builds, a fuller simulation (including attraction) arranges everything into a natural layout. Once it settles, the field switches to push-only and hands control to you.

Your arrangement is saved. Node positions persist between sessions, so the graph you shape is the graph you get back.

Selecting:

- Click a node, the dock shows the category: health, member count, and a breakdown such as for example “8 healthy · 1 warning”.
- Click a satellite, the dock shows that individual item, with a breadcrumb (`Users › alice`) and its own fields.

Selected items are ringed in gold. Click and hold on a node drags it instead of selecting.

Sunburst View

The Sunburst shows the same system as a set of concentric rings, in the style of a network monitoring dashboard.

What you are looking at:

- The centre disc is your current focus, named in the middle.

- Each ring outward is one level deeper in the hierarchy: subsystem > group > item, up to six levels.
- Wedge size reflects how much lives inside it, how many items it ultimately contains.
- Colour is health, with the same worst-child-wins rule.

The rings touch, and white lines are the only separators. Because every child sits exactly inside its parent's angle, those white lines form continuous spokes running back to the centre. To trace any wedge to its origin, follow its two edges inward.

Names:

Names appear along the centre line of their own wedge, and a name is drawn only when it completely fits inside that wedge.

This means zooming in reveals more names. At the default framing you will see the larger areas labelled; as you zoom, smaller wedges become big enough to carry their names. Zooming out only ever hides names again, nothing jumps around, and the same zoom always produces the same labels.

If a wedge has no name showing, hover it, a tooltip gives its name and Health. Hover tooltips are a Sunburst feature; the Node View uses the dock instead.

Use **“T+ / T−”** in the dock if the text is too small to read comfortably.

Drilling:

- Click any wedge to inspect it in the dock. If it has contents, it also becomes the new focus and the rings redraw around it.
- Click the centre disc to step back one level.
- Click the Home button to jump straight back to the top from any depth.

A breadcrumb at the top-left of the view shows your path. Each level opens

framed as large as the window allows.

Navigating

The same controls work in both views.
Zoom: Mouse wheel
Pan: Drag with the “middle mouse button” anywhere, or drag with the left button on empty space
Reframe: The Fit button

Zoom range is very wide, from a whole-system overview down to extreme magnification. This is deliberate: in a dense fan or a crowded ring, zoom in until the item you want is comfortably large, and it becomes easy to click precisely.

Tip: middle-button drag is the safest way to pan while deep in the Sunburst, because it never selects or drills. Left-clicking a wedge will drill into it.

What Crate remembers:

For this session
- Zoom and position, kept separately for each view
- In the Sunburst, zoom and position for every depth level, walk back up and each level is exactly as you left it
- Label text size, kept separately for each view

Between sessions
- Node positions in the Node View
- Connections you have added or removed
- Which view you were last using

You can leave the GRAPH pane for any other analytics page, BRANCHES, VOLUMES,

LIBRARIES, CACHE, CAMERAS, TEMPLATES, CARTS, MARKET, PULSE, leave to other Crate modules and return to exactly where you were.

The Dock (the bar along the bottom of the pane)

Dock Left side

The health key: the four states and their colours.

The inspector: empty until you click something, then it shows that

item's properties. This is the main readout for both views. If the info is too long and you can ´t read it all, go to Crate´s Console and you'll see the entire print there.

Dock Right side controls:

The controls are grouped by purpose, separated by thin dividers:

[+ link] [− link] | [graph toggle] [home] | [T+] [T−] [Fit] [Re-layout] [Alerts Page]

| + link | Add a connection between two nodes (Node View only)

| − link | Remove a connection between two nodes (Node View only)

| graph toggle | Switch between Node View and Sunburst

| home | Jump to the top of the Sunburst (Sunburst only)

| T+ | Larger label text

| T− | Smaller label text

| Fit | Frame everything back into view

| Re-layout | Reset the Node View to defaults (Node View only)

| Alerts Page| Set what turns the graphs orange or red

Buttons that do not apply to the view you are in are greyed out and dimmed.

Editing node connections/links buttons:

To “add” a connection: press “+ link” (it lights cyan), click the first

node, then the second. Both turn cyan as you pick them and the connection

appears.

To “remove” one: press “− link” (magenta) and pick the two nodes whose link you want gone.

While either mode is armed, clicking selects nodes for linking only, you cannot accidentally drag or deselect. Press the button again, click empty space, or switch views to cancel.

Your edits are saved between sessions, and built-in connections can be removed too.
Label/Text size button:

“T+” and “T−” step the label text through a range of sizes. You can also scroll the mouse wheel over either button.

The two views keep independent sizes, so you can run the Sunburst large for reading while the Node View stays compact. Sizes reset when Nuke closes.

Changing the size never moves the graph, only the letters change. Note that larger text needs more room, so at big sizes you may need slightly more zoom before the longest names appear.

Re-layout button:

“Re-layout” is a full reset. It discards your saved arrangement and any connections you have added or removed, then rebuilds the factory default graphs state.

Because that is destructive, it asks twice, and both prompts default to Cancel. Cancelling at either step changes nothing.

Fit button: use if you want to reframe the view.

Understanding Warnings

Selecting anything orange or red tells you “what” is wrong and “where” to look.

For a single item you get the reason directly, for example “⚠ cache coverage 45% (target 70%)”.

For a container, Crate traces the problem down to its source and gives you the path: “⚠ check “Libraries Cache/smoke - cache coverage 45% (target 70%)”

So even when the alert turns out to be a false positive, it always sends you to a specific place to verify. Typical reasons include low cache coverage, orphan thumbnails, a missing engine executable, low free disk space, a file type no longer in use, incomplete camera records, etc.

Use Crate´s Console for more info on alerts:

The dock line is short by necessity. Clicking any orange or red item also prints a full report to Crate's Console with:

- The item and its complete path
- Its state, and the originating item if the problem lies deeper
- Every reason, in full, where the dock abbreviates a long list, the console spells it out
- A member breakdown for containers
- The item's raw values

Healthy items print nothing, so the Console stays quiet unless there is something to report.

Alerts Page:

Crate colours the Infrastructure Graphs green, orange and red to tell you what needs attention. The Alerts page is where you decide what "needs attention" means for your studio, and where you can read every rule that can colour the graph in one place.

Opening it: The 🔔 button sits at the far right of the graph dock

It is available in both the node graph and the sunburst, because these thresholds colour both. Click it to open the Alerts page. BACK TO GRAPH at the bottom right returns you.

What the colours mean:

GREEN	healthy, nothing to do
ORANGE	worth looking at, but nothing is broken
RED	needs attention now
GREY	no data or not scanned yet

Grey means the data has not been gathered, or a path could not be reached, so Crate has nothing to report. Visit the page that owns that data and rescan.

The worst state travels upward: if one library is red, the Libraries wedge is red, and so is the centre. That is intentional, so a problem anywhere is visible without drilling.

Facility size:

Three presets fill every field below with sensible values:

	Solo (1-5)	**Studio** (5-100)	**Facility** (100+)
Cache coverage	80 / 40 %	70 / 30 %	60 / 25 %
Disk free	25 / 12 %	20 / 10 %	15 / 8 %
Orphan thumbnails	3 %	5 %	10 %

Cache clutter	10 %	15 %	20 %
Thumbnail age	180 days	365 days	365 days
Cache speed floor	MB/s	MB/s	MB/s

The presets set tolerance, not scale. Every value is a percentage, a rate or a duration, so it means the same thing at any studio size. What changes is how much mess is worth complaining about: a solo artist fixes an orphan the day it appears, a large facility cannot chase every one.

STUDIO is the default and reproduces the values Crate used before this page existed. If you never open Alerts, nothing about your graph changes.

Editing any field switches the selection to CUSTOM. Click a preset to go back.

The thresholds:

Cache coverage healthy

Above this share of assets having a thumbnail, a library is green. Lower it if your libraries hold a lot of files nobody browses; raise it if you expect every asset to preview instantly.

Cache coverage critical

Below this, a library is red rather than orange. Between the two numbers it is orange.

Disk free warning

Orange when a drive holding libraries or caches drops below this much free space. Caches grow quietly, so headroom is worth keeping.

Disk free critical

Red when free space drops below this. At this point writes can start failing.

Orphan thumbnails

Thumbnails still on disk for assets that no longer exist, usually left behind when files are deleted or moved outside Crate. Expressed as a percentage of cached cards so it means the same thing whether you run one library or two hundred. Clear them with DEFRAG on the CACHE page.

Cache clutter

Files in your cache folders that are not thumbnails: temporary files, system junk like Thumbs.db, and anything unrecognised. A little is normal. A lot usually means an interrupted operation.

Thumbnail age

Flags thumbnails older than this. They still work, but they were made with whatever engine versions you had at the time, so if you have upgraded F3D, OIIO or ImageMagick since, regenerating may look with improvements.

Cache speed floor

The slowest read/write speed you would accept from a cache drive. Ships DISABLED (0) in every preset, because a local SSD and a busy network share differ too much to guess a number. It only applies to libraries you have speed tested on the CACHE page.

Every field carries the same text as a tooltip. Hover anything you are unsure about.

All graph alarms:

Below the fields is every rule that can colour the graph, each on its own row with a coloured dot, the subsystem it applies to, its current value, and a badge:

tunable >	set on this page
tunable . Libraries page>	set elsewhere, see below
always >	no threshold; it fires whenever it is true

Some rules are tunable on this page or in other Crate modules settings. The other are conditions with no sensible number to tolerate, such as a path Crate cannot reach or a conversion engine it cannot find.

Hover any row for a description of what it means and what to do about it.

The list is there so the whole alarm system is visible in one place, rather than discovering one red wedge at a time.

Storage Pool Quota for Libraries:

Is configured in the LIBRARIES page on STORAGE POOL QUOTA, and sets your studio's total allocation in TB and the percentage at which to alert. The graph reads that setting and colours the Libraries wedge and nodes red when total library storage reaches it.

It is deliberately two-state, green to red with no orange, matching the card that owns it, so the same number is never described two different ways.

Leave the allocation at 0 (MONITORING OFF) and the alarm never fires.

Clicking the red wedge reports, for example:

“Libraries STORAGE POOL QUOTA exceeded - 92% of 100 TB used (limit 85%)”

Reading an alarm:

Click any coloured wedge or node. The dock shows a short description naming the problem, and the Crate console prints the full detail including every affected item. Every alarm in the list produces a description; none of them colour the graph without telling you why.

Where the settings live:

analytics_alert_thresholds.json beside menu.py

Studio-wide, not per-user: what counts as healthy is a facility policy.
Written atomically, so an interrupted save cannot leave the studio without thresholds.
Changes apply immediately. Go back to the graph and the colours have already updated, with no rescan.
If the file is missing or damaged, Crate falls back to the STUDIO preset, per value rather than wholesale, so one bad entry cannot discard the rest.
Deleting the file resets everything to defaults.

Notes:

- A brand new install shows FILE TYPES red until people start using carts. The "format never used in any cart" rule compares your library contents against cart history, and on day one there is no history. It clears itself as Crate gets used.

- The speed floor only applies to libraries you have speed tested on the Cache page. Untested libraries are never flagged by it.
- Orphans, clutter and thumbnail age all need a completed scan before they can fire. Until then those libraries read grey, not green.

8.2 THE BRANCHES PAGE

The Branches page (tab index 0) visualizes the storage footprint of all top-level folders configured in Crate's Discover module.

The main graph is a full-viewport exploded sunburst, storage by tree branch. Each petal represents one Discover branch, sized proportionally by its total disk footprint in bytes.

Ring 1 shows top-level branches; Ring 2 breaks each branch into its immediate subdirectories. The center circle displays the grand total (e.g., '1.84 TB') with the branch count below.

The bars list on the right side ranks all branches by size, with each bar colored to match its sunburst petal. When the list exceeds the available height, an internal scrollbar appears.

Scanning:

The first time you open the Branches page, Crate performs a recursive disk scan of every configured branch path. This runs on a background thread and does not freeze the Nuke UI. Progress is shown in real-time, petals grow as data arrives. Results are cached in analytics_cache.json (per-user for write safety). Use the RESCAN button at the bottom to force a fresh scan.

8.3 THE VOLUMES PAGE

The Volumes page shows disk usage donut wheels for every storage volume that Crate touches. It discovers volumes from four sources:

1. Discover branches: every branch path resolves to its volume root (e.g., S:\, L:\, \\Server23\z:\).

2. Library paths: both asset directories and cache directories from Crate Settings, which often live on different volumes (fast SSD for cache, large NAS for assets).

3. Template paths: category folders and their cache paths from crate_templates.txt.

4. System disk: the workstation's OS volume (C:\ on Windows, / on macOS and Linux), always included.

Duplicate volumes are automatically deduplicated, if branches, libraries, and templates all point to the same drive, only one donut appears.

Crate vs System Volumes:

Volumes actively used by at least one Crate module (branches, libraries, or templates) are displayed in color and sorted first. The system disk, if not used by Crate, appears at the end in grey with a SYSTEM label. Its percentage, text, and title are all rendered in grey to clearly distinguish it from Crate-managed storage.

Low-Space Alert:

When a Crate volume's free space falls below the low-space threshold, its donut ring and text turn red, providing an at-a-glance warning.

8.4 THE LIBRARIES PAGE

The Libraries page provides the deepest analytics view, with five stacked cards in a single-column scrollable grid.

A single-ring exploded sunburst where each petal is one configured library, sized by total disk footprint. The bars on the right rank libraries by size. Each library retains the same identity color across all cards on this page.

A single-ring sunburst showing every file extension found across all libraries, ranked by count. Each petal is labeled with the extension name (.exr, .jpg, .fbx, .usd, etc.). This graph uses logarithmic curve mapping, dominant extensions are visually compressed so that rare formats remain visible as petals instead of being invisible slivers. The center shows the type count (e.g., '36 TYPES') and the full file count appears in the footer status line. All extensions are shown.

Horizontal bars showing the file count per library, using the same identity colors as the sunburst above.

Horizontal bars showing the thumbnail cache size per library in bytes.

Storage Pool Quota:

An interactive card for IT storage budget monitoring. It also affects GRAPH page sunburst and node view alarms.

It contains:

Total Library Footprint: the headline metric - the sum of all library sizes, displayed in bold gold text.

Pool Quota input: a numeric field (0–9999.99 TB, 0.01 step) where the Curator enters the storage budget allocated by IT. Expressed in terabytes with two decimal places, so small studios can enter values like 0.09 TB (≈ 92 GB).

Alert Threshold input: a percentage field (1–99%) defining when the warning fires.

Status Indicator_ a colored dot with text: green 'CAPACITY OPTIMAL' when usage is below the threshold, red 'POOL QUOTA CRITICAL' when at or above it.

Four donut wheels spanning the card width:

POOL USED: total footprint as a percentage of quota (green/red based on threshold).

CACHE RATIO: cache footprint as a fraction of total library footprint.

THRESHOLD: outer ring shows the configured alert level, inner ring shows current usage.

ITEMS: total file count across all libraries, with compact formatting (e.g., 4.5K, 1.15M).

Studio-wide persistence: Quota and threshold values are saved in analytics_storage_quota.json in the Crate installation directory (beside menu.py), not per-user. Every Curator on every machine sees and edits the same numbers, the storage server is always one.

Status Footer:

The bottom of the Libraries page shows: number of libraries scanned, total file count, total cache footprint, and cache age.

8.5 THE CACHE PAGE

The cache page provides a comprehensive health analysis of Crate's thumbnail cache system across all enabled libraries.

Unlike other Analytics tabs that focus on asset content and storage, the CACHE page answers operational questions: Are thumbnails being generated? Is the cache bloated with orphans? How fast can Crate read and write to the cache locations? Are there empty or abandoned folders consuming directory entries?

Cache Health (Sunburst):

A two-ring radial chart giving an instant visual overview of all library caches.

- **Inner ring:** Each library's cache, sized by total disk footprint (including sequence and video subfolders).
- **Outer ring:** Breakdown per library into Valid Thumbnails, Orphan Thumbnails, Sequence/Video Caches, and Debris.

  A library with a large "Orphans" or "Debris" slice has dead weight that can be cleaned up with the Defrag button.

Cache vs Assets Ratio:

Paired horizontal bars for each library showing asset size (grey, top bar) against cache size (colored, bottom bar). The percentage label on the right is the cache-to-asset ratio.

Color thresholds:

- **Green (≤5%)**: healthy, cache is proportional to assets.
- **Amber (5–15%)**: slightly heavy, worth monitoring.
- **Red (>15%)**: bloated cache, likely contains orphans or redundant data.

Cache Coverage:

Percentage-fill bars showing what fraction of each library's visible assets have at least one cached thumbnail. This counts both hash-based thumbnails (for images, models, splats) and sequence/video subfolders containing a `best.png`.

Color thresholds:

- **Green (≥90%)**: almost all assets have cached thumbnails, browsing will be fast.
- **Amber (50–90%)**: a significant portion of assets generate thumbnails on the fly each session.
- **Red (<50%)**: most assets lack cached thumbnails, expect slow browsing.

A library with at low coverage may indicate that thumbnail generation was interrupted, that many assets are in formats Crate can't thumbnail, or that the cache was recently cleared.

Cache I/O Speed:

Horizontal bars showing write and read throughput (in MB/s) for each cache path. Results appear after pressing the **RUN SPEED TEST** button in the footer.

The test writes a 10 MB temporary file to each cache directory, measures sequential write speed, reads it back to measure read speed, then deletes the temporary file. Deletion is confirmed in Crate's bottom status bar.

Color thresholds:

- **Green (≥100 MB/s)**: SSD-class or fast local storage.
- **Amber (10–100 MB/s)**: typical local HDD or fast network share.
- **Red (<10 MB/s)**: slow network share or storage problem. Browsing and thumbnail generation will feel sluggish on these paths.

The bar scale adjusts dynamically to the fastest result, so bars are always proportionally readable whether your fastest cache does 120 MB/s or 4,000 MB/s. Speed values include thousands separators for readability (e.g. `Write 1,250 · Read 1,480 MB/s`).

Results are cached and shown instantly on subsequent visits. Press **RUN SPEED TEST** again to re-measure.

Stale Cache:

Bars showing how many days have passed since the newest thumbnail was generated in each library's cache. This surfaces "cold" libraries that haven't had any thumbnail activity recently.

A library showing a high day count may indicate:

- The library's assets haven't changed (normal).
- Thumbnail generation is broken or stalled for that library.
- The cache path is inaccessible or read-only.

The detector checks `best.png` modification times inside sequence/video subfolders as well as hash-based thumbnails in the cache root.

Orphan & Debris Inventory

Bars showing the total debris bytes per library. Debris includes:

- **Orphan thumbnails:** `.png` files in the cache root whose hash doesn't match any current asset in the library. These accumulate when assets are renamed, moved, or deleted but their old thumbnails remain.
- **Orphan cache subfolders:** sequence/video cache folders whose name doesn't correspond to any current asset at any depth in the library tree.
- **System junk files:** OS-generated files like `Thumbs.db`, `.DS_Store`, `desktop.ini`, and macOS resource forks that serve no purpose in cache directories.
- **Temp files:** `.tmp` files left behind by interrupted operations.

- **Stale lock files:** `.crate_lock` files older than 24 hours, indicating an abandoned thumbnail generation lock.
- **Unknown files:** anything in the cache directory that isn't a recognized thumbnail, lock, temp, or system file.
- **Empty/junk-only folders:** directories that are completely empty or contain only system junk files. These are reported for awareness but are not deleted by Defrag. Crate's own infrastructure folders (`_Crate_Data`, `Library_Bin`, `Discover_Bin`, `Templates_Bin`, `_splat_temp`) are excluded from this detection.

Footer controls:

Three buttons appear at the bottom-right of the CACHE page:

RUN SPEED TEST

Launches the I/O speed benchmark on a background thread. Each cache path is tested sequentially. Progress is shown in the status area on the left. The button changes to "TESTING…" while running.

RESCAN

Re-runs the full cache health scan. This walks every library's asset directory and cache directory, cross-referencing thumbnails, detecting orphans, measuring coverage, and finding empty folders. Progress updates appear in the status area. Press again to cancel a running scan (the button changes to "CANCEL" during scanning).

DEFRAG CACHE

Appears only after a scan completes and debris is found. Opens a confirmation dialog showing exactly what will be deleted, broken down by category and by library.

What Defrag deletes:

- Orphan thumbnail `.png` files (hash doesn't match any current asset).
- System junk files (`Thumbs.db`, `.DS_Store`, etc.).
- Temp `.tmp` files.
- Stale `.crate_lock` files older than 24 hours.
- Unknown files that don't match any recognized cache file pattern.

What Defrag never deletes:

- Valid thumbnails (any `.png` whose hash matches a current asset).

- Sequence and video cache subfolders (even orphaned ones, these are reported for manual review).
- Empty folders (reported only).
- Any file less than 24 hours old (safety margin).
- Anything outside a known cache directory.

Safety guarantees:

Orphan detection uses the exact same path normalization and MD5 hashing algorithm as Crate's thumbnail generator. A thumbnail is only classified as orphan if its 12-character hash prefix doesn't match any file currently present anywhere in the library's asset directory.

After deletion, the status bar reports how many files were removed and how much space was freed. If any files could not be deleted (permission denied, locked by system, hidden by firewall), the count of failures is shown and details are printed to the Crate Console.

A rescan runs automatically after defrag to update all cards.

Console report

Every completed scan prints a full health report to the Crate console (Nuke's Script Editor). The report uses emoji markers for quick visual scanning:

- 🟢 Healthy metric, no action needed.
- 🟠 Needs attention.

Each library shows its type, asset/cache sizes, ratio, coverage percentage, valid thumbnail count, sequence/video cache count, debris breakdown, and newest thumbnail age. The library name itself is marked 🟢 or 🟠 based on overall health.

The Totals section summarizes all libraries: total assets, total cache, average coverage, orphan counts, debris totals, and empty folder counts.

If empty or junk-only folders were found, a detailed listing appears at the end showing each folder's classification (`EMPTY` or `JUNK-ONLY`), source (`CACHE` or `LIBRARY`), and full path. For junk-only folders, the hidden/system files found inside are listed.

The Defrag dialog also prints detailed information to the console: unknown file details (name, extension, size, path, access flags) and orphan cache subfolder details, providing a text record for investigation.

Tips

- **First-time scan** may take a few minutes on large libraries with thousands of assets on network storage. Subsequent visits show cached results instantly.
- **Coverage below 100%** is normal for libraries containing file formats that Crate cannot generate thumbnails for (e.g. certain proprietary formats).
- **Ratio above 5%** usually indicates orphan accumulation over time, run Defrag to clean up.
- **Speed test results below 10 MB/s** suggest the cache path is on a slow network share. Consider moving the cache to faster local storage via Crate Settings.
- **Empty folders in library directories** may be intentional (work-in-progress structure), they are reported but never deleted.

8.6 THE CAMERAS PAGE

The Cameras page visualizes Crate's built-in camera sensor database (crate_cameras.json, ~550 KB, 341+ camera models).

Exploded Sunburst, CAMERA DATABASE and MODELS

A two-ring sunburst: Ring 1 shows camera manufacturers (Sony, Canon, ARRI, RED, etc.) sized by model count. Ring 2 breaks each manufacturer into individual camera models. The center shows the total model count. The bars on the right rank manufacturers.

Additional Bars:

Below the sunburst, additional bar cards show sensor width rankings, resolution rankings, and shooting mode distributions.

Camera data loads instantly from the local JSON file with mtime-based change detection, if the database is updated while Analytics is open, switching away and back to the Cameras tab picks up the changes automatically.

8.7 THE TEMPLATES PAGE

The Templates page shows analytics about Crate's curated .nk script template system.

Exploded Sunburst TEMPLATES BY CATEGORY

A single-ring sunburst where each petal is one template category (as configured in crate_templates.txt), sized by the number of .nk scripts in that category folder. The bars list on the right shows the script count per category.

Bars: DISK FOOTPRINT PER CATEGORY

Total bytes of .nk files per category folder, showing which template collections are the largest.

Bars: HIDDEN TEMPLATES

Count of templates hidden via crate_templates_data.txt per category.

Helps Curators audit what's been suppressed from users' view.

Status Footer

Shows total template count, category count, hidden count, and total disk footprint.

8.8 THE CARTS PAGE

The Carts page analyzes every user's shopping cart. the cart.txt file in each user's folder under the Crate users directory.

Exploded Sunburst. CART ITEMS BY USER

A two-ring sunburst: Ring 1 shows users sized by their total cart item count. Ring 2 breaks each user's cart into file-type categories (Images, Video, Geometry, Splats, Nuke Scripts, Sequences, etc.). This reveals both who has the most items and what types they collect.

Bars: OWNED (CREATED) PER USER

Items flagged with OWN content created by the user via Nuke's 'Add to Cart' or Discover export. Shows how much original content each user produces.

Bars: COLLECTED (BOOKMARKED) PER USER

Items without the OWN flag: assets collected from libraries or the Market. Shows browsing and curation activity.

Bars: MOST POPULAR ASSETS (2+ USERS)

Assets appearing in two or more users' carts, ranked by popularity. Reveals the studio's most valued shared assets.

8.9 THE MARKET PAGE

The Market page provides a storage footprint analysis of the Crate user directory structure. It shows how much shared disk space each user consumes.

Exploded Sunburst: USER DATA FOOTPRINT

A two-ring sunburst: Ring 1 shows users sized by their total data footprint under the users directory. Ring 2 breaks each user into storage categories: Media Cache (generated thumbnails), Card Thumbs (cart item previews), and Profile (profile images). Heavy users of the cart system will show larger footprints.

Bars: MEDIA CACHE PER USER

Bytes in each user's user/media_cache/ folder, generated thumbnail data. Large caches indicate heavy cart usage.

Bars: PROFILE & CARD THUMBS PER USER

Combined size of profile/ and card_thumbs/ folders per user.

Bars: .nk EXPORTS PER USER

Count of .nk script files saved to each user's root folder.

Status Footer

Shows total user count, curators vs visitors, users with carts, and users with profile images.

8.10 THE PULSE PAGE

ADOPTION FUNNEL (sunburst): three-segment exploded pie: "Available Only" (library files never carted), "Carted" (in at least one user's cart), "Exported" (became .nk scripts). The drop-off ratios reveal how much of the library is actually being consumed. A 17,000-file library where only 200 items appear in carts tells the curator the library needs better discoverability or pruning.

DEAD FORMATS (bars): file extensions that exist in libraries but appear in zero carts across all users. If .3ds, .odt, or .tmp show up here, those files are occupying disk but delivering no value. Ranked by file count.

USER ENGAGEMENT SCORE (bars): composite per-user score: cart items + 2× owned + 3× exports + 5 if profile exists. Distinguishes power creators (high owned + exports) from passive browsers (items but not owned) from inactive accounts (zero score). Not just "who has the most", weighted toward production activity.

TEMPLATE UTILIZATION (bars): visible percentage per template category. A category at 30% means 70% of its scripts are hidden. possible over-curation. A category at 100% means nothing is suppressed. Surfaces the balance the curator is striking.

CACHE EFFICIENCY (bars): cache bytes per file per library, ranked worst-first. A library with 50 KB per file cache cost is normal; one at 500 KB may have oversized thumbnails or stale cache entries. Helps the curator decide which caches to regenerate or clean.

PULSE lazily triggers the scans for any tabs that haven't been visited yet (Carts, Market, Templates) so it works even if you jump straight to it. The REFRESH button recomputes everything.

ANALYTICS CACHING & PERFORMANCE

Analytics uses a layered caching strategy to balance freshness with performance.

Branches and Libraries: scanned via background threads (ThreadPoolExecutor). Results are cached in analytics_cache.json per user. A RESCAN button at the footer forces a fresh scan. Cached data loads instantly on subsequent visits.

Cameras: parsed from the local JSON database with mtime-based change detection. No explicit cache; re-parses only when the file changes.

Templates, Carts, and Market: instant scans (reading small text files and shallow directory listings). Results are cached in memory and in analytics_cache.json. RESCAN buttons are provided on each page.

Volumes: refreshed on a timed interval (shutil.disk_usage calls on a daemon thread). No disk cache; values are always live.

Storage Pool Quota: studio-wide, stored in analytics_storage_quota.json in the Crate installation directory.

Palette Preference: per-user, stored in analytics_palettes.json in the user's folder.

PALETTE DROPDOWN MENU

All palettes work with the adjacency-aware color separation engine (farthest-point sampling + greedy neighbor placement + 2-swap refinement), ensuring that visually similar colors are never placed next to each other in sunburst graphs. When a chart needs more identities than the palette's base colors, lighter and darker HSV variants are generated automatically.

9. Templates

9.1 Overview

Templates provides a browsable collection of `.nk` Nuke script templates, pre-built node setups such as render-engine configurations, slates, or comp skeletons, organised by category. Template sources are configured in Settings/Templates (`crate_templates.txt`), and each top-level source folder becomes one browsable **category**; the `.nk` scripts inside it become the template cards.

Like the Market, Templates is a two-level browser: the opening view is a grid of category cards, and entering a category shows the scripts within it. Importing a template pastes its node graph straight into your script. Templates are published from Nuke rather than authored here, their thumbnails are set by hand rather than auto-rendered, and Visitor access is controlled per-module, per-category, and per-script. Each source keeps its own `_Crate_Data/Templates_Bin` for safe, recoverable deletion.

9.2 Navigating Templates

Templates use the same two-level navigation as the Market.

Level 1: Category list. Opening Templates shows a grid of category cards, one per template source folder configured in Settings. Each card shows a thumbnail, either an image a Curator has set, or the default connected-crates placeholder icon, with the category name and a live count of the scripts inside it beneath, e.g. `Render Setups (12)`. The path label reads "Templates." Double-click a category to enter it.

Level 2: Scripts. Inside a category, the grid shows one card per `.nk` script. Each card shows its own thumbnail (a Curator-set image, or the "NK" placeholder if none is set) and the script name. Double-click a script or right-click and choose Import to Nuke, to paste it into the node graph. The Back button, or clicking the Templates button once, returns you to the category list; clicking Templates again exits the module.

The search box filters by context: "Search templates…" filters the category list, and "Search scripts…" filters the `.nk` list inside a category.

9.3 Importing a Template

Select a single template and double-click it (or use the right-click menu). The template's `.nk` contents are pasted into the node graph via `nuke.nodePaste()`. Multi-selection import is disabled, the label changes to "Import to Nuke (select single item)".

9.4 Edit / Add Thumbnails to Templates

Template cards don't have auto-generated thumbnails, a `.nk` script has no inherent image, so a Curator sets them by hand. A golden pencil (✎) button appears in the top-right corner of cards, on **both** the category cards (Level 1) and the individual script cards inside a category (Level 2), letting you give each its own image. The button is **Curator-only**; Visitors browse and import but don't set thumbnails.

Click the pencil to choose an image; Crate scales it to a centred square and stores it in that category's configured cache path, keyed to the card's name. Until an image is set, the card falls back to a placeholder, “Foundry's Hobbit" `.nk` placeholder for scripts, which can itself be changed at `crate_media\placeholders\.nk`. Because these images are deliberately chosen rather than rendered, clearing a template cache simply returns the affected cards to the placeholder until images are set again.

9.5 How Templates Are Created

Templates aren't authored inside the Templates view, that view only browses and imports them. A new template is **published from the Nuke node graph**: select the nodes you want to save, then use **Crate/Add to Templates** from the Nuke menu bar. Crate writes the selection as a new `.nk` file into the template category you choose, and immediately offers to set a thumbnail for it so the new card isn't left on the placeholder. (Whether Visitors are offered this action is governed by the `VISITOR_MENU_TEMPLATES` flag) The categories themselves are defined and managed in Settings/Templates; this is the one place templates are written into a source folder, alongside Add to Discover, as a deliberate, user-initiated action.

9.6 Visitor Visibility

What a Visitor sees in Templates is controlled at three levels, all set by Curators (who always see everything):

- **The whole module:** the "Available to Visitors" master toggle in Settings/Templates. When off, Visitors don't get Templates mode at all.
- **A category:** each category's "Visible to Visitors" flag. An otherwise-available module can still hide specific categories (e.g. expose "Slates" to everyone but keep "R&D" Curator-only).
- **A single script:** right-click/**Hide From Grid** (Curator) removes one `.nk` from the Visitor view without deleting it. Hidden scripts are managed in Settings/Templates and can be un-hidden or moved to the Templates bin.

9.7 Right-Click Context Menu

Import to Nuke: Paste template into node graph (single selection only).

Open With…: External application submenu.

Show in Explorer/Finder: Reveal the `.nk` file on disk.

Copy Path: Copy to clipboard.

Add to Cart: Add to user's Cart.

View Meta: Display metadata.

Hide From Grid (Curator): Remove from Visitor view without deleting.

10. Camera Database (CAMDB)

10.1 Overview

A built-in optical reference module containing 340+ real-world camera sensor specifications loaded from crate_cameras.json. Artists select a camera model, choose a sensor crop mode, configure lens parameters, and create properly configured Nuke camera nodes with accurate film-back aperture values. The module also provides a real-time sensor visualization with optical analysis overlays including image circle coverage, probabilistic lens distortion, field of view, depth of field, focus breathing, anamorphic de-squeeze, and diffraction awareness, all probabilistic approximations that you can use as a creative starting point.

10.2 Interface Layout

The Camera Database occupies the full Crate visualization area and is toggled via the camera icon button in the toolbar. The interface is divided into two regions:

Left panel: camera dropdown at the top, then the data readouts, box data entry for different camera values and the camera export dropup at the bottom.

Right area: the sensor visualization canvas showing the full sensor outline, crop rectangle, image circle, min coverage reference, distortion grid, FOV cone with DOF marks, and the focus sharpness disc. Above the sensor: aspect ratio and pixel pitch. Below the sensor: physical dimensions in mm and pixel resolution.

10.3 Camera Selection

The Camera dropdown at the top of the left panel lists all cameras in the database sorted alphabetically. Clicking the dropdown opens a full-panel overlay chooser. The chooser supports keyboard navigation, scrolling, and closes on Esc or click-outside. Selecting a camera loads its sensor type, shutter type, and all available sensor modes.

10.4 Sensor Imaging Area

A list widget showing all sensor modes (crops) available for the selected camera. Each entry displays resolution, mode name, and physical sensor dimensions in mm. Selecting a mode updates the crop rectangle on the visualization, recalculates all derived values (crop factor, min coverage, 35mm equivalents, FOV, DOF), and adjusts all graphical overlays accordingly. The first entry (index 0) is treated as the full-sensor open gate mode and defines the full sensor outline.

10.5 Data Readouts

Horizontal Aperture: the selected crop's physical width in mm.

Vertical Aperture: the selected crop's physical height in mm.

Crop Factor: the ratio of the 35mm full-frame diagonal (43.27mm) to the selected crop's diagonal. Values below 1.0 indicate a sensor larger than full-frame; values above 1.0 indicate a smaller sensor.

Min Coverage: the minimum lens image circle diameter (in mm) required to fully cover the selected crop without vignetting. Equals the crop diagonal. Always a circle regardless of anamorphic mode.

10.6 Camera Parameters

Shutter Frame Length: editable field with Nuke-style arrow-key stepping. Value between 0 and 1 representing the fraction of a frame the shutter is open. Synced bidirectionally with the shutter angle field next to it (angle = frame length x 360).

Shutter Offset: dropdown with options centred, start, end, custom. Affects the shutter angle pie position in the exposure dashboard.

Focal Length: in mm. Drives the 35mm equivalent calculation, FOV angle, FOV cone geometry, distortion grid shape, and DOF computation. The 35mm equivalent is displayed inline to the right. For anamorphic mode, the equivalent accounts for the squeeze factor and is labelled "(ana)".

FOV: read-only horizontal field of view in degrees, calculated as 2 x atan(crop_width x squeeze x breathing / (2 x focal_length)). Accounts for anamorphic squeeze and focus breathing. Matches the angle displayed on the FOV cone in the visualization.

Focal Distance: in meters. Affects focus breathing (FOV narrows at close focus), the distortion grid breathing scale (subtle zoom at close focus), and DOF near/far limits. Default 2.0m. Values beyond approximately 15m are effectively infinity for breathing purposes.

Fstop: the lens aperture f-number. Drives the 35mm equivalent f-stop, DOF calculation, the exposure dashboard iris visualization, and the focus sharpness disc on the sensor visualization. The 35mm equivalent f-stop is displayed inline.

DOF: read-only depth of field readout showing near limit, far limit, and total depth in meters. Calculated using the standard photographic DOF formulas with circle of confusion derived from the crop diagonal (CoC = diagonal / 1500). When the far limit extends to infinity, displayed as the infinity symbol. When the Airy disc diameter (1.34 x fstop, microns) exceeds the sensor's pixel pitch, a "diff. soft" warning appears in muted orange indicating diffraction-limited softening.

"diff. soft" Diffraction warning: When this indicator appears next to the Depth of Field readout, your current f-stop has stopped the lens down far enough that diffraction, the physical blurring of light passing through a small aperture, is softening the image beyond what the selected sensor can resolve. Crate compares the diffraction blur spot (the Airy disc, ≈1.34 µm × f-stop) against the pixel pitch of the selected camera and sensor mode, and warns once the blur spans more than two photosites. The result on screen is a uniform softening of the entire frame that no amount of refocusing or sharpening fully recovers. To clear it, open up to a wider aperture (lower f-number), or choose a sensor mode with larger photosites; small high-resolution sensors reach this limit at much shallower stops than large ones. Depth of field gained past this point comes at the cost of overall sharpness, the warning marks where that trade stops being free. Film formats and modes without a stored pixel resolution do not trigger this warning.

The four corner graphics: Around the sensor visualization, four small animated diagrams translate your current settings into the optical behavior they produce, each one reads the panel live and reacts as you type.

Top-left (Airy pattern): a faint grid where each cell is one photosite of the selected sensor mode, with the diffraction pattern of a point of light drawn over it at true relative scale. Wide open, the light lands as a dot inside a single pixel; as you stop down, or choose a denser sensor, the disc grows and its rings appear, and the moment it spans two cells the graphic turns orange, in step with the *diff. soft* alert.

Bottom-right (aperture diffraction): the cause behind that pattern, plane light waves passing through the iris. With a wide aperture the transmitted wavefronts stay nearly parallel; stopping down closes the bars and bends the waves into circular radiation, turning orange together with the alert.

Top-right (focus cone): the ray side of image formation, light from a subject point bending through a glass element toward the sensor plane. Focal length and f-stop set the width of the cone (the real entrance pupil), and Focal Distance slides the convergence point: land it on the plane and the point stays sharp; miss near or far and the rays open into a circle of confusion whose marker warms toward orange as the blur grows.

Bottom-left (bokeh signature): how an out-of-focus point light renders a clean circle with spherical glass that stretches into a vertical oval matching your squeeze factor when Ana is engaged, flare streak included. Together they bracket the image: what diffraction and defocus each cost you in sharpness, and what your lens choices contribute in character. The graphics are purely informative, they never affect the camera data or exports.

Camera Name: the node name used when creating Nuke camera nodes. Default "shotCam". Collision resolution appends incrementing numbers if the name already exists in the node graph.

10.7 Lens Image Circle

An editable field where the artist enters the manufacturer's rear physical circle diameter in mm, as printed on the lens spec sheet. This is the actual round pool of light the lens projects onto the sensor, always circular regardless of spherical or anamorphic design. Default 0.0 (not set).

When a value is entered, the visualization draws a solid dark disc representing the lens coverage, and the status text shows either "covers +X.X mm" (lens fully covers the crop) or "vignettes -X.X mm" (lens falls short of the crop diagonal). When the lens circle doesn't fully cover the crop, dark orange crescents appear in the uncovered corners of the crop rectangle showing the vignetting zone.

The Ana checkbox enables anamorphic mode. When checked, a squeeze factor selector appears (2x, 1.8x, 1.65x,1.6x 1.5x, 1.33x, 1.25x, 1.2x). Anamorphic mode affects: the FOV calculation (wider horizontal field of view), the 35mm equivalent focal length, the distortion grid shape (per-axis barrel/pincushion with characteristic anamorphic mumps), and the aspect ratio display above the

sensor. The image circle itself remains circular because the physical light boundary is always round. The squeeze factor dropdown defaults to 2x and is only visible when Ana is checked.

10.8 Exposure Dashboard

Four visual icons below the camera parameters showing shutter angle (pie chart), f-stop (iris ring with variable aperture hole), shutter speed (running figure with motion blur ghosts), and ISO (grain density circle). All four derive from the shutter and fstop values entered above using a photographic exposure model (ISO = 100 x N² / (t x 2^EV), EV=11 indoor/overcast scene).

10.9 Export Dropup

The Camera dropup at the bottom of the left panel (labelled "Camera", default text "Select export type...") opens a full-panel overlay chooser containing the six export actions:

Create Camera Classic: creates a Camera2 node in Nuke with correct h-aperture, v-aperture, focal length, focal point, fstop, shutter, and shutter offset values. Available on all Nuke versions. Node is labelled with camera model, crop mode, and aperture dimensions. Tile colour red (Classic 3D convention).

Create Camera USD: creates a Camera4 node. Nuke 17+ only. On older versions: Crate-styled dialog explaining version requirement. Tile colour green (USD 3D convention).

Create Camera Tracker Classic: creates a CameraTracker node with filmBackSize set to the selected sensor dimensions. Requires NukeX or Nuke Studio. Sets focalLengthType to "Known" with the entered focal length. Classic 3D tile colour.

Create Camera Tracker USD: creates a CameraTracker configured for USD 3D workflow. Nuke 17+ only and requires NukeX. USD tile colour. Functionally identical CameraTracker node but labelled and coloured for USD pipeline visibility.

Create 3DEqualizer Lens: exports a 3DEqualizer4 lens file (.txt) with filmback dimensions converted to centimeters, focal length, film aspect, and zeroed distortion parameters using the "3DE4 Radial - Standard, Degree 4" model. Opens a save dialog.

Export Camera: multi-format camera export via crate_cam_export.py. Opens a save dialog with format filter options. Supports various DCC interchange formats.

10.10 Sensor Visualization

The right-side canvas draws the following overlays in this z-order (back to front):

Lens Image Circle: a solid dark disc centered on the sensor, representing the physical pool of light from the lens. Always circular. Only appears when a Lens Image Circle value is entered.

Vignette fill: dark orange crescents in the crop corners where the lens circle doesn't cover. Appears only when the lens circle is smaller than the crop diagonal.

Full sensor outline: grey rectangle showing the physical sensor boundaries at full open gate dimensions.

Crop rectangle: dark yellow 1px outline showing the active recording area for the selected sensor mode.

Focus sharpness disc: a fixed-size radial gradient circle at the sensor centre. At high f-stops (f/22) it appears as a pixel-sharp solid dot; at wide apertures (f/1.4) it fades into a diffuse glow. Illustrates how aperture affects image sharpness.

Min Coverage circle: a dashed yellow circle that always inscribes the crop rectangle (touching its four corners). Represents the minimum lens image circle needed for full coverage.

Distortion grid: a ghostly 15x11 line grid warped by the Brown-Conrady division model. The grid spans 1.5x the sensor dimensions (extending beyond and clipped to the crop boundary so no corners are visible). Distortion is estimated from focal length vs sensor diagonal: wide-angle produces barrel distortion (lines bow inward), telephoto produces pincushion (lines bow outward). For anamorphic, the horizontal and vertical axes use different k1 coefficients (effective horizontal FL = FL / squeeze), producing the characteristic eye-shaped anamorphic mumps. The grid also responds to focus breathing with a subtle scale effect at close focal distances. The distortion model uses an asymmetric response: barrel scales linearly up to k1=0.15 for wide angles, pincushion follows a square root curve up to k1=-0.25 for telephoto, ensuring visible but non-collapsing distortion across all focal lengths. (This grid is illustrative, semi-accurate, and intended for creative and educational use.)

Crop label: yellow text inside the crop rectangle showing mode name and dimensions in mm.

Aspect ratio and pixel pitch: grey text above the sensor showing the crop aspect ratio (with common named ratios like 16:9, 4:3, scope), the de-squeezed ratio when anamorphic is active, and the pixel pitch in microns.

Sensor dimensions: grey text below the sensor showing physical width x height in mm.

Resolution: grey text below the sensor dimensions showing pixel resolution.

FOV cone: two subtle yellow lines radiating upward from a stationary nodal point near the bottom of the visualization. The lines move freely based on the computed horizontal FOV, they will extend narrower or wider than the crop depending on focal length, anamorphic squeeze, and focus breathing. At wide focal lengths the cone fans beyond the crop edges; at long focal lengths it narrows to a tight V well inside the crop. The lines extend past the crop rectangle and sensor outline, clipped only by the widget bounds, matching the visual behavior of the image circle and min coverage overlays. The horizontal FOV angle is displayed as a label with an arc near the nodal point.

DOF tick marks: four tiny 1px horizontal dashes on the FOV cone lines (two per line) marking the near and far depth of field limits. The ticks follow the cone lines as they move with focal length and breathing changes. A very low opacity DOF distance label appears next to the right pair. The ticks spread apart at small apertures (deep DOF) and close together at wide apertures (shallow DOF).

Scene footprint: Along the bottom edge of the crop rectangle, the sensor visualization prints the physical area of the world your frame covers at the current Focal Distance, width × height in meters, followed by feet, in the same yellow as the filmback line at the top. Together the two lines describe the crop completely: what the sensor *is* above, what it *sees* below. The value updates live with focal length, focus distance, sensor mode, and anamorphic squeeze (an Ana setup correctly covers a wider horizontal slice of the world), making it a direct answer to on-set questions like "how large a greenscreen do I need at four meters?" or "will this cyc fill frame?".

The line hides itself when the optics can't produce a meaningful value (focus distance at or inside the focal length) or when the crop rectangle is drawn too small to fit both lines cleanly.

Moon: The Moon checkbox, next to the bokeh graphic below the sensor view, overlays the moon on the sensor visualization at its true angular scale, the moon subtends a fixed 0.52° in the sky, so its size on your sensor depends only on focal length. It draws as a plain grey circle (solid line, so it can't be confused with the yellow dashed coverage references) centered on the sensor. On normal focal lengths it's a humbling dot or invisible entirely, below roughly 35 mm it's smaller than the drawing can resolve, which is itself the point, while long lenses make it bloom: around 800 mm it claims a third of a Super 35 crop's width. With Ana engaged, the moon renders as a vertically stretched ellipse, exactly as the squeezed negative would record it before de-squeeze.

10.11 Database List

The camera database (crate_cameras.json) contains 340+ cameras spanning cinema cameras (ARRI, RED, Sony, Blackmagic, Canon, Panasonic), DSLRs, mirrorless, film formats (35mm Academy through 8x10 large format), drones (DJI), action cameras (GoPro), high-speed cameras (Phantom), and medium format (Fujifilm GFX, Hasselblad). Each camera entry includes manufacturer, sensor type, shutter type, camera type, an optional reference URL, and an array of sensor dimension modes with pixel resolution and physical aperture dimensions in mm.

The database is editable from Settings and supports adding, editing, duplicating, and deleting cameras and their sensor modes. All changes are written back to crate_cameras.json on disk. The module supports hot-reload: after editing the database in Settings, the open Camera Database panel refreshes automatically without requiring a Nuke restart.

10.12 Credits

Camera reference data derived in part from VFXCamDB.com (Tony D'Agostino) and Matchmove Machine (camdb.matchmovemachine.com). Additional curation and data by CrateTools. Licensed under Apache 2.0 (code) and CC-BY-4.0 (data). Users redistributing or integrating the camera data

are encouraged to preserve credit to both VFXCamDB and Matchmove Machine alongside CrateCamDB.

11. Crate Settings

This is Crate´s general control panel. Visual editor for engine paths, access control, and system configuration. Curator-only access. Built by crate_settings_module.py. Access in the top right “Settings…” dropdown.

11.1 Engine Test Buttons

Test F3D: Runs f3d --version and reports success/failure.

Test ImageMagick: Runs magick --version.

Test MediaInfo: Runs MediaInfo --Version.

Test FFmpeg: Runs ffmpeg -version.

Test OpenImageIO: Runs oiiotool --help and checks exit code.

Test Splat Transform: Verifies the playcanvas/splat-transform npm package exists on disk and detects a portable Node.js binary; reports ONLINE READY or OFFLINE with install instructions.

11.2 Lock/Unlock/Lock Selected logics

Two buttons in Crate Settings: LOCK CRATE and UNLOCK CRATE. Lock prevents Visitors from accessing Crate entirely (they see a "Crate is locked" message). Curators always have access.

“Lock Selected” (the dropdown menu next to the camera database button):

It allows a Curator to keep a folder of objects/elements offline for maintenance without Visitors being able to access it. For example, *temporarily closing access to the “Smoke” library for maintenance purposes.*

While this dropdown is not inside Crate Settings it belongs to the same family logic.

11.3 Lock a Settings Page

Directly above the LOCK CRATE button sits **LOCK PAGE**, a per-section editing lock that a Curator claims before changing a settings page. Where LOCK CRATE governs studio-wide access to Crate as a whole, LOCK PAGE is scoped to the single Settings section currently open Engines, Access, Users, and so on through Nuke. It exists so that when several Curators are in Settings at the same time, one Curator can reserve a page before editing it, guaranteeing that no one else changes and saves the same configuration file underneath them.

When a section is free to edit, the LOCK PAGE button pulses between grey and orange, a reminder to claim the page before you begin. Clicking it writes the lock and the button turns solid orange, reading PAGE LOCKED; from that moment other Curators see the page as locked and may view but not edit it.

Crate also claims a page automatically the instant you make your first change, so a page is never left unprotected, but locking it up front with the button is the reliable habit, especially on a busy office.

If a section is already held by another Curator, its sidebar button is tinted orange with a small padlock icon, the LOCK PAGE button is disabled and reads LOCKED by *(name)*, and an overlay across the page shows who is editing it and from which machine. You may read everything but cannot change or save it until they release it. Because the lock is per section, different Curators can safely work in different sections at once.

While Settings is open, Crate re-checks every section's lock a few times a minute, so a page becoming locked (or freeing up) appears on your screen within a few seconds, without you having to click away and back. If a page you are viewing is claimed by another Curator while you have unsaved changes on it, Crate tells you so and disables your Save, so the conflict is never silent.

A page lock releases automatically when you save the section, when you leave it for another section, and when you close Settings, whichever comes first. There is no manual unlock. If you try to leave a page, or close Settings, while it still holds unsaved changes, Crate asks how to proceed:

Save: writes your changes to the section's configuration file, then releases the lock and continues.

Don't Save: discards your changes and restores the page exactly as it was on disk, leaving the configuration file untouched, then releases the lock and continues.

Cancel: keeps you on the page with your changes and your lock intact.

As a safety net, any page lock older than thirty minutes is treated as stale and cleared automatically, so a crash or a forgotten open window can never block a section permanently.

11.4 Settings Sections

The Settings panel is the single place where a Curator configures Crate. It is Curator-only and Visitors never see it, and nothing in it can be changed without Curator access. Settings is organised into twelve navigable sections, listed as buttons in a scrollable sidebar; selecting a button shows that section's controls on the right.

The **CHECK FOR UPDATES, LOCK PAGE, LOCK CRATE, SAVE** and **CLOSE** buttons stay pinned.

Per-section editing locks:

Because a studio may run several Curators at once, each section has its own advisory edit-lock so two people cannot overwrite each other's changes to the same configuration file. When a Curator begins editing a section, Crate writes a small lock file recording who holds it (username and machine) and when.

Other Curators then see a lock icon on that section's button and can view it but not edit it until it is released. The lock is per section, so technically different Curators can safely work in different sections at the same time (not recommended).

Locks release automatically when the Curator closes Settings, and a Curator always keeps access to sections they themselves locked. As a safety net, any lock older than 30 minutes is treated as stale and cleared automatically, so a crash or a forgotten open window can never block a section permanently. Each section writes to its own plain-text configuration file (listed below), so settings remain transparent and inspectable outside the application.

Engines: Configures the external tools Crate drives for thumbnail/preview generation and format conversion. Sets the engine base path and per-engine path overrides, plus the F3D worker limit (the maximum number of concurrent generation processes, shared across all libraries so a workstation is never overloaded). Six Test buttons: F3D, ImageMagick, MediaInfo, FFmpeg, OpenImageIO, Splat Transform, confirm each tool is found and runnable before you rely on it. This section also hosts the Lock / Unlock Crate control. Backed by `crate_engines.txt`.

Access: The Curator roster, read from and written to `crate_access_credentials.txt`. Add promotes a username to Curator, Remove deletes the entry, and Pause temporarily suspends a Curator (stored as a `#PAUSED:` entry) so they revert to Visitor behaviour at runtime without losing their place, fully reversible. Anyone not listed here is a Visitor by safe default. Visitor-facing capabilities themselves are configured in the Nuke and Discover sections.

Users: Two related things. First, USERS_PATH: the directory where all per-user data lives, each person's cart, exported `.nk` scripts, GUI preferences, and their `media_cache`, `card_thumbs`, and `profile` folders, stored in `crate_users.txt`. Second, Active / Archived user management. The Active list shows every user folder under the users root, each with a quick summary (cart item count, number of scripts, folder size, and role). Archive moves a user's entire folder into an `_archived` subfolder: their data is fully preserved but they are hidden from the Market for everyone. Restore moves the folder back and they reappear exactly as before. Nothing is deleted by archiving, it is intended for people who have left, while keeping their curation recoverable.

Curators Info: Curator metadata and display information (crate_curators_info.txt), surfaced in Crate's welcome/about context.

Hide Prefixes: Folder-name prefixes that cause any matching folder to be hidden from the asset grid (`crate_hide_prefixes.txt`). Useful for keeping work, archive, or scratch folders out of the browser without moving them on disk. Keep unwanted cards to be seen in a Library.

Libraries: The library definitions editor (`crate_libraries.txt`). Add, remove, and reorder libraries, and edit each one's name, asset path, cache path, type, visibility, and enabled state. This is where shared asset sources and their (often central) caches are wired up.

Templates: The template source definitions editor (`crate_templates.txt`): the template categories/folders Crate exposes, plus the master enable for the Templates module.

Discover: Discover project definitions (`crate_discover.txt`): the project paths and their per-project visibility, the master enable for the Discover module, the FILE_OPS_TO_VISITORS toggle (whether Visitors may perform file operations within Discover at all), and the per-user Visitor allowlist that grants those operations to specific named Visitors.

Open With: External application definitions (`crate_open_with.txt`). Add, remove, and reorder the applications offered in the "Open With" menu, each with a browse button to point at its executable.

Camera Database: Settings for the Camera Database module (`crate_cameras.json`), used for camera metadata.

Logs: Two console-output toggles: LOG_LEVEL (basic vs. dev verbosity) and CONSOLE_EMOJIS (on/off). These affect only diagnostic output, not Crate's behaviour.

Nuke: Nuke menu-integration flags, each a Yes/No toggle pair: VISITOR_MENU_DISCOVER, VISITOR_MENU_CART, and VISITOR_MENU_TEMPLATES (whether Visitors are offered the corresponding "Add to…" actions in the Nuke node-graph menu) and IMPORT_BACKDROP (whether imported assets are wrapped in a labelled backdrop node).

11.5 Settings Sections - Libraries (in depth)

Libraries are the core of Crate. A library is a named pointer to a folder of assets on disk, paired with a place to cache that folder's thumbnails and a rule for how its contents should be previewed. Everything a user browses in Crate ultimately comes from a library defined here, which is why this section is the most important one in Settings, in a real sense it is the reason Crate exists. The section is Curator-only and writes to `crate_libraries.txt`

The library list

The upper panel lists every defined library. Each row is shown as [ON] name (type) or [OFF] name (type), where the prefix reflects the Enabled state and the trailing word is the library's type. Select a row to load it into the editor below.

+ Add Library creates a new entry (a blank "standard" library) ready to fill in. **Delete** removes the currently selected library definition. Deleting a library removes only the *definition*, it never touches the asset files on disk, and it does not delete the cache folder; it simply stops Crate from showing that source. The order of rows here is the order in which libraries appear in Crate's library dropdown, so arrange new entries with that in mind. The Library at the top of the list is the Default/Home one when you open Crate.

Per-library settings

When a library is selected, the editor exposes the following controls.

Enabled: A toggle. When off, the library is fully defined but excluded from Crate's library dropdown for everyone. Use this to retire or temporarily pull a source without losing its configuration or its cache. This is the `[ON]/[OFF]` state in the list.

Show Path in Dropdown: A toggle. When on, the library's full asset path is shown beneath its name in the dropdown; when off, only the name appears. Purely cosmetic, it does not change where assets are read from, and useful for keeping long network paths out of the picker once a library is established.

Name: The library's display name. It must not contain spaces (use underscores); the name is also used internally to key the library's cache, so changing it later effectively points the library at a fresh cache.

Asset Path: The folder on disk where the actual asset files live. This is the **source**, and Crate treats it as read-only: it reads files here to display and import them, but never writes, renames, or moves anything in this location (the only writes Crate ever makes to a source tree are the explicit, user-initiated publish and delete-to-bin actions covered elsewhere). The **|…|** button opens a folder browser to set this path.

Cache Path: The folder where Crate stores the generated thumbnails and hover previews for this library. It is deliberately separate from the asset path so that previews can live on fast or central storage while the assets stay wherever the pipeline keeps them; nothing irreplaceable is ever stored here, only regenerable previews. Fast local or central cache storage is recommended. The **|…|** button browses for the folder.

Shared Cache Folder Configuration Recommendation:

For tiny setups, single freelancers or small teams of up to five users working with a very small library, it is technically possible to use a single shared cache folder for all libraries.

However, once you scale to hundreds of libraries, each containing hundreds of subfolders and files and maintaining its own cache on disk, assigning a separate cache path to each library is strongly recommended. We think that even if you are a solo artist using a small library, you should also keep an independent cache folder per each library you setup.

This separation is required for three reasons, listed in order of importance:

1. Correctness

Sequence and video cache subfolders are keyed only by the clean name of the media (for example, `fog_####.exr` becomes a subfolder named `fog_exr`; a video becomes a subfolder named after its basename). No path hash is included.

When multiple libraries share the same cache folder, two libraries that contain a sequence or video with the same name will write to the identical subfolder and overwrite each other’s thumbnails and hover sprites. This is the cause of incorrect thumbnails appearing on sequence cards.

2. Locking

Each cache folder uses a single **`.crate_lock`** file. A shared folder therefore causes generation activity in any one library to block generation in every other library on every machine, creating unnecessary studio-wide contention.

3. Performance

A shared folder that accumulates PNG files from every library (often reaching thousands of files) slows every SMB existence check.

Per-library cache folders are the intended design. The libraries file includes a `cache_path` column for each library, and the initialization code comments explicitly describe “isolated cache folders.” A shared layout works against this architecture at scale.

Migration (settings only)

Give each library its own dedicated cache path.

The flat `{hash}_256.png` files are named by path hash, so you can copy them into the new folders to avoid regenerating them.

Only the sequence and video subfolders that previously collided on name should be allowed to regenerate, because those may already contain cross-contaminated data.

Organizing your cache folders (Libraries and Templates). Crate keeps a separate cache for every library and every template category you configure, each one has its own cache path setting. Take advantage of this: create one parent folder for all Crate caches, then a dedicated subfolder per entry, named to match. A clean layout looks like this:

Crate Caches/

├── **Libraries/**

│ ├── Smoke/

│ ├── Fog/

│ ├── Explosions/

│ └── HDRIs/

└── **Templates/**

├── Keying/

├── Grain/

└── CopyCat/

So the library "Smoke" points its cache path at `Crate Caches/Libraries/Smoke`, and the template category "Keying" points at `Crate Caches/Templates/Keying`.

Avoid pointing two libraries or two template categories at the same cache folder.

Delete Thumbs Cache: Sits beside the Cache Path field. It deletes every cached preview inside this library's cache folder, flat thumbnails, the sprite-sheet folders used for sequence and video hover animation, and any temporary preview files, while keeping the cache folder itself.

Crate then regenerates previews automatically the next time the library is browsed. Because this is destructive to the cache (though never to assets), it requires two confirmations: a first prompt, then a final warning. Reach for it when previews look stale or a source has changed underneath Crate.

Type: Sets how Crate previews and groups the contents of this library. There are four options:

- **standard:** General assets such as still images, 3D models, and Gaussian splats. Each file becomes its own card with a single thumbnail.
- **video:** Video files. Each card shows a representative "best frame" and plays a sprite-sheet animation on hover.
- **sequence:** Frame-numbered image sequences. Crate groups the frames of a sequence into a single card (rather than one card per frame), again with a best frame and hover animation.
- **mixed:** A library that contains a combination of the above. Crate detects and handles each item or folder by its actual content. Choose this for catch-all libraries; choose a specific type when a library is uniform, as it is the most predictable.

Folder-level controls

These let a single library behave differently for specific subfolders, so you rarely need to split one source into several libraries.

Folder Overrides: Override the library's Type for a named subfolder. Type a folder name, pick the type for it (**standard / sequence / video / mixed**), and press **+ Add Override**; each override appears in the list as `folder > type`, and **Delete** removes the selected one. The classic use is a subfolder that needs different processing from its parent, for example a folder of image sequences living inside an otherwise "standard" library, or vice-versa.

Folder Hides: Hide specific subfolders of this library from the browser. Type a folder name and press **+ Add Hide**; hidden folders will not appear in Crate's grid for any user, and **Delete** removes a hide. This is a per-library complement to the global Hide Prefixes section: Hide Prefixes hides by name pattern everywhere, while Folder Hides removes named folders from one library only.

On disk

Each library is stored in `crate_libraries.txt` as a single pipe-delimited line:

```
name | asset_path | cache_path | visible | enabled | type
```

where `visible` is the "Show Path in Dropdown" flag and `enabled` is the Enabled flag. Any folder overrides and hides for that library follow it on their own lines:

```
FOLDER_OVERRIDE | folder_name | type
FOLDER_HIDE | folder_name
```

The file is human-readable and edited safely through this panel; the per-section edit lock prevents two Curators from changing it at once.

11.6 Settings Sections - Templates (in depth)

Where a library points Crate at a folder of *assets*, a template points it at a folder of *Nuke setups*. A template is a `.nk` script, a pre-built node graph (a show sequence comp lead script, a standard slate, a comp skeleton, and so on) that a user can browse in Crate's Templates mode and drop straight into their script. This section defines those folders, controls who can see them, and manages individual scripts that have been hidden. It is Curator-only and is backed by two files: `crate_templates.txt` (the category definitions) and `crate_templates_data.txt` (the hidden-script list).

The guiding idea is simple: **each top-level folder you add here becomes one browsable category** in Templates mode, and the `.nk` scripts inside it become the template cards.

The category list

The upper panel lists every template category as `[ON] name > folder path` or `[OFF] name > …`, where the prefix reflects that category's "Visible to Visitors" state. Select a row to edit it.

+ Add Template creates a new category entry to fill in; **Delete** removes the selected *category definition*. As with libraries, deleting a category here removes only Crate's pointer to it, the folder of `.nk` scripts on disk is untouched.

Available to Visitors (module-level)

At the top of the section is a single **Available to Visitors** ON/OFF toggle. This is the master Visitor gate for the entire Templates feature (stored as `TEMPLATES_ENABLED`): when OFF, Visitors do not get Templates mode at all, regardless of the per-category settings below; Curators always have it. Think of it as the outer switch, with each category's own visibility as the inner switch.

Per-category settings

Selecting a category exposes:

Display Name: The category's name as shown in Crate's Templates picker. Unlike library names, this is a display label.

Folder Path: The folder on disk that holds this category's `.nk` scripts. Crate reads the scripts from here; the **…** button opens a folder browser. This is the source of the category, and as with assets Crate does not rewrite it behind your back, new templates are added by publishing from Nuke (see below) or by dropping `.nk` files into the folder.

Cache Path: Where this category's thumbnails are stored; the **…** button browses for it. This matters because **template thumbnails are not auto-generated**. A template has no inherent image, so until a Curator sets one (via the edit/pencil button on the thumbnail while browsing Templates), the card shows a neutral "NK" placeholder of “Foundry´s Hobbit” (can be changed in crate_media\placeholders\.nk). The image a Curator chooses is written here, keyed to the template's name. Because these are deliberately chosen images rather than rendered previews, clearing this cache means the cards fall back to the placeholder until the images are set again; it does not regenerate them automatically.

Visible to Visitors: A per-category checkbox. When unchecked, this specific category is hidden from Visitors even while the module is available to them. This gives you category-by-category control: you might expose a "Slates" category to everyone but keep an "R&D" category Curator-only. This is the `[ON]/[OFF]` state shown in the list.

Hidden Templates

Individual template scripts (not whole categories) can be hidden from the grid by a Curator via right-click/Hide From Grid while browsing. The **Hidden Templates** list at the bottom of this section is where those hidden scripts are managed for the selected category. Each entry shows the script filename and its folder. The list supports multi-select, with two actions:

Un-Hide Selected: Returns the selected scripts to the grid. Note that their thumbnail must be re-created afterward (un-hiding restores visibility, not the previously chosen image).

Delete Selected: Removes the script for good *from Crate's point of view*, but safely: it **moves the actual `.nk` file into a `_Crate_Data/Templates_Bin` folder** beside the category (and removes its thumbnail), exactly mirroring how asset and Discover deletion move files to a recoverable bin rather than erasing them. The file can be recovered from that bin in the file explorer; Crate never hard-deletes a template script.

On disk

The category definitions live in `crate_templates.txt`:

```
TEMPLATES_ENABLED | true|false
 TEMPLATE | display_name | folder_path | visible | cache_path
```

(one `TEMPLATE` line per category; `visible` is the per-category "Visible to Visitors" flag, `cache_path` may be blank if no thumbnails have been set yet).

The hidden-script list lives separately in `crate_templates_data.txt`:

```
HIDDEN | template_folder_path | filename.nk
```

Both files are human-readable and edited safely through this panel under the section's edit lock.

How templates get created

Templates aren't authored in Settings, this section only *registers and governs* the folders. New templates are published from the Nuke node graph: a user selects nodes and uses **Add to Templates**, which writes a new `.nk` into the chosen category folder (this is one of Crate's two intentional, user-initiated writes into a source tree). Whether Visitors are offered that publish action is controlled separately by the **VISITOR_MENU_TEMPLATES** flag in the Nuke section. So the full picture spans three places: this section defines the categories and visibility, the Nuke section decides who may publish into them, and the thumbnail edit button (while browsing) sets each card's image.

11.7 Settings Sections - Discover (in depth)

Discover is Crate's second browsing mode and the counterpart to Libraries. Where a library is a curated, named source with its own cache and type, **Discover points Crate straight at live folders**

so users can browse a tree as-is, no library definition, no separate cache configuration. Each folder you register here appears as a browsable in the Discover module. This section is Curator-only and is backed by `crate_discover.txt`. Discover is also a media management, transcoding, object transformation and ingestion system that speeds up processes.

Two facts frame everything below: **Curators always have full Discover access**, so every toggle in this section governs *Visitors only*; and Discover is where Crate's "live" file operations and one of its two source-tree writes happen, so its permissions are a real part of how the pipeline is protected.

The two Visitor toggles (top of the section)

Enable Discover module to Visitors: Whether Visitors get Discover module at all. (Curators always do.)

Enable File Operations to Visitors: Whether Visitors who have Discover access may perform file operations within it, rename, new folder, cut, copy, paste, and delete-to-bin. This is independent of the access toggle: a Visitor can be allowed to *browse and import* from Discover while still being blocked from *modifying* anything. Curators always have file operations.

Visitor Access list (the precedence rule)

Below the toggles is the **Visitor Access** allowlist, a list of specific Visitor usernames, with an "Enter Visitor username…" field plus **+ Add** and **Remove**. This list interacts with the module toggle by a precedence rule that's worth stating exactly, because the list *overrides* the toggle:

- **If the list is empty:** the module toggle decides. On > all Visitors get Discover; Off > no Visitors do.
- **If the list has any names:** only those named Visitors get Discover, **regardless of the toggle**. Everyone else is blocked even if the toggle is On.

In other words, adding names switches Discover from "all-or-nothing for Visitors" to "these specific Visitors only." (Curators are never affected by either setting.)

Discover Setup (the Branches folders)

The lower panel is the list of tree folders/branches, each shown as `[ON] path` or `[OFF] path`, where the prefix is that branches "Visible to Visitors" state. Selecting a row exposes:

Path: The folder on disk that becomes a browsable Discover folder. The **|…|** button opens a folder browser. Crate reads this tree to display and import from it; it does not rewrite it except for the two explicit, user-initiated actions noted below.

Visible to Visitors: A per-tree folder checkbox. Unchecked hides this specific folder from Visitors who otherwise have Discover access, so you can expose most of the tree while keeping one folder Curator-only. This is the `[ON]/[OFF]` shown in the list.

At the bottom, an "Enter folder path…" field with **+ Add Project** registers a new folder, and **Delete** removes the selected folder's registration (the folder on disk is untouched, it just stops appearing in Discover).

Deletion and the recycle bin

When a project is registered, Crate auto-creates a `_Crate_Data/Discover_Bin` folder inside that project's root. Deleting an asset from Discover (a Curator action, or an allowed Visitor when file operations are on) **moves the file into that bin** rather than erasing it, the same recoverable-deletion pattern used by Libraries (`Library_Bin`) and Templates (`Templates_Bin`). Permanent removal is a deliberate, separate step in the file explorer.

Publishing into Discover

The other write Crate makes into a Discover tree is publishing: from the Nuke node graph, **Add to Discover** writes a new `.nk` setup file into the current project folder. Whether Visitors are offered that action is governed separately by the **VISITOR_MENU_DISCOVER** flag in the Nuke section, so as with Templates, the full picture spans this section (who can access and modify folders) and the Nuke section (who can publish into them).

On disk

`crate_discover.txt` uses these keys:

`SHOW_ENABLED | true|false:` the "Enable Discover module to Visitors" toggle
`SHOW_FILE_OPS | true|false:` the "Enable File Operations to Visitors" toggle
`SHOW_VISITOR | username:` one line per allow-listed Visitor
`SHOW | project_folder_path | visible:` one line per project (`visible` = "Visible to Visitors")

These keys are prefixed “`SHOW_`” because, as noted in the naming box of the appendix, "Discover" is named **show** at code level, the keys and the module are the same thing.

11.8 Settings Sections - Camera Database (in depth)

The Camera Database is Crate's reference library of cameras and their sensor geometry. Each camera carries identity information plus one or more **sensor modes**, the full sensor surface and any

cropped formats derived from it, so the rest of the pipeline can reason about resolution and physical sensor dimensions consistently. It is Curator-only and is stored as a single JSON file at `crate_camera_database/crate_cameras.json`. Unlike the plain-text config files used elsewhere in Settings, this is structured JSON, which is why it has dedicated import/export tools rather than free-form editing.

Selecting and managing cameras

At the top, the **Camera** dropdown selects which camera you're editing. Beside it, a small camera icon with a number shows the **total camera count** in your database (a quick sanity check after an import).

Three buttons manage the roster:

+ New creates a blank camera ready to fill in. **Duplicate** copies the currently selected camera, the fast way to add a variant of an existing body without re-entering every sensor mode. **Delete** removes the selected camera from your local database.

Identity

The **Identity** block holds the descriptive fields, all free text:

Name: The camera's display name (e.g. a body or format name). This is also what the dropdown shows.
Manufacturer: The maker or, for film formats, the format family.
Sensor Type: e.g. CMOS, CCD, or Film.
Shutter: e.g. Rolling, Global, or Mechanical.
Camera Type: A short classifier (e.g. cinema, film, mirrorless).
Reference URL: Optional link to a spec sheet or manufacturer page, kept with the record for reference.

Sensor Modes

This is the heart of a camera record: a table of sensor modes, with columns **Label**, **Res W**, **Res H**, **Width (mm)**, and **Height (mm)**.

The rule that governs the table is stated in the on-screen help and is important: **the first row defines the camera's full sensor surface (open gate) and is the reference frame for every crop below it.** You enter the open-gate resolution and physical dimensions first, then list each cropped mode beneath it. Width and Height are in **millimetres**; pixel resolution (Res W / Res H) is **optional, enter 0 for film**, where there is no pixel grid. Because of its special role, the first row is visually tinted as the reference.

Four buttons manage the rows: **+ Add Mode** appends a new mode, **Remove** deletes the selected one, and ↑ **Up** / ↓ **Down** reorder rows, useful for keeping the open-gate row at the top and arranging crops in a sensible order.

Import / Export

Because the database is a shareable JSON file, the bottom of the panel offers three actions.

The key design principle across all of them is that **importing never overwrites cameras you already have**, your local edits are always preserved.

Import JSON…:

Loads a local `crate_cameras.json` file from disk and **adds only the cameras you don't already have**. Cameras already in your database are left completely untouched, so any local edits (renames, sensor adjustments, custom modes, URL notes) are preserved by design. To replace an existing camera with the imported version, **delete your local copy first, then import**, the importer will then add the incoming one.

Import JSON from CrateTools…:

Does exactly the same adds-only merge, but instead of reading a local file it **downloads the latest official database directly from the CrateTools online repository**. This is how you pick up newly released cameras without waiting for a new Crate build. Before downloading it reminds you to export a backup first, and if you have unsaved edits it offers to save them before proceeding.

Export JSON…: Saves a copy of your current database to any location you choose. This is your backup mechanism, and the tool explicitly recommends using it before any import.

How Crate uses the CrateTools repository

The official camera database is maintained by CrateTools at **https://github.com/CrateTools/CrateCamDB**, and "Import JSON from CrateTools…" is Crate's bridge to it. When you click it, Crate fetches the raw file directly from the repository's main branch:

```
https://raw.githubusercontent.com/CrateTools/CrateCamDB/main/crate_cameras.json
```

The download uses a standard library HTTP request (no extra dependencies) with a **5-second timeout**, so a flaky or blocked network fails quickly rather than hanging Nuke. The downloaded list is then merged into your local database with the same **adds-only** rule described above: any camera whose entry you don't yet have is added; everything you already have, including your edits, is left alone. On save, Crate recomputes the database's camera count and last-updated metadata.

If the workstation can't reach the internet from inside Nuke (a common studio situation), the download simply reports a clear error and points you to the **manual fallback**: download `crate_cameras.json` yourself from **https://github.com/CrateTools/CrateCamDB** and load it with **Import JSON…**. The two paths produce the same result, one fetches the file for you, the other lets you supply it.

Backing up before an update (recommended workflow)

Because imports are additive and can't, on their own, undo a rename or a sensor tweak you later regret, keep a backup before any import. The safest routine, using only the buttons in this panel:

1. **Before importing, click Export JSON…** and save a dated copy somewhere safe, e.g. `crate_cameras_backup_2026-06-22.json`. This is a complete snapshot of your current database.
2. **Then run your import** (either "Import JSON from CrateTools…" or a local "Import JSON…"). New cameras appear; your existing ones are untouched.
3. **If you need to undo:** to bring back a camera you deleted, just **Import JSON…** your backup, the adds-only merge re-adds anything missing. To roll back *edits* to a camera you still have, first **Delete** that camera locally, then **Import JSON…** the backup so the backed-up version is re-added in its place.
4. **For a full revert**, the most direct route is at the file level: with Crate closed, replace `crate_camera_database/crate_cameras.json` with your backup copy. Because import won't overwrite existing cameras, swapping the file on disk is the cleanest way to restore the entire database exactly as it was.

A good habit is to Export before every CrateTools update and keep the last few dated exports; they're small JSON files and make any update completely reversible.

On disk

`crate_cameras.json` has two top-level keys:

- `_metadata:` version, last-updated, camera count, and credits (maintained automatically).
- `cameras:` an object keyed by a slug per camera, each holding `name`, `manufacturer`, `sensor_type`, `shutter`, `cam_type`, `url`, and `sensor_dimensions` (the list of sensor modes, each with `label`, `res_w`, `res_h`, `width_mm`, `height_mm`).

Any extra fields present from an official import that aren't edited in this panel are preserved on save, so records round-trip cleanly. The file is edited safely through this section under the standard per-section edit lock.

Part III — Processing Panels in Discover Module

12. Processing Panels Overview

How the convert panels work

Most of the conversion panels share one interaction model, so it's worth learning once. You bring files in as **jobs**, set your conversion options on the left, and then send those settings to a **queue**. Pressing **Apply to Queue** stamps the current settings onto the selected jobs; the queue header updates to show how many jobs are waiting (e.g. *Queue (12)*), and **Clear All** empties it. When you're ready, **Convert Queue** runs every job in turn, reporting progress in Crate's status line as it goes. Because settings are applied per batch, you can queue one group of files at one setting, change the options, and queue a second group at another, then convert everything in a single pass.

Each panel's queue is saved continuously to a file in your Crate user directory, so it survives a Nuke crash or a normal close, reopen the panel and your queue is exactly as you left it.

The conversions themselves run outside Nuke wherever possible, as direct subprocess calls to the bundled engines, which keeps them fast and stops them from blocking your Nuke session. The engines live in Crate's Engines directory and are auto-detected; a panel will tell you up front if the engine it needs isn't found. Two panels (W-4MAT and W-GEO) are the exception and use Nuke's own nodes, since reformatting and 3D I/O are best handled by Nuke itself.

Every converter lets you choose where output lands. Leave the output folder blank to write next to each source file, or set a folder to collect results in one place. An optional suffix (for example `_proxy` or `_h264`) is appended to each output name so conversions never overwrite their sources.

12.1 W-EXR (EXR Writer)

io:VIDEO/SEQUENCE>>>EXR SEQUENCE

W-EXR builds a real Nuke node chain “Read > optional rescale > Write” behind the scenes, with the Write node locked to EXR.

Source. You point W-EXR at a source with **Read From**, and it handles both video files and image sequences. **Detect** works out the frame range for you, reading it from the container for a video (using a fresh Read node so Nuke's codec metadata initializes fully) or scanning the disk for a sequence.

You can then confirm or override the **Frame Range**, tell it what the **First Frame Is**, and set the **Frame Pad** for the output numbering.

Input Transform. Because clean EXR delivery depends on interpreting the footage correctly, W-EXR exposes an OCIO-aware **Input Transform**. The list is pulled live from your active colorspace/OCIO config and defaults to *raw*, so you can declare exactly how the source should be read before any output transform is applied.

ReScale. Between Read and Write you can optionally insert one rescale node. **TVIscale (2x)** and **Reformat** are offered as mutually exclusive options, and each can be opened in Nuke for configuration - *Show TVIscale in Nuke Properties* or *Show Reformat in Nuke Properties* - so you set up the real node and W-EXR applies it to the render. Leave both off to write at source resolution.

EXR Settings. Specific Write knobs, surfaced in order and populated with the node's live values. You get channels, output transform, **datatype** (half or float), **compression** and its DWA **compression level**, autocrop, metadata, the *no Nuke prefix* option, the layer-naming controls (standard layer names, full layer names, truncate channel names), interleave, write hash, first part, hero view, and render order. For anything not surfaced here, *Show Write Node in Nuke Properties* opens the underlying node (recommended before running a large job queue just to make sure everything looks good).

Output. W-EXR builds the output path for you and previews it as you work. You choose whether **all outputs go in a single folder** or **each output gets its own folder**, set the **Folder** and an optional **Subfolder** (with *Use same name as subfolder* to keep names aligned), and add a **File Name**, **File Suffix**, or **Folder Suffix**. The **Output Path** preview shows exactly what will be written before you commit.

Queue and render. W-EXR uses the same queue model as the other converters: **Add to Queue** or **Apply to Queue** to stage jobs, **Edit** and **Remove** to manage them, **Clear All** to empty the queue, and **Render Queue** to process everything. The status line tells you what's still missing if a source or output path hasn't been set.

12.2 W-EXR-Q (Batch EXR Queue)

io:VIDEO/SEQUENCE>>>EXR SEQUENCE

W-EXR-Q is the batch counterpart to W-EXR. Where W-EXR is built around one interactive source, W-EXR-Q is built for volume: select multiple files or folders in Discover, send them all to the queue in a single action, and apply one shared set of settings to every job at once. It's a separate panel that leaves W-EXR untouched, reach for W-EXR when you're dialing in a single delivery, and W-EXR-Q when you have many that should all be written the same way.

The controls are the same ones W-EXR exposes, the **Input Transform**, the **ReScale** stage (TVIscale 2x or Reformat), the **EXR Settings** knob set, and the **Output** location, suffix, and frame-pad fields, but here they act as *batch settings*: whatever you set is applied uniformly across all queued jobs rather than to a single source. **Apply to Queue** pushes the current settings onto every job in the list.

The control unique to the batch panel is **Group Range**. Leave it empty and each job keeps its own original detected frame range; fill it in and that range is forced across the whole group. Individual

jobs can still be adjusted with **Edit**, dropped with **Remove**, or cleared together with **Clear All**, and **Render Queue** writes them all in one pass.

W-EXR-Q is launched from the **W-EXR-Q** action on a multi-selection in Discover, which collects every selected path into the queue at once.

12.3 W-MOV (MOV/MP4 Writer)

io:SEQUENCE>>>MOV/MP4

W-MOV is the video-delivery sibling of W-EXR. It shares the same shape, a “Read > optional rescale > Write” chain built in Nuke, the same **Source** and **Detect** frame-range handling, the OCIO-aware **Input Transform**, the **ReScale** stage (TVIscale 2x or Reformat), and the same **Output Path** builder and queue, but the Write node targets a movie container instead of EXR.

A **Container** toggle switches between **MOV** and **MP4**.

What's specific to W-MOV is the codec-side knob set it surfaces: **Codec** and **Codec Profile** (including the ProRes profiles), **FPS**, **Quality**, **YCbCr Matrix**, **Data Range**, **Fast Start** for streaming, **Write Timecode**, and an **Audio File** with **Audio Offset** for laying sound against the render, alongside the output transform, premultiplied, and raw options. As with W-EXR, *Show Write Node in Nuke Properties* opens the underlying node for anything not surfaced (recommended before running a large job queue just to make sure everything looks good).

Use **Add to Queue** and **Render** for a single source, or stage several and use **Render Queue**.

12.4 W-MOV-Q (Batch MOV/MP4 Queue)

io:SEQUENCE>>>MOV/MP4

W-MOV-Q is to W-MOV what W-EXR-Q is to W-EXR: the batch version. Send multiple sources to the queue in a single action and apply one shared set of MOV/MP4 settings (container, codec and profile, FPS, quality, and the rest) uniformly across every job with **Apply to Queue**. It carries the same **Group Range** behavior (leave it empty to keep each job's own range, or set a value to force a shared range on the group) and the same **Output Location** choice between one shared folder and a folder per output. **Edit**, **Remove**, and **Clear All** manage the list, and **Render Queue** writes everything in one pass. It's launched from the **W-MOV-Q** action on a multi-selection in Discover.

12.5 IMG-C (Image Conversion)

io:STILL IMAGE>>>STILL IMAGE

IMG-C is the single-image converter. It works on standalone image files on disk, no Nuke nodes involved, using OpenImageIO's `oiiotool` as the primary engine and ImageMagick as a fallback.

You choose an **output format** from EXR, PNG, JPG/JPEG, TIFF, DPX, HDR, TGA, BMP, or PSD, and a **bit depth** of 8, 16, or 32-bit. The panel understands which depths each format can actually

hold and steers you accordingly: JPG, TGA, and BMP are 8-bit only; PNG, TIFF, and DPX cover 8 and 16-bit; EXR and HDR carry 16 or 32-bit float. The default is 16-bit as the highest-quality common ground.

A **colorspace** menu handles conversions through an OCIO-aware matrix. You can leave it at *No conversion (raw)*, use one of the *Guess >* modes to auto-detect the input based on file type and convert to a chosen target, or pick an explicit transform between sRGB, Linear (sRGB), ACEScg, ACEScct, Rec.709, and ACES2065-1. An optional **Resize to %** scales every image proportionally on the way out.

Set an output folder and suffix, click **Apply to Queue**, then **Convert Queue**. Each job shows its bit-depth tag in the queue so you can confirm at a glance what each batch will produce.

12.6 SEQ-C (Sequence Conversion)

io:IMAGE SEQUENCE>>>IMAGE SEQUENCE

SEQ-C is IMG-C's counterpart for image sequences. Where IMG-C treats each file as a job, SEQ-C treats each *sequence* as a single job and loops through its frames with per-frame progress, preserving the frame numbering throughout. It runs as pure subprocess work against `oiiotool` (primary) or ImageMagick (fallback) for maximum speed across long sequences.

It carries the same format, bit-depth, colorspace, and resize controls as IMG-C, and adds the controls a sequence needs. A **Frame Range** section lets you confirm or set the range and tell Crate what the *first frame is*, so numbering stays correct. An **Output Frames** section controls zero-padding, with a **Pad** field that previews as *match source* until you override it. **Auto-folder per sequence** drops each converted sequence into its own subfolder, which keeps large multi-sequence batches tidy rather than interleaving thousands of frames in one directory.

12.7 VID-C (Video Transcoding)

io:VIDEO>>>SEQUENCE/VIDEO

VID-C is Crate's video transcoder, built directly on FFmpeg. It converts video to video and assembles image sequences into finished movies, and because it drives FFmpeg as a direct subprocess rather than building a Nuke graph - no node tree to cook, no OCIO config to load - it is fast, staying responsive even across long sequences. The trade-off defines where it fits: VID-C owns the standard, mathematically defined colour spaces (Rec.709, sRGB, scene-linear, the Rec.2020 HDR transfers) that FFmpeg's `zscale` converts deterministically, and deliberately leaves camera logs (S-Log3, LogC) and ACES to **W-MOV**. The rule of thumb: standard spaces, fast and Nuke-free, use VID-C; camera-log or ACES colour science, use W-MOV.

The Settings tab is split into **Data In** (how many sources are loaded and how to interpret the input **pixel aspect**) and **Data Out**, which holds **Codec** (format plus a CRF **Quality** menu for H.264/H.265/VP9; ProRes and DNxHR carry their own), **Video** (resolution, frame rate, pixel format, output pixel aspect), **Audio** (copy, strip, or re-encode), and **Destination** (output folder and suffix).

Its standout is the **Colour** model, *Interpret* > *Deliver*. **Interpret As** says how to read the incoming pixels (auto, Rec.709, sRGB, Linear, or Raw); **Deliver As** says what to write (Rec.709, sRGB, HDR10, HLG). When the two match (a 709 master to 709, or anything marked *Raw*) VID-C writes only metadata and leaves the pixels untouched. When they differ, a scene-linear EXR to 709, sRGB stills to 709, linear to HDR10, it actually converts through `zscale`, fixing transfer, primaries, matrix and range.

This is what stops the classic mistake of tagging washed-out log or linear footage as "Rec.709" and shipping a flat result: VID-C touches pixels only when the spaces differ. *Auto* suits tagged video; image sequences carry no metadata, so set their interpret space explicitly (*Linear* for EXRs, *sRGB* for stills). Choosing HDR automatically moves you to a 10-bit codec, and SDR>HDR up-conversion is refused, since that is a grade, not a transcode. (Conversion needs the bundled FFmpeg to include `zscale`/libzimg; tagging works without it.)

A live **Output Preview** lists every source and its exact output name, and the **Queue** tab tags each job with its output range and bit depth *[1001-1096 10-bit]* for a sequence, *[1920x1080 8-bit]* for a video. Output names are collision-safe: identically named sequences from different folders sent to one output folder become `render.mp4`, `render_1.mp4`, `render_2.mp4` rather than overwriting one another.

12.8 W-GEO (Geometry Conversion)

io:Geo USD/CLASSIC>>>Geo USD/CLASSIC

W-GEO is a batch geometry converter that uses Nuke's built-in 3D systems, so it needs no external engine. It offers two pathways depending on which 3D system you target.

Classic 3D uses Nuke's ReadGeo2 and WriteGeo nodes and moves between OBJ (Wavefront), FBX (Autodesk), and Alembic. **USD 3D** uses the GeoImport and GeoExport nodes available in Nuke 16 and later, and works across Alembic and the USD family (binary `.usdc`, text `.usda`, and `.usd`) including reading Alembic into the USD pipeline and writing Alembic back out through it. You pick the **output format** from a single list that spans both pathways; Crate routes each job through the correct node set based on your choice and the input file's type.

12.9 W-4MAT (Batch Reformat)

io:VIDEO/SEQUENCE>>>REFORMATTED SEQUENCE

W-4MAT is a batch reformatter for image sequences, and it is deliberately narrow: it is a pure transform operation built on Nuke's **Reformat** node, with no color or colorspace science involved. Its node chain is simply Read > Reformat > Write.

Rather than reinventing Reformat's controls, W-4MAT lets you configure the real thing. Use **Show Reformat in Nuke Properties** to open a Reformat node, dial in exactly the resize, format, filter, and

centering behaviour you want, and those settings are then applied to every job in the queue. This means anything Nuke's Reformat can do, W-4MAT can do in batch. **Use source frame range per file** processes each file across its own range, and **Auto-folder per video file** keeps each result separated into its own folder.

12.10 CONTACT (Contact Sheet Generator)

io:IMAGES>>>CONTACT SHEET

CONTACT generates contact sheets from a set of images using ImageMagick's `montage`. You select your images, choose a grid and a resolution, watch a live preview update, and generate a publication-ready sheet.

The **Grid** section sets columns, rows, and padding with spin controls, and a lightweight drawn preview shows the layout (just the cells, not the images) so you can compose quickly. **Resolution** offers HD (1920×1080) for email and quick review, 4K (3840×2160) for client presentations and prints, and 6K (6144×3456) for high-res catalogs and wall prints. A fit menu decides how each image sits in its cell, *Fit (letterbox)* to show the whole frame, or *Fill (crop)* to fill the cell, and **Show filenames as labels** captions each image at Small, Medium, or Large size. When you have more images than the grid holds, CONTACT automatically splits the overflow into numbered pages rather than cramming or dropping frames. Output defaults to a `contactsheet` name in the format you choose.

12.11 BATCH (File Renaming)

io:FILES>>>RENAMED REFORMATTED

BATCH is a filesystem-level batch renamer written in pure Python, no engine, no Nuke nodes. It renames with `os.rename`, so changes are instant, and it shows a live preview with collision detection before you commit.

It offers six operations that you can combine, applied in a fixed order so the result is predictable:

1. **Replace:** find and replace text, with an **Aa** toggle for case sensitivity.
2. **RegEx:** pattern-based find and replace for anything the simple replace can't express.
3. **Remove:** strip characters by position (First N / Last N) or by class: Digits, Symbols, or Spaces.
4. **Add:** insert a prefix, suffix, or text at a position.
5. **Case:** recase to UPPER, lower, or Title, or leave unchanged.
6. **Numbering:** append a sequential number as a prefix, suffix, or full replacement. Configure Start, Increment, Padding, and Separator.

Reset Numbering Per Folder

When multiple folders are selected, Numbering normally counts continuously across all files (1 through N). Enabling **Reset Numbering Per Folder** restarts the counter from Start for each folder, so every folder gets its own independent sequence.

Options

Below the six operations, the **Options** section contains settings that apply to the entire batch:

- **Protect frame numbers** keeps trailing frame padding intact, which matters when you're renaming sequences and don't want the rename operations to clobber the frame counter. For example, `shot_0042.exr` - the `0042` is peeled off before any operation runs and reattached at the end, so Replace, Remove, and Case can't accidentally destroy it.

- **Apply changes to Selected Folders too** extends the rename to the folders themselves, not just the files inside them.(The Discover branch root folder is never renamed.)

Preview and Collision Detection

The preview updates live as you type or toggle any option. Files that will change show their new name in yellow; files that won't change are dimmed. If two files end up with the same name, both are highlighted in red, a collision warning appears, and the Rename button is disabled, you can't accidentally overwrite one file with another.

When the "Apply changes to Selected Folders too" checkbox is on and folders are present, a **Folders -** separator appears below the file list, followed by folder entries in blue showing old name → new name.

Progress Bar

A progress bar is pinned at the bottom of the panel.

Buttons

- **Rename N Items** applies the changes shown in the preview. Disabled when there are no changes or when a collision is detected.
- **Clear** resets all fields, checkboxes, and the file/folder lists back to defaults.
- **Undo** reverts the last rename batch. Multiple undo levels are supported (one per Rename click). Folder renames undo before file renames.
- **Close** closes the BATCH panel.

12.12 SPLAT-C (Gaussian Splat Converter)

io:GAUSSIAN SPLAT>>>GAUSSIAN SPLAT

In Crate 2, this is the most experimental panel of all at the moment.

Batch Gaussian Splat format converter using splat-transform by playcanvas, Node.js CLI, Splat by antimatter15 and 3dgsconverter by Francesco Fugazzi.

SPLAT-CONVERT reads PLY, Compressed PLY, SOG, SPZ, SPLAT, KSPLAT, and LCC, and writes PLY, Compressed PLY, SOG, SPZ, KSPLAT and GLB.

Beyond format conversion it exposes the cleanup and optimization controls splats usually need. **Filter NaN / Inf values** strips invalid points. A spherical-harmonics control lets you keep all bands or truncate to band 0 (no view-dependent color) or up through bands 1, 2, or 3, which is the main lever for trading file size against view-dependent quality. **Decimate to %** thins the splat count, **Min opacity %** drops near-transparent points, and **Spatial reorder (Morton)** improves locality for streaming (the panel notes when this is redundant, as it is for SOG output). A coordinate-system menu applies axis rotations (for example *Rotate −90 X* to go from Z-up to Y-up) and for SOG output a compression-level menu chooses between Float32 (lossless), an f16 block, and a u8 SH block.

The engine is configured in `crate_engines.txt` via the `SPLAT_TRANSFORM` entry (and an optional `NODE_PATH`), or by installing `@playcanvas/splat-transform` into the Engines directory.

PLY (standard): Keep as your master archive; only convert from this when you need delivery, never convert into this from other formats unless you're re-exporting from a training pipeline.

PLY (compressed): Convert to this only when you need moderate file reduction but must stay inside PLY-compatible tools; otherwise skip it for better options.

SPLAT (antimatter15): Tiny, universally-loadable web format. No spherical harmonics (flat colour). Lightweight web previews where view-dependent colour isn't needed.

SOG (super-compressed): Convert to this if you're delivering exclusively through PlayCanvas/SuperSplat, especially for mobile or memory-constrained devices.

SPZ (Niantic): Convert to this as your default go-to delivery format for broadest compatibility, web/AR use, and glTF integration.

GLB (glTF binary): Convert to this when your splat needs to sit inside a larger 3D scene alongside meshes, animations, or other glTF content.

KSPLAT: The GaussianSplats3D three.js viewer. Spatially bucketed, SH up to band 2, with a selectable compression level (see Options).

Filter NaN / Inf values: Filter NaN / Inf values - Checkbox.

Runs a cleanup pass that removes broken splats. During capture or training a few Gaussians can end up holding invalid numbers ("NaN" (not-a-number) or infinity) usually from a bad calculation upstream. In a viewer these appear as black blotches, flickering, or missing patches of the scene. Switching this on quietly drops those bad points so everything else renders cleanly. If a file looks glitchy, try this first, it's safe and only removes the corrupt splats, nothing good.

SH Bands: Dropdown (Keep all · Band 0 only · Up to band 1 · Up to band 2 · Up to band 3).

Controls how much view-dependent colour the splats keep. "Spherical harmonics" (SH) are the extra data that let a splat shift colour as you move around it, the subtle highlights, reflections, and shading changes that make a scene feel real instead of flat. More bands look better but make larger files; fewer bands are smaller and faster but flatter.

Keep all: full quality, largest file.

Band 0 only: flat colour, no view-dependent shading (smallest).

Up to band 1 / 2 / 3: progressively more shading detail and file size.

Most web viewers don't need high bands. Note that the KSPLAT format supports up to band 2, and the SPLAT format has no SH at all.

Decimate to % : Checkbox + spinner.

Thins the scene to a target percentage of its splats, for example, 50% keeps roughly half. Fewer splats means smaller files that load and render faster, at the cost of fine detail (edges and textures soften). Useful for web previews, lightweight versions, or heavy scenes that are slow to display. The splats removed are chosen at random across the scene. (This step is processed on the graphics card, so it needs a GPU-equipped machine.)

Min opacity: Checkbox + spinner.

Removes nearly-invisible splats. Captures often carry a faint "haze" of semi-transparent Gaussians that add file size and slightly muddy the image without adding much. This drops any splat whose opacity is at or below the percentage you set (true 0–100% transparency). Around 5–10% is a gentle clean-up; 20–30% is stronger and crisper but can thin soft edges. Off by default.

Spatial reorder (Morton): Checkbox.

Re-sorts the splats so points that are close together in space are also close together in the file (along a "Z-order" curve). This speeds up rendering and helps the file compress better, especially worth enabling for compressed KSPLAT (levels 1–2). It changes only the storage order, never how the scene looks.

Up-axis fix: Dropdown.

Rotates the whole scene to correct orientation problems. Capture and training tools disagree about which way is "up" (some Y-up, some Z-up), so an imported scene can arrive on its side or upside-down. The presets apply a corrective rotation, and the colour/shading data is rotated correctly along with it.

KSPLAT level: Dropdown (0 Float32 · 1 f16 block · 2 u8 SH block).

Compression level used only when exporting to KSPLAT (ignored for other formats):

- 0: Float32: no compression, best quality, largest file (default).
- 1: f16 block: stores positions and scales at half precision, roughly half the size.
- 2: u8 SH block: also compresses the colour/shading data to 8-bit, smallest file.

For levels 1 and 2, turning on Spatial reorder (Morton) is recommended for the best result.

12.13 SCRIPT (1 comp fanned across many inputs)

io:SCRIPT>>>MULTI SEQUENCE/VIDEO

Input scanning now shows progress

(Settings tab > Input)

When you choose an Input folder, SCRIPT has to inspect every item to determine its frame range. For image sequences this is quick, but video files, particularly large formats such as ProRes 4K and above, have to be opened to read their true range, which can take a few seconds each.

SCRIPT now shows a **progress dialog** while it scans, with a bar and a "Scanning N of M" label naming the current file. This does two things: it tells you the tool is working rather than frozen, and it keeps the application responsive so the operating system does not display its "busy"/spinning-wheel indicator. The dialog appears only when there is real work to do (video inputs, or a larger batch); a couple of quick sequences will not trigger it.

If a folder contains many heavy video files, expect the bar to pause briefly on each one as it is opened, that is normal, and the count will keep advancing file by file.

The Output Preview updates live

(Settings tab > Output Preview)

The Output Preview now refreshes **as you type**. Editing any of the four naming fields (Folder Prefix, Folder Suffix, File Prefix, File Suffix) or the Output folder immediately updates every line in the preview, so you can see exactly how your naming will land without clicking away from the field first. The prefixes and suffixes remain highlighted in gold so it is easy to see what has been added to each name.

(The Input folder is the one field that still updates on commit rather than per keystroke, because changing it triggers the full disk scan described above, which you would not want to run on every character.)

Numeric-aware ("natural") ordering

(Settings tab > Output Preview, and the Queue)

Inputs are now ordered so that embedded numbers sort mathematically, regardless of how many digits they have. Previously a plain text sort placed `Vapor10` immediately after `Vapor1` (before `Vapor2`), because it compared character by character. The list now orders them the way you would expect:

```
Vapor1, Vapor2, Vapor3, … Vapor9, Vapor10, Vapor21, Vapor100
```

This applies to both the Output Preview and the queue you build from it, so the order you see is the order that gets processed. It is case-insensitive and works for any digit count.

Two Reset buttons on the Queue tab

(Queue tab)

The frame-range controls on the Queue tab now provide a **Reset** on both range bars:

- **All jobs range > Reset** returns *every* job to its own originally detected range. This is the quick way to undo range edits across the whole queue in one click — each job goes back to exactly what was detected for it (so different jobs return to their own different ranges, not one shared range).

- **Selected job range > Reset** returns just the currently selected job to its detected range.

Both bars keep their **From / To / Apply** controls exactly as before: type a range and click Apply to set it (on all jobs, or on the selected job). Reset is simply the counterpart that restores the detected values. The button previously labelled "Reset detected" is now just **Reset**.

What carries over from your Write node

(Settings tab > Capture the treatment, clarification)

Worth stating clearly, as it is central to how SCRIPT behaves: when SCRIPT renders each input, the **only** thing it changes on your captured Write node is the output **path** (the folder and filename for that job). Every other Write setting is preserved exactly as you configured it, datatype (e.g. 16-bit half, 32-bit float), compression, colorspace/output transform, metadata options, channels, and so on. The output file format follows your Write's own type as well.

In practice this means you should set your Write up completely (the format and all its options) *before* capturing the treatment. Those settings then travel with the treatment to every job in the queue. The folder and filename you happened to have typed into the Write are ignored; SCRIPT supplies those per job from the Output folder and naming fields.

Part IV - Reference

Note: Almost everything described below is fully managed from Crate´s user interface.
Manual editing of these files is not recommended.

13. Engines - Reference

13.1 Engine Discovery

Engines are discovered from ENGINES_BASE in crate_engines.txt. Platform defaults:

Windows: C:/Users/Public/Crate Image Engines/

macOS: /Users/Shared/Crate Image Engines/

Linux: /usr/local/share/Crate Image Engines/

13.2 Graceful Degradation

No F3D: No 3D model or Gaussian splat thumbnails. Inspect 3D context menu greyed out.

No ImageMagick: No standard-image thumbnails. CONTACT panel unavailable.

No OIIO: Falls back to ImageMagick for EXR/HDR/DPX. IMG-C and SEQ-C use ImageMagick.

No FFmpeg: VID-C panel unavailable. No video thumbnails.

No MediaInfo: Status bar shows limited metadata (filename and size only).

13.3 Worker Limits

Configurable via UI in Crate Settings/Engines and in crate_engines.txt: F3D_MAX_WORKERS, MAGICK_MAX_WORKERS, OIIO_MAX_WORKERS. Controls maximum concurrent subprocess count per engine to prevent system overload during batch thumbnail generation.

14. Configuration Files - Reference

All files use pipe-delimited (|) format. Comments start with #. Empty lines ignored.

14.1 crate_libraries.txt

Format: name | asset_path | cache_path | visible | enabled | type

Types: standard, sequence, video, mixed.

Additional directives (placed after a library line):

FOLDER_OVERRIDE | folder_name | type: Override the type for a specific subfolder.

FOLDER_HIDE | folder_name: Hide a specific subfolder from the grid.

Boolean fields accept: true/yes/1 (enabled) or false/no/0 (disabled).

14.2 crate_discover.txt

Defines Discover project paths and access control. Line types:

SHOW_ENABLED | true/false: Controls whether Visitors see the Discover button. Curators always see it regardless. Default: true.

SHOW_FILE_OPS | true/false: Controls whether Visitors can perform file operations (Rename, New Folder, Cut/Copy/Paste, Delete). Curators always have file ops. Default: false.

SHOW | path | visible: Defines a project folder. Path must exist as a directory. Optional third field controls Visitor visibility (default: true). Crate auto-creates _Crate_Data/Discover_Bin inside each show root.

SHOW_VISITOR | username: Adds a username to the Visitor allowlist. When SHOW_ENABLED is false, only listed Visitors can see Discover. Multiple SHOW_VISITOR lines can be added.

14.3 crate_templates.txt

Defines template source directories. Line types:

TEMPLATES_ENABLED | true/false: Controls whether Visitors see the Templates button. Curators always see it. Default: true.

TEMPLATE | display_name | path | visible | cache_path: Defines a template source. display_name is shown in the UI. path must be an existing directory containing .nk files organised by category subfolder. visible controls whether the source appears (hidden sources are skipped). cache_path is optional, if provided, Crate caches thumbnails there. Each template source auto-creates _Crate_Data/Templates_Bin for safe deletion.

14.3.1 crate_templates_data.txt

Auto-managed by Crate. Do not edit manually. Stores the list of hidden template entries (set via the Hide From Grid context menu action). When a Curator hides a template, its path is appended here and its cached thumbnail is deleted. Hidden templates are excluded from the Visitor view but remain on disk.

14.4 crate_access_credentials.txt

Curator usernames, one per line. Paused: #PAUSED:username. Auto-created by first-launch bootstrap.

14.5 crate_curators_info.txt

Free-form text file displayed as-is in the Curators Contact Info popup (Settings... > Curators Contact Info...). Intended for human-readable contact details. Suggested format per curator: Name, Role,

Email, Slack handle, Phone/Extension, and Notes (availability hours). Comments (#) are not displayed. Accessible to all users, not just Curators.

14.6 crate_engines.txt

Format: KEY | value. Keys: ENGINES_BASE, F3D_MAX_WORKERS, MAGICK_MAX_WORKERS, OIIO_MAX_WORKERS, and per-engine path overrides.

14.7 crate_users.txt

Format: USERS_PATH | /path/to/users/directory. Migrated automatically from crate_engines.txt on first launch if the key existed there.

14.8 crate_nuke.txt

Four boolean flags: VISITOR_MENU_DISCOVER | true, VISITOR_MENU_CART | true, VISITOR_MENU_TEMPLATES | true, IMPORT_BACKDROP | true. All default to true if file is missing.

14.9 crate_hide_prefixes.txt

Format: HIDE_PREFIX | prefix. Folders starting with the prefix are hidden. Default: _ (underscore).

14.10 crate_open_with.txt

Format: application_name | executable_path. Defines entries in the Open With... context menu submenu. Paths are verified on load; unavailable applications show "(not found)" and are greyed out in the menu. When multiple items are selected, labels change to "Application (N items)" and the application is launched once per file. The submenu appears in Library, Cart, Market, Discover, and Templates context menus.

14.11 crate_logs.txt

Two settings: LOG_LEVEL | basic (or dev), CONSOLE_EMOJIS | false (or true).

14.12 Queue Files

Each panel saves queue state to a JSON file in the user's data directory. Writes use a temp file + os.fsync() + os.replace() pattern for crash safety. Queue is restored when the panel re-opens.

15. Nuke Menu Bar Integration - Reference

Crate registers a "Crate" menu in Nuke's top menu bar with the following entries:

⬡ **Launch Crate:** Opens the Crate panel (or brings it to front if already open). Always visible to all users.

Add to Discover: Exports selected Nuke nodes as a .nk script into a Discover project folder. Gated by VISITOR_MENU_DISCOVER.

Add to Cart: Exports selected nodes as a .nk to the user's Cart folder and adds the path to their Cart. Gated by VISITOR_MENU_CART.

Add to Templates: Exports selected nodes as a .nk into a Template category folder. Gated by VISITOR_MENU_TEMPLATES.

All three export actions check the global lock before proceeding. If Crate is locked, a Crate-styled popup appears showing the lock owner and machine, with the message "All Crate operations are suspended until the lock is released." If Crate is not open when an export action is triggered, an info popup says "Open Crate and navigate to a Discover folder first" (or "Open Crate first" for Cart/Templates).

16. F3D Shortcuts - Reference

Available from "Settings.." menu in the toolbar. The Inspect 3D action (right-click context menu) launches F3D as an interactive 3D viewer. Key shortcuts inside F3D:

H: Display cheatsheet of all hotkey interactions.

Enter: Reset the camera to default position.

Space: Play animation (if the file contains animation data).

G: Toggle the horizontal ground grid.

17. Right-Click Context Menus - Reference

Nine context menus, each adapting dynamically to selection count, file type, engine availability, and access mode. Multi-selection labels update (e.g. "Import 5 Items to Nuke"). Actions shown below; Curator-only actions are marked.

17.1 Library Asset Menu

Import to Nuke | Import set |1001| | Import as Cam | (separator) | Open With... | (separator) | Show in Explorer/Finder | Copy Path | (separator) | Add to Cart | Send to Write > [SPLAT Convert, GEO Convert] | (separator) | View Meta

17.2 Cart Item Menu

Import to Nuke | Import set |1001| | Import as Cam | (separator) | Open With... | (separator) | Remove from Cart | Clear Cart (N items) | (separator) | View Meta

17.3 Market Item Menu

Import to Nuke | Import set |1001| | Import as Cam | (separator) | Open With... | (separator) | Show in Explorer/Finder | Copy Path | (separator) | Add to My Cart | (separator) | View Meta

17.4 Discover File Menu

Import to Nuke | Import set |1001| | Import as Cam | Inspect 3D | (separator) | Open With... | (separator) | Show in Explorer/Finder | Copy Path | (separator) | Add to Cart | Send to Write > [EXR, EXR-Queue, MOV, MOV-Queue, SPLAT Convert, GEO Convert, IMG-C, SEQ-C, W-4MAT, VID-C, Batch Rename] | Convert > [...] | Scale > [...] | Sequence to Video > [...] | Extract Frames > [...] | Export Layers > [...] | (separator) | Delete Object [Curator]

17.5 Discover Folder Menu

Import to Nuke | Import set |1001| | (separator) | Open With... | (separator) | Show in Explorer/Finder | Copy Path | (separator) | Rename | New Folder | (separator) | View Meta | Layer Contact Sheet | Send to Write > [EXR, EXR-Queue, MOV, MOV-Queue, SPLAT Convert, GEO Convert, IMG-C, SEQ-C, W-4MAT] | (separator) | Delete Folder [Curator]

17.6 Folder Tree Menu

Set Folder Color > [palette of preset colour dots] | Reset to Default | Reset All to Default

17.7 Detail View Menu

New Folder | Paste (N cut/copied) | Set Folder Color > [...] | (separator) | Import to Nuke | Import set |1001| | Import as Cam | Inspect 3D

17.8 Templates Menu

Import to Nuke | (separator) | Open With... | (separator) | Show in Explorer/Finder | Copy Path | (separator) | Add to Cart | (separator) | View Meta | (separator) | Hide From Grid [Curator]

17.9 Library Folder Menu

Show in Explorer/Finder | Copy Path | Delete Folder / Delete N Folders [Curator] | (separator) | Import to Nuke | Import set |1001| | Send to Write > [EXR, EXR-Queue, MOV, MOV-Queue, SPLAT Convert, GEO Convert, IMG-C, SEQ-C, W-4MAT]

18. Gaussian Splats and the Future - Reference

Gaussian Splats format types notes and future updates:

PLY (standard): The "RAW photo" of splats. This is the original training output from every Gaussian splatting pipeline (Nerfstudio, gsplat, Polycam, Luma, etc.). A typical 500K Gaussian scene averages 118 MB. It stores full per-vertex attributes including all spherical harmonics coefficients for view-dependent colour. Use PLY as your master/archive format, for research, reprocessing, or

further training, raw PLY is the only choice because anything else has lost information you might need. Never distribute PLY directly to clients or web.

PLY (compressed): PlayCanvas/SuperSplat's compressed variant. Offers approximately 4x compression by dropping spherical harmonics data, trading some view-dependent colour accuracy for smaller files. It remains a valid PLY file, which helps interoperability with tools that read standard PLY. Use when you need moderate compression but want to stay within the PLY ecosystem for tool compatibility.

SPLAT (antimatter15): SPLAT files are about 85% smaller than PLY (32 bytes vs ~236 bytes per Gaussian) but much larger than SPZ, and they lose spherical harmonics, your scene will look flat, with no view-dependent lighting. A 500K scene comes to about 16.2 MB (7.3x smaller than PLY). This was one of the earliest compressed formats. Only use this for compatibility with the antimatter15 viewer or legacy web setups. Think of it as the "GIF" of splats, works everywhere but limited quality.

SOG (Spatially Ordered Gaussians): A compact container achieving high compression via quantization (lossy by design), typically yielding files 15–20x smaller than equivalent PLY. Developed by PlayCanvas. It stores splat data in Morton order, meaning it's "GPU-ready" and doesn't require processing on load. It provides around 2–3x the compression of Compressed PLY and crucially supports full spherical harmonics compression, preserving view-dependent colour that SPLAT discards. Use SOG for PlayCanvas/SuperSplat web delivery, especially on memory-constrained mobile devices. It can be a single .sog bundle or an unbundled set of WebP images + meta.json.

SPZ (Niantic): Typically around 10x smaller than corresponding PLY files, with minimal visual differences. Open-sourced by Niantic under MIT. A 500K scene compresses to about 11.8 MB (10x smaller than PLY). SPZ is the current betting favourite for a default delivery format, open spec, Niantic backing, aggressive compression, broad uptake. It preserves spherical harmonics and has been officially adopted into the glTF standard via the KHR_gaussian_splatting_compression_spz extension, which means SPZ blobs can be embedded directly inside glTF assets for standardised web/AR delivery. Use SPZ as your go-to compressed delivery format, it has the broadest industry momentum.

GLB (glTF binary): Not a splat-specific format but the binary container for the glTF 3D standard. Khronos announced a release candidate for the KHR_gaussian_splatting baseline extension, enabling storing 3D Gaussian splats in glTF 2.0. GLB embeds splat data (often SPZ-compressed) inside a standard 3D asset container alongside meshes, materials, and animations. Use GLB when you need splats to coexist with traditional 3D content in a single file, or when your delivery pipeline is built around glTF (Three.js, Babylon.js, AR viewers, geospatial platforms like Cesium). It's the format for standardised interoperability across engines and platforms.

Quick decision guide: Archive in PLY. Deliver via SPZ (broadest support, glTF-standard). Use SOG for PlayCanvas/SuperSplat workflows. Use GLB when splats need to live inside a larger 3D scene. Use compressed PLY for tool-compatible moderate compression. Avoid SPLAT unless you're targeting the legacy antimatter15 viewer.

Future Splat updates in Crate:

SOGS (Self-Organizing Gaussians): This is the new version of SOG, developed by Fraunhofer HHI and adopted by PlayCanvas. It's a groundbreaking compression technique that reorganizes Gaussian data into 2D "attribute images" and compresses them with WebP, achieving over 20x compression (e.g., a 1GB PLY down to 55MB) . PlayCanvas's current recommendation for best runtime performance is to use SOGS .

SPZ Version 4 & 2.0: The SPZ format has seen significant updates. SPZ v4 now uses ZSTD compression with a plaintext header and supports configurable spherical harmonics quantization for flexible quality/size trade-offs, plus support for SH degree 4 . SPZ v2.0.0 introduced a more precise rotation encoding scheme using 10-bit integers for the three smallest quaternion components, which is especially beneficial for long, thin geometry like antennas and fences .

LCC: New format.Recently open-sourced by XGRIDS, the LCC format is designed for large-scale or high-density 3D scenes with a focus on Level-of-Detail (LOD) streaming and cross-platform compatibility, offering significantly smaller files than standard PLY .

HYP: New format.Designed specifically for object representation (as opposed to large environments) and optimized for web apps with limited resources. It provides up to a 15x reduction from PLY (and 2x from SPLAT) using scaled integer representations for positions and scales, and a color palette table for compression .

SplatPB: New format. An academic storage format that achieves 50-60% storage reduction through property-wise organization, delta coding, and lightweight quantization, preserving rendering quality (SSIM ≥ 0.88) . This suggests a focus on efficient storage with minimal quality loss.

19. Automatic File Cleanup - Reference

This chapter documents every process in Crate that automatically deletes files without explicit user action. "Automatic" means Crate deletes the files on its own as part of normal operation, no button press, no confirmation dialog. User-initiated deletions (Regenerate Thumbnail, Defrag Cache, Clear Cart, etc.) are listed separately at the end for completeness.

19.1 Automatic Deletions (No User Action Required)

These happen silently during normal Crate operation. Users do not see a prompt or confirmation.

19.1.1 Thumbnail Staging Orphan Cleanup

Module: `menu.py` - `ThumbnailCache.__init__()`

When it runs: Once per session, at Crate startup, when each library's `ThumbnailCache` is constructed.

What it targets: Files in the local OS temp directory (`{TEMP}/nuke_3d_temp/`) that match the pattern `temp_*.png` or `temp_*.tif` and are older than 1 hour.

Why these files exist: When Crate generates a thumbnail (via F3D, OIIO, ImageMagick, or FFmpeg), the external tool writes to a temporary file in this staging folder first. Crate then atomically moves the finished file to the library's network cache. If Nuke crashes or is force-killed during generation, the staging file is left behind, it never reaches the cache and is now orphaned.

What is deleted: Only orphaned staging files (prefix `temp_`, extensions `.png` or `.tif`) older than 1 hour. This never touches finished thumbnails in any library cache.

Location on disk: `{OS_TEMP}/nuke_3d_temp/`, e.g. `C:\Users\{user}\AppData\Local\Temp\nuke_3d_temp\` on Windows or `/tmp/nuke_3d_temp/` on Linux/macOS.

Risk: None. These files are incomplete intermediates that no part of Crate will ever read. The 1-hour age threshold ensures that any actively-running generation (which typically completes in seconds) is not interrupted.

19.1.2 Contact Sheet Temp Folder Cleanup

Module: `menu.py`, `ThreeDAssetBrowser.__init__()` and contact sheet dialog close handler.

When it runs: Twice:

- At Crate startup, once during browser initialization.
- When the user closes a Contact Sheet dialog.

What it targets: The entire per-user contact sheet temp folder: `{USERS_BASE_DIR}/{username}/temp/contactsheets/`.

Why these files exist: When generating a contact sheet, Crate renders individual tile PNGs to this temp folder, then composites them into the final output. On dialog close, or at next startup if Nuke crashed, the folder is wiped.

What is deleted: The entire `contactsheets` temp folder and all its contents (`shutil.rmtree`). This is a scratch workspace, the final contact sheet output goes to the user's chosen destination, not here.

Location on disk: `{USERS_BASE_DIR}/{username}/temp/contactsheets/`

Risk: None. This folder is purely transient. The final exported contact sheet is saved elsewhere by the user.

19.1.3 Discover Thumbnail Age-Out and Size Cap

Module: `menu.py` - `_discover_thumb_cleanup()` , triggered from `_discover_submit_engine_thumbs()`

When it runs: Once per session, the first time a Discover page submits engine thumbnails (OIIO/F3D/FFmpeg) for generation.

What it targets: Files in the Discover-specific temp folder: `{OS_TEMP}/crate_discover_thumbs/`.

Two-stage cleanup:

1. **Age-out:** Any file whose modification time is older than 7 days (`_DISCOVER_THUMB_MAX_AGE_DAYS = 7`) is deleted unconditionally.
2. **Size cap:** After the age pass, if the remaining files total more than ~750 MB (`_DISCOVER_THUMB_MAX_BYTES = 750 * 1024 * 1024`), files are sorted oldest-first and deleted until the total drops below the cap.

Why these files exist: Discover (Show/Discover mode) generates preview thumbnails for EXR, HDR, DPX, 3D models, and video files using OIIO, F3D, and FFmpeg. These are stored in the OS temp directory (not the library cache) and keyed by file path + modification time + size, so editing the source automatically invalidates the old thumbnail.

What is NOT deleted: This process is completely isolated from the Library thumbnail cache. Library cache directories (configured per-library in `crate_libraries.txt`) are never touched by this cleanup.

Location on disk: `{OS_TEMP}/crate_discover_thumbs/` - e.g. `C:\Users\{user}\AppData\Local\Temp\crate_discover_thumbs\` on Windows.

Risk: Minimal. Discover thumbnails regenerate on demand when a Discover page is browsed. Losing cached thumbnails only means a brief regeneration delay on the next visit.

19.1.4 Pipeline Intermediate Cleanup (Per-Operation)

These are not session-wide sweeps but per-thumbnail-generation cleanups that happen immediately after each operation completes.

Module: `menu.py` - within `try_async_f3d_generation()`, `try_async_oiio_generation()`, `try_async_imagemagick_generation()`, `try_async_sequence_generation()`, `try_async_video_generation()`, and the corresponding Discover engine functions.

What they clean:

- **Temp staging file after successful move:** After a thumbnail is generated to `{TEMP}/nuke_3d_temp/temp_{uuid}.png` and successfully moved to the library cache, the staging file no longer exists (it was moved, not copied). If the move fails because another machine already wrote the same cache file, the staging file is deleted as redundant.
- **Intermediate TIFF files (OIIO DPX pipeline):** The DPX-to-PNG conversion uses a two-step pipeline (DPX > TIFF > PNG). The intermediate TIFF is deleted immediately after the second step completes or fails.
- **Temp .splat files (PLY > Splat conversion for F3D):** When generating a thumbnail for a `.ply` Gaussian Splat file, Crate may convert it to `.splat` format first for F3D rendering. The temp `.splat` file is deleted after F3D finishes.
- **Failed/empty output files (Discover video):** If FFmpeg fails to produce a usable thumbnail (e.g., the codec is unsupported), Crate deletes any 0-byte output file so the next visit retries instead of caching a broken file.

Risk: None. These are intermediate processing artifacts, never the final cached thumbnails.

19.1.5 Atomic Write Temp Files (On Failure)

Module: `menu.py` - `_atomic_write_text()` and similar patterns in `CrateCart._save()`, `_save_templates_data()`, and other config writers.

When it runs: Whenever a config file write fails during the atomic rename step (`os.replace()`).

What it targets: The temp file (`{path}.tmp` or `{path}.{uuid}.tmp`) that was created as part of the atomic write strategy.

Why: Crate writes config files (cart, templates data, etc.) by first writing to a temp file, then atomically renaming it over the original. If the rename fails (e.g., permissions, cross-volume), the temp file is cleaned up before falling back to a direct write.

Risk: None. The temp file contains the same data that will be written directly as a fallback.

19.1.6 Cache Write-Test Probe File

Module: `menu.py` — `ThumbnailCache.__init__()`

When it runs: Once per library, at startup.

What it does: Creates a file `test_write.tmp` in the library's cache directory, writes the word "test", then immediately deletes it. This verifies the cache directory is writable.

Risk: None. The file exists for a fraction of a second.

19.1.7 Lock File Temp Cleanup (On Write Failure)

Module: `crate_cache_lock.py` , `crate_manual_lock.py`

When it runs: If writing a lock file fails during the atomic rename step.

What it targets: The temp lock file created for the atomic write, in the same directory as the target lock file.

Risk: None. Identical to the atomic write pattern above.

19.1.8 Lock File Release

Module: `crate_cache_lock.py` , `crate_manual_lock.py`

When it runs: When Crate finishes a cache write operation (thumbnail generation, regeneration) or when a Curator releases a manual lock.

What it deletes: The `.crate_lock` file in the library cache directory.

Note: This is a normal part of the lock lifecycle, not a cleanup. The lock file is created before a write operation and deleted after it completes. If Nuke crashes during a write, the lock file is left behind, this is what the Defrag scanner detects as a "stale lock."

19.1.9 Conversion Panel Temp Files

Module: `crate_splat_panel.py`, `crate_write_panel.py`, `crate_geo_panel.py`

When it runs: During and after conversion operations (Splat conversion, W-EXR write, Geo conversion).

What they clean: Intermediate temp files created during format conversion:

- **Splat panel:** Temp `.ply` files created during SPZ/PLY conversion, and intermediate qcachepath files.
- **Write panel:** Intermediate qcachepath files used during EXR write operations.
- **Geo panel:** Temp directories used during geometry conversion (`shutil.rmtree`).

These are cleaned up immediately after each operation, whether it succeeds or fails.

Risk: None. The final conversion output goes to the user's chosen destination.

19.1.10 Script Panel Session Files

Module: `discover script panel/crate_script_panel.py`

When it runs: When the user clears a Script Panel session (Reset button), or when the panel is closed with no active session to preserve.

What it deletes: The per-user queue file and temp `.nk` script file used to persist the Script Panel state between sessions. If the user has an active session on close, these files are saved (not deleted); they are only deleted when the session is explicitly cleared or was already empty.

Risk: None. The files are only deleted when there is no session data to lose.

19.2 What Crate Never Auto-Deletes

For clarity, these are explicitly **not** subject to automatic cleanup:

- **Library cache thumbnails** (`{library_cache_path}/*.png`): Never aged out, never size-capped, never automatically cleaned. The docstring in `ThumbnailCache`, claims "7-day expiry for cached thumbnails" but no implementation exists. Once a library thumbnail is generated, it persists indefinitely until a user explicitly regenerates it or runs Defrag.
- **Library cache sequence/video folders** (`{library_cache_path}/{sequence_name}/`): Same as above, never auto-deleted.
- **User data** (carts, GUI preferences, exported scripts, profile images): Never auto-deleted.
- **Configuration files** (`crate_*.txt`): Never auto-deleted.
- **Analytics cache** (the pickled scan data in the user's cache folder): Never auto-deleted.

19.3 User-Initiated Deletions (For Completeness)

These require explicit user action (button click, context menu, confirmation dialog). They are not automatic but are listed here for a complete picture of what Crate can delete.

Action	Module	What It Deletes
Regenerate Thumbnail (single asset)	`menu.py` - `delete_file_thumbnails()`	The specific asset's cached PNG(s), video/sequence subfolder, and memory cache entry.
Regenerate All Thumbnails (library)	`menu.py` - `ThumbnailCache` clear method	All `.png` files in the library's cache directory. Requires cache write lock.
Regenerate Thumbnails (Settings)	`crate_settings_module.py`	All files and subfolders in the library's cache directory (double confirmation required).
Defrag Cache (Analytics)	`crate_graphs.py` - defrag execution	Orphan thumbnails, `.tmp` files, stale lock files (>24h), unknown files, and OS junk (`Thumbs.db`, `.DS_Store`). All with a 24-hour safety floor, files younger than 24 hours are never deleted.
Remove from Cart (owned items)	`menu.py` - `_purge_owned_card_caches()`	Per-user media_cache thumbnails and custom card_thumb image for the removed card. Only for Discover/Nuke-menu-origin cards. Library favourites are unaffected.

Clear Cart	`menu.py` - `_clear_cart_with_confirmation()`	Same as Remove from Cart, applied to all owned items in the cart. Confirmation dialog required.
Remove Template Thumbnail	`menu.py` - template context menu	The cached thumbnail PNG for a specific template `.nk` script.
Remove Card Thumbnail	`menu.py` - cart card context menu	The user's custom card_thumb image for a specific cart card.
Remove Profile Image	`menu.py` - market user context menu	The user's profile image file.
Brightness Adjustment (failure rollback)	`menu.py` - brightness safety finalize	On failure: removes the partial regeneration folder and restores from backup. On success: removes the temp backup folder.
Delete Asset (Curator)	`menu.py` - asset context menu	The asset file itself and its cached thumbnails.

19.4 Summary Table - Automatic Deletions Only

#	What	Where on Disk	Trigger	Age Threshold	Size Cap
1.1	Staging orphans	`{TEMP}/nuke_3d_temp/`	Startup (per library)	1 hour	—
1.2	Contact sheet temps	`{USERS}/*/temp/contact sheets/`	Startup + dialog close	- (full wipe)	—
1.3	Discover thumbs	`{TEMP}/crate_discover_thumbs/`	First Discover engine submission	7 days	750 MB
1.4	Pipeline intermediates	`{TEMP}/nuke_3d_temp/`	Per-operation (immediate)	— (immediate)	—
1.5	Atomic write temps	Same dir as target file	On write failure	— (immediate)	—
1.6	Write-test probe	Library cache dir	Startup (per library)	— (immediate)	—
1.7	Lock file temps	Library cache dir	On lock write failure	— (immediate)	—
1.8	Lock files	Library cache dir	End of write operation	— (lifecycle)	—

1.9	Conversion temps	`{TEMP}/` or operation-specific	Per-operation (immediate)	— (immediate)	—
1.10	Script panel session files	User folder	Reset or empty-session close	— (conditional)	—

None of these processes touch Library cache thumbnails, user data, or configuration files.

20. Advanced ENGINE Deployment - Reference

Overriding the Engines Base per machine with an environment variable

---

Audience: Experienced Pipeline TDs and IT. Requires editing one line region of `menu.py` in your Crate deployment.

Take instructions below as a basic guide for what you would end up implementing. Experiment in sandbox first if Crate is already live in your pipeline.

20.1. What this achieves, and when you want it

Out of the box, every machine reads the same **Engines Base** path from the shared `crate_engines.txt` (set by a Curator in **Crate Settings > Engines**). That is the right default for most studios: one path, engines mirrored to each workstation at that path.

This tutorial adds a small, optional hook so that a machine-level environment variable - **`CRATE_ENGINES_BASE`** - takes priority over the shared value *on that machine only*. With it you can:

- **Scale to hundreds of machines** using the deployment tooling you already have (GPO, SCCM, Ansible, login scripts): push one variable instead of hand-configuring Crate anywhere.
- **Run mixed Windows / Linux / macOS floors**: each OS points at its own engines location, while the shared config stays untouched.
- **Point at a network share** on machines where mirroring isn't wanted.

Machines *without* the variable behave exactly as today. The hook changes nothing for anyone else.

20.2. How it works

Crate resolves all engine executables from a single global, set once at Nuke startup in `menu.py`:

```
engines_base = SYSTEM_SETTINGS.get('ENGINES_BASE')
```

Everything downstream - the per-OS subpaths (`f3d\bin\f3d.exe` on Windows, `f3d/bin/f3d` on Linux/macOS, etc.) and the fallbacks used when individual overrides are empty - derives from that one line. Inserting the override immediately after it therefore covers the entire engine system in one place, with Crate's own per-OS resolution doing the rest.

Precedence after the hook, highest first:

1. **`CRATE_ENGINES_BASE` environment variable** (this machine only)
2. `ENGINES_BASE` from the shared `crate_engines.txt` (set in Crate Settings)
3. Unset > Crate launches normally, thumbnail generation limited (warnings in the console, "Set Engines Base first" in Settings/Engines)

20.3. Step 1 - Add the hook to `menu.py`

Open `menu.py` in your Crate deployment folder and find this line (around line 1124 in v2.0.0; search for the exact text):

```
engines_base = SYSTEM_SETTINGS.get('ENGINES_BASE')
```

Insert directly **below** it:

```
# --- SITE HOOK (optional): per-machine Engines Base override ----------

# If CRATE_ENGINES_BASE is set in the environment, it wins over the shared

# crate_engines.txt value ON THIS MACHINE ONLY. Lets IT deploy engines via

# env vars (GPO / profile.d / init.py) instead of per-machine Crate config,

# and supports mixed-OS floors. Unset or empty variable = no change.

_env_engines_base = os.environ.get("CRATE_ENGINES_BASE", "").strip()

if _env_engines_base:

    _crate_log(f"Engines: base overridden by CRATE_ENGINES_BASE -> {_env_engines_base}")

    engines_base = _env_engines_base

# ----------------------------------------------------------------------
```

That is the entire code change. Save, done. (The hook is inert until the variable exists, so it is safe to ship in your deployment permanently.)

20.4. Step 2 - Prepare the engines folder(s)

Crate expects this layout under whatever base it is given. Windows:

```
<base>\f3d\bin\f3d.exe

<base>\imagemagick\magick.exe

<base>\ffmpeg\bin\ffmpeg.exe

<base>\mediainfo\MediaInfo.exe
```

<base>\OpenImageIO\oiiotool.exe

<base>\splat-transform\node_modules\@playcanvas\splat-transform\ (directory)

Linux / macOS:

<base>/f3d/bin/f3d

<base>/imagemagick/magick

<base>/ffmpeg/bin/ffmpeg

<base>/mediainfo/mediainfo

<base>/OpenImageIO/oiiotool

<base>/splat-transform/node_modules/@playcanvas/splat-transform/ (directory)

All **six** engines resolve from the base. Note the sixth is different in kind: Splat Transform is an **npm package directory**, not an executable - create it once per engines folder with:

cd <base>/splat-transform

npm install @playcanvas/splat-transform

(the `node_modules/@playcanvas/splat-transform` path above is what npm produces; the layout is identical on every OS). When mirroring engines to workstations, mirror this directory tree like any other - npm is only needed when *building* the master engines folder, never on the seats.

Build one folder per OS you support and distribute it however you already distribute software (or host it on a share - see recipes below).

20.5. Step 3 - Set the variable

Pick ONE of these per machine/fleet. The variable must exist **before Nuke starts**.

Windows (fleet, recommended): push a *system* environment variable via GPO / SCCM / Intune: `CRATE_ENGINES_BASE = C:\CrateEngines` Manually on one machine (admin prompt): `setx /M CRATE_ENGINES_BASE "C:\CrateEngines"` (new processes only - restart Nuke).

Linux (fleet): drop a profile script, e.g. `/etc/profile.d/crate.sh`:

export CRATE_ENGINES_BASE="/opt/crate-engines"

Or set it in the wrapper/launcher script your studio uses to start Nuke - often the cleanest place, since it guarantees ordering.

macOS: set it in your Nuke launcher script, or via `launchctl setenv` in a LaunchAgent for GUI-launched Nuke.

Any OS, via Nuke itself (no OS config at all): add to an `init.py` on the `NUKE_PATH` (init.py runs before `menu.py`, so the ordering is guaranteed):

```
import os

os.environ["CRATE_ENGINES_BASE"] = r"C:\CrateEngines"   # per-OS value here
```

This variant is handy when the same `init.py` already branches per platform:

```
import os, platform

os.environ["CRATE_ENGINES_BASE"] = (

    r"C:\CrateEngines" if platform.system() == "Windows"

    else "/opt/crate-engines")
```

20.6. Step 4 - Verify

1. Restart Nuke on the machine.
2. The Nuke script editor / console shows: `Engines: base overridden by CRATE_ENGINES_BASE -> <your path>`
3. (Curators) **Crate Settings > Engines**: the individual overrides show green **Ready**.
4. (Anyone) **Settings menu > Show Crate Status...** lists the resolved engine paths - they should point under your env-var base.
5. Browse to any 3D asset or video and confirm thumbnails generate.

20.7. Deployment recipes

A - Local mirror at scale (the 300-machine case). Keep engines in your software repo; sync them to `C:\CrateEngines` (or `/opt/crate-engines`) with the deployment tool you already run, and push the variable with the same tool. Crate itself never copies anything - distribution stays where it belongs, in IT's tooling, with AV exclusions and permissions handled there.

B - Network share, zero mirroring. Point the variable (or simply the shared Settings value, if the whole floor is one OS) at a share: `\\server\pipeline\crate-engines` or `/mnt/pipeline/crate-engines`. Trade-offs are IT's call: slower process spawn over the wire, and some AV products dislike executing from shares. Test with one seat first.

C - Mixed OS floor. Shared `crate_engines.txt` keeps whichever notation your Curators use (it becomes the default for that OS); every *other* OS gets the variable:

Machines	Mechanism	Value
Windows floor	shared Settings value	`L:\Pipeline\CrateEngines-win`
Linux floor	`/etc/profile.d/crate.sh`	`/mnt/pipeline/crate-engines-linux`
macOS seats	Nuke launcher script	`/Volumes/pipeline/crate-engines-mac`

Note: library **asset/cache paths** in `crate_libraries.txt` remain single-notation in v2.0.0 (see Known Limitations) - mixed-OS support here covers the engines layer.

20.8. Notes, gotchas, rollback

- **Scope:** the variable is strictly per-machine. It never writes to, or changes, the shared `crate_engines.txt`.
- **Settings UI display:** Crate Settings always shows the *shared* value in the Engines Base field - on an env-overridden machine the field and the actual base can legitimately differ. "Show Crate Status..." shows the truth for the running session.
- **Restart popup:** if a Curator edits and saves Engines Base on an env-overridden machine, the "Restart Required" popup may appear; after restarting, the environment variable still wins on that machine. Expected.
- **Empty variable:** an empty/whitespace `CRATE_ENGINES_BASE` is ignored (falls through to the shared value) - safe to template.
- **Rollback:** unset the variable and restart Nuke > shared value applies again. Removing the hook itself = delete the inserted block; the surrounding code is untouched.

20.9. Testing full centralization on a single server

A popular deployment goal: one server (bare metal or VM - Crate cannot tell the difference; only share throughput and IOPS matter) hosting everything. Be precise about what "everything" means here, because one third of it is already Crate's native design and one third is not possible:

Cache files - already central, by design. Every library's cache path points at a share; thumbnails, folder-card sidecars, lock files and the config txts all live server-side today. Nothing to enable.

Engine binaries - central via Recipe B. Point the Engines Base at the server share. On a single-OS floor the shared Settings value alone does it; the CRATE_ENGINES_BASE hook from this tutorial is what makes the same server work on a mixed floor, where each OS needs its own mount notation for it (\\server\engines-win, /mnt/pipeline/engines-linux).

Generation compute - never central. Workstations always spawn the engine processes locally: with server-hosted engines each seat reads the binaries over the wire and writes cache files back over the wire, but the CPU work stays at the seat. A "server generates, seats consume" model would be a different architecture (a generation service), not a configuration of this hook.

With that understood, test the achievable configuration in this order:

One seat, cold. Set the base to the server path, restart Nuke, confirm all six engines show green Ready in Settings/Engines and in "Show Crate Status...". Generate a thumbnail of each family - image, EXR, video, 3D model, sequence, and a splat conversion - and note the first-spawn latency: executing from SMB/NFS is measurably slower than local, and this single number is your main go/no-go signal.

Windows execution policy. Some AV / SmartScreen configurations block or delay executables launched from UNC paths. Test on a seat with your real security stack, not an admin's clean machine.

Concurrency. Have several seats regenerate thumbnails for some libraries simultaneously by just a first time visit. Expected behavior: no corruption (cache writes are last-write-wins with atomic patterns), but watch the server's NIC and disk - effective load is seats × Max Workers engine processes reading binaries and writing PNGs against one box. If the server saturates, lower Max Workers in Settings/Engines.

Server-down drill. Unmount or firewall the share and launch Nuke: Crate must still boot (engines report "Not found", thumbnails simply don't generate, cached ones may be unavailable) - degraded, never crashed. Restore the share, restart, confirm recovery.

Scale rehearsal. Before flipping 300 seats, run steps 1–4 with the largest pilot group you can (10–20 seats) and extrapolate the server load linearly - it genuinely is close to linear.

Rule of thumb from the trade-offs above: centralize storage and binaries freely; if step 1's spawn latency or step 3's saturation disappoint, fall back to Recipe A (local engine mirrors pushed by IT) while keeping cache and config on the server - that hybrid is the sweet spot for most floors.

20.10. Final Notes on Centralization

If your studio has 5 to 20 comp seats, and you have fast high quality hardware network NIC´s (1gbe to 10gbe) both on server and seats, cat6 cabling and above, and a powerful server, you might want to just point the Engines Base path to that server and have the Engines centralized instead per seat/workstation. Try this setup for a few days, checking the speeds and thumbnail generations as you visit the libraries. Once your libraries have a high thumbnail % coverage, you won't be using the engines as much on the Library module.

21. Special Thanks

Crate is built on the shoulders of extraordinary open-source projects, people and communities. Without them, none of this would exist.

Engines

F3D (f3d.app): The 3D viewer that gave Crate its defining capability: instant thumbnails from geometry and Gaussian splats, straight from the file system. BSD 3-Clause.

OpenImageIO by the Academy Software Foundation (openimageio.org): Colour-managed image processing done right. Every EXR, HDR, and DPX thumbnail Crate produces with OCIO fidelity is thanks to this project. Apache 2.0.

FFmpeg (ffmpeg.org): The engine behind every video thumbnail, every transcode, every frame extraction. LGPL v2.1+.

ImageMagick (imagemagick.org): Reliable image conversion and the montage command that powers Crate's contact sheets. ImageMagick License.

MediaInfo by MediaArea (mediaarea.net): Clean, fast metadata extraction that makes the status bar useful. BSD 2-Clause.

PlayCanvas splat-transform (github.com/playcanvas/splat-transform): The Gaussian splat format converter that powers the SPLAT-C panel. MIT.

Node.js (nodejs.org): The runtime behind splat-transform.

Camera Database

Tony D'Agostino and VFXCamDB (vfxcamdb.com): The foundational community reference for digital cinema sensor specifications. Crate's Camera Database would not exist in its current form without his work.

Matchmove Machine (camdb.matchmovemachine.com): A vital second source of verified sensor data. Their database, alongside VFXCamDB, ensures Crate ships with accurate film back values for hundreds of cameras.

CINEd Lens Database (cined.com/lens-database): Linked from Crate's interface for lens reference.

Included Code

Kevin Kwok / antimatter15 (github.com/antimatter15/splat): The original WebGL 3D Gaussian Splat Viewer whose convert.py script is the basis of Crate's PLY-to-SPLAT converter. MIT.

Host Application

The Foundry: Nuke, the compositing application that Crate lives in. Crate uses Nuke's Python API and Qt bindings (PySide2 / PySide6).

Qt for Python (qt.io): The UI framework, provided by Nuke, that every button, grid, dropdown, and popup in Crate is built with.

Recommended Companion Tools

The open-source and freeware community that gives artists the viewers and utilities linked through Crate's Open With menu: Open RV (Academy Software Foundation), DJV, VLC, tev, Notepad++, XnView MP, Bulk Rename Utility, LizardQ Pano Viewer, and SuperSplat by PlayCanvas.

---

22. Crate Bugs/Issues/Tickets

Any report and suggestion to improve Crate is welcome

https://github.com/CrateTools/Crate/issues

---

23. Crate on Linux and Mac

Crate on Linux and macOS

Crate is developed and tested on Windows. The codebase is cross-platform by design, PySide2/PySide6 detection, platform-aware engine paths, and standard Python file operations, but Linux and macOS have not been through the same exhaustive testing. File permissions, executable flags, and engine binary availability may require adjustment.

We welcome developers and studios that take on this challenge;

we are happy to support this initiative and would love to

collaborate with you.

Known challenges for porters:

Case-sensitive file systems: Linux file systems are case-sensitive. Several path comparisons in Crate use `.lower()` for normalisation, which is correct on Windows (case-insensitive NTFS) but could cause missed matches on ext4/XFS where `Render_v01.exr` and `render_v01.exr` are different files.

OpenImageIO shared library resolution: The Windows build loads DLLs from an adjacent `bin/` subfolder via PATH injection. On Linux this becomes LD_LIBRARY_PATH, on macOS DYLD_LIBRARY_PATH, same concept, different environment variable and potentially different build structures.

Engine binary permissions: Windows executables run without special flags. On Linux and macOS, every engine binary (f3d, magick, ffmpeg, oiiotool, mediainfo, node) needs the executable flag set (`chmod +x`). A fresh download or network mount may not have this.

Network path conventions: Crate's multi-user deployment relies heavily on UNC paths (`\\Server\share\path`) for libraries, caches, and user data. Linux and macOS use POSIX mount points (`/mnt/server/share` or `/Volumes/share`). The path normalisation function includes drive-letter casing fixes (`c:\ > C:\`) that would need to be bypassed on non-Windows systems.

macOS Gatekeeper and quarantine: Downloaded engine binaries on macOS are quarantined by default. Each binary may need `xattr -d com.apple.quarantine` before Crate's subprocess calls can execute them, or the entire engines folder must be explicitly approved in Security & Privacy.

Qt rendering differences: Fixed-width pixel values, font metrics, and widget sizing may render differently across Qt backends (xcb on Linux, Cocoa on macOS, Windows native).

The gold-themed UI should work but padding, alignment, and scrollbar widths may need visual tuning.

24. Crate Flight Recorder

Crate includes a built-in flight recorder that can silently log activity in the background while you work. If Nuke ever crashes or behaves unexpectedly, these logs help the development team diagnose what happened.

Enabling and disabling

The flight recorder is disabled by default. To enable it, open `menu.py` in a text editor, go to **line *30418***, and change `CRATE_FR_ENABLED = False` to `CRATE_FR_ENABLED = True`. Save the file and restart Nuke. Crate will confirm the status in Nuke's Script Editor on launch. When finished collecting logs, set it back to `False` and restart Nuke.

Where to find the logs

Logs are saved automatically to a folder called `crate_flight_recorder_logs` inside your Crate installation directory. Each log file is named with the date, time, your username, and machine name, for example:

```
flight_20260812_174804_jane_RENDER-PC03.log
```

If a crash occurs, a companion file ending in `_fault.log` may also appear. This file contains a detailed snapshot of what was running at the exact moment of the crash.

What to do if Nuke crashes

1. Navigate to the `crate_flight_recorder_logs` folder.
2. Find the most recent `.log` and `_fault.log` files (sort by date).

The logs contain only Crate activity, button clicks, folder navigation, thumbnail generation progress, and system status. They do not contain personal data, file contents, or Nuke project information.

Housekeeping

The flight recorder keeps only the 10 most recent log files and automatically deletes older ones. No manual cleanup is needed.

25. License by file

menu.py Apache-2.0

crate_4mat_panel.py Apache-2.0

crate_analytics_network.py Apache-2.0

crate_batch_panel.py Apache-2.0

crate_cache_lock.py Apache-2.0

crate_contact_panel.py Apache-2.0

crate_convert.py Apache-2.0

crate_geo_panel.py Apache-2.0

crate_graphs.py Apache-2.0

crate_imgc_panel.py Apache-2.0

crate_manual_lock.py Apache-2.0

crate_ply_to_splat.py Apache-2.0 (contains code based on antimatter15/splat, MIT - attribution preserved in file)

crate_ply_to_ksplat.py Apache-2.0 (reimplements logic from francescofugazzi/3dgsconverter, MIT - attribution preserved in file)

crate_seqc_panel.py Apache-2.0

crate_splat_panel.py Apache-2.0

crate_vidc_panel.py Apache-2.0

crate_write_panel.py Apache-2.0

crate_settings/crate_settings_module.py Apache-2.0

discover script panel/crate_script_panel.py Apache-2.0

discover script panel/crate_script_engine.py Apache-2.0

discover script panel/crate_script_analyze.py Apache-2.0

crate_camera_database/crate_camdb.py Apache-2.0

crate_camera_database/crate_cam_export.py Apache-2.0

crate_camera_database/crate_cameras.json CC-BY-4.0 (CrateCamDB dataset - reusable outside Crate with attribution; see CrateCamDB LICENSE.txt / NOTICE.txt)

Configuration files (crate_*.txt) Not licensed works - your deployment's own data

External engines (F3D, ImageMagick, OIIO, MediaInfo, FFmpeg, splat-transform) Not distributed with Crate - each governed by its own license

Full license texts ship in the crate_licence_info_about/ folder: Apache-2.0 (LICENSE + NOTICE), CC-BY-4.0 for CrateCamDB, and the third-party MIT licenses for splat, 3dgsconverter, and PlayCanvas splat-transform.

THANK YOU FOR USING CRATE

CrateTools 2026